C언어 자료구조와 알고리즘

- 문장별 근거 · 인쇄 점검

목차

검증 범위 3
01 자료구조·알고리즘·추상 자료형 4
02 C언어 메모리·배열·구조체·포인터 8
03 알고리즘 성능 분석 13
04 순환과 재귀 18
05 연결 리스트 23
06 스택·수식·미로 28
07 큐·덱·시뮬레이션 33
08 트리·순회·트리 응용 38
09 이진 탐색 트리·균형 트리 44
10 우선순위 큐·힙·허프만 53
11 그래프·순회·위상 정렬·분리 집합 59
12 최소 비용 신장 트리·최단 경로 65
13 정렬 71
14 탐색 78
15 해싱·해시 테이블 83
16 문자열 탐색 90
17 분할 정복 95
18 동적 계획법 100
19 탐욕 알고리즘 105
20 백트래킹 110

검증 범위

중복을 합친 934개 항목의 문장·코드·수식과 원문 근거를 대조할 수 있습니다. 번호는 학습 노트·치트시트와 같습니다. 출처는 파일 코드와 원문 절·인쇄 쪽수·PDF 면 번호를 구분해 표시했습니다.

A는「(C언어)자료구조 (1).pdf」, B는「C언어_자료구조_알고리즘.pdf」입니다. 인쇄 쪽수는 구간별로 검증한 쪽수 대응표를 적용했습니다. 근거 대조 문구는 원문 발췌 또는 원문을 줄인 확인 문구입니다. 인용한 관련 면은 원본 스캔 이미지와 대조했습니다.

인쇄 품질 점검

파일	점검 면	잘림·넘침	폰트	제목 고립	표 분리
학습 노트	1-144	이상 없음	이상 없음	이상 없음	분리 없음
확인·시험	1-22	이상 없음	이상 없음	이상 없음	분리 없음
근거·점검	1-114	이상 없음	이상 없음	이상 없음	분리 없음

최종 PDF의 모든 페이지를 이미지로 변환해 확인했습니다. 좌표·글꼴 포함·대체 글리프·제목·표 배치 검사 결과를 함께 대조했습니다.

01 자료구조·알고리즘·추상 자료형

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-001	자료들을 정리하여 보관하는 여러 가지 구조를 자료구조(data structure)라고 부른다. 근거 대조 문구: 자료들을 정리하여 보관하는	(A 1.1 자료구조와 알고리즘, 인쇄 p.12 / PDF p.7)
U01-002	추상 자료형(ADT: abstract data type)은 추상적·수학적으로 자료형을 정의한 것이다. 근거 대조 문구: 수학적으로 자료형을 정의	(A 1.2 추상 자료형, 인쇄 p.18 / PDF p.13)
U01-003	알고리즘은 컴퓨터로 문제를 풀기 위한 단계적인 절차이다. 근거 대조 문구: 컴퓨터로 문제를 풀기 위한 단계적인 절차	(A 1.1 알고리즘이란?, 인쇄 p.15 / PDF p.10)
U01-004	알고리즘은 특정한 일을 수행하는 명령어들의 집합이며, 알고리즘이 되기 위한 조건을 만족해야 한다. 근거 대조 문구: 특정한 일을 수행하는 명령어들의 집합	(A 1.1 알고리즘이란?, 인쇄 p.15 / PDF p.10)
U01-005	최고 성적 예제에서 배열 scores는 자료구조이고, largest를 첫 요소로 초기화하여 나머지 요소들과 순차 비교하는 절차는 알고리즘이다. 근거 대조 문구: 배열 scores가 자료구조	(A 1.1 프로그램 1.1, 인쇄 p.14 / PDF p.9)
U01-006	자료구조와 알고리즘은 밀접한 관계가 있어, 각 응용에 적합한 자료구조와 알고리즘을 선택해야 한다. 근거 대조 문구: 가장 적합한 자료	(A 1.1 자료구조와 알고리즘, 인쇄 p.14 / PDF p.9)
U01-007	B는 자료구조를 데이터를 효율적으로 다루게 돕는 데이터 보관 방법과 데이터에 관한 연산의 총체로 정의한다. 근거 대조 문구: 데이터 보관	(B 0.1 자료구조, 인쇄 p.3 / PDF p.14)
U01-008	알고리즘 설계는 문제 풀이 절차를 설계하는 것이고, 구현은 프로그래밍 언어로 실제 동작하는 코드를 작성하는 것이다. 근거 대조 문구: 알고리즘을 설계한다는 것은	(B 0.2 알고리즘, 인쇄 p.6 / PDF p.17)
U01-009	입력 조건은 0개 이상의 입력이 존재해야 한다는 것이다. 근거 대조 문구: 0개 이상의 입력	(A 1.1 정의 1.1 알고리즘, 인쇄 p.15 / PDF p.10)
U01-010	출력 조건은 1개 이상의 출력이 존재해야 한다는 것이다. 근거 대조 문구: 1개 이상의 출력	(A 1.1 정의 1.1 알고리즘, 인쇄 p.15 / PDF p.10)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-011	명백성은 각 명령어의 의미가 모호하지 않고 명확해야 한다는 것이다. 근거 대조 문구: 모호하지 않고 명확	(A 1.1 정의 1.1 알고리즘, 인쇄 p.15 / PDF p.10)
U01-012	유한성은 한정된 수의 단계 후에 반드시 종료되어야 한다는 것이다. 근거 대조 문구: 한정된 수의 단계	(A 1.1 정의 1.1 알고리즘, 인쇄 p.15 / PDF p.10)
U01-013	유효성은 각 명령어가 종이와 연필 또는 컴퓨터로 실행 가능한 연산이어야 한다는 것이다. 근거 대조 문구: 실행 가능한 연산	(A 1.1 정의 1.1 알고리즘, 인쇄 p.15 / PDF p.10)
U01-014	알고리즘 기술 방법에는 자연어, 흐름도(flowchart), 의사 코드, 프로그래밍 언어가 있다. 근거 대조 문구: 4가지의 방법	(A 1.1 알고리즘의 기술 방법, 인쇄 p.16 / PDF p.11)
U01-015	의사 코드는 자연어보다 체계적이고 프로그래밍 언어보다 덜 엄격하며, 알고리즘을 기술하는 데 사용하는 코드이다. 근거 대조 문구: 자연어보다는 더 체계적	(A 1.1 알고리즘의 기술 방법, 인쇄 p.16 / PDF p.11)
U01-016	자료형(data type)은 데이터의 종류이며, 자료형을 다룰 때에는 데이터 간에 가능한 연산도 고려한다. 근거 대조 문구: 데이터 간에 가능한 연산	(A 1.2 추상 자료형, 인쇄 p.17-18 / PDF p.12-13)
U01-017	자료구조는 추상 자료형을 프로그래밍 언어로 구현한 것이라 할 수 있다. 근거 대조 문구: 프로그래밍 언어로 구현	(A 1.2 추상 자료형, 인쇄 p.18 / PDF p.13)
U01-018	추상화는 시스템의 간략한 기술 또는 명세이며, 핵심적인 구조나 동작에 집중한다. 근거 대조 문구: 핵심적인 구조나 동작	(A 1.2 추상 자료형, 인쇄 p.19 / PDF p.14)
U01-019	ADT는 데이터나 연산이 무엇(what)인지 정의하며, 어떻게(how) 구현할 것인지는 정의하지 않는다. 근거 대조 문구: 무엇(what)	(A 1.2 추상 자료형, 인쇄 p.19 / PDF p.14)
U01-020	ADT의 연산에는 이름·매개변수·반환형을 정의하지만 구체적인 구현 코드는 주어지지 않는다. 근거 대조 문구: 매개 변수, 반환형	(A 1.2 추상 자료형, 인쇄 p.19 / PDF p.14)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-021	ADT의 사용자는 구현 세부사항이 아닌 인터페이스를 사용하므로, 인터페이스가 지켜지면 구현 방법을 변경할 수 있다. 근거 대조 문구: 인터페이스만을 공개	(A 1.2 정보은닉, 인쇄 p.20 / PDF p.15)
U01-022	B의 추상 데이터 형식(ADT)은 자료구조의 동작 방법을 표현하며, 자료구조가 갖추어야 할 일련의 연산이다. 근거 대조 문구: 자료구조가 갖춰야 할 일련의 연산	(B 0.1 추상 데이터 형식, 인쇄 p.4 / PDF p.15)
U01-023	B는 ADT가 개념을 제시하고 자료구조가 구현을 포함한다고 구분한다. 근거 대조 문구: ADT는 개념을 제시	(B 0.1 자료구조, 인쇄 p.5 / PDF p.16)
U01-024	단순 자료구조는 프로그래밍 언어에서 통상 제공하는 기본 데이터 형식이며, B는 복합 자료구조를 설명한다. 근거 대조 문구: 기본 데이터 형식	(B 0.1 자료구조, 인쇄 p.3 / PDF p.14)
U01-025	선형 자료구조는 데이터 요소를 순차적으로 연결하며, 배열·링크드 리스트·스택·큐 등이 해당한다. 근거 대조 문구: 데이터 요소를 순차적으로 연결	(B 0.1 자료구조, 인쇄 p.3 / PDF p.14)
U01-026	비선형 자료구조는 데이터 요소를 비순차적으로 연결하며, 트리와 그래프가 해당한다. 근거 대조 문구: 비순차적으로 연결	(B 0.1 자료구조, 인쇄 p.4 / PDF p.15)
U01-027	첫 번째 배열 요소를 largest에 저장한다. 근거 대조 문구: largest	(A 1.1 ArrayMax, 인쇄 p.14,17 / PDF p.9,12)
U01-028	나머지 요소를 차례대로 비교하여 현재 요소가 더 크면 largest를 갱신한다. 근거 대조 문구: return largest	(A 1.1 ArrayMax, 인쇄 p.14,17 / PDF p.9,12)
U01-029	배열 끝까지 비교한 뒤 largest를 반환한다. 근거 대조 문구: return largest	(A 1.1 ArrayMax, 인쇄 p.14,17 / PDF p.9,12)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-030	알고리즘 1.1 ArrayMax 원문 의사 코드 (원문 코드 발췌) ArrayMax(list, N): largest←list[0] for i←1 to N-1 do if list[i]>largest then largest←list[i] return largest 근거 대조 문구: ArrayMax	(A 1.1 알고리즘 1.1, 인쇄 p.17 / PDF p.12)
U01-031	그림 1-3의 프로그램 구성: 프로그램 = 자료구조 + 알고리즘; 원문 그림의 표기. 근거 대조 문구: 프로그램 = 자료구조 + 알고리즘	(A 1.1 프로그램 1.1, 인쇄 p.14 / PDF p.9)
U01-032	추상 자료형 근거 대조 문구: ADT	(A 1.2 추상 자료형, 인쇄 p.19 / PDF p.14)
U01-033	자료구조 근거 대조 문구: 자료구조	(A 1.2 추상 자료형, 인쇄 p.18 / PDF p.13)
U01-034	입력은 0개 이상, 출력은 1개 이상이어야 한다. 근거 대조 문구: 0개 이상의 입력	(A 1.1 정의 1.1, 인쇄 p.15 / PDF p.10)

02 C언어 메모리·배열·구조체·포인터

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-001	배열은 동일한 타입의 데이터를 여러 개 만들 때 사용하며, 연속적인 메모리 공간에 인덱스로 접근한다. 근거 대조 문구: 동일한 타입의 데이터	(A 3.1 배열, 인쇄 p.70 / PDF p.63)
U02-002	동적 메모리 할당은 필요한 메모리를 운영체제로부터 할당받아 사용하고, 사용이 끝나면 시스템에 반납하는 기능이다. 근거 대조 문구: 필요한 만큼의 메모리	(A 3.6 동적 메모리 할당, 인쇄 p.95 / PDF p.88)
U02-003	배열 ADT의 객체는 <인덱스, 값> 쌍의 집합이며, 수학적으로 인덱스에서 값으로의 사상(mapping)이다. 근거 대조 문구: 사상(mapping)	(A 3.1 배열 ADT, 인쇄 p.70-71 / PDF p.63-64)
U02-004	배열 ADT의 create(size)는 배열 생성, get(A,i)는 i번째 요소 반환, set(A,i,v)는 i번째 위치에 값 v 저장이다. 근거 대조 문구: create(size)	(A 3.1 ADT 3.1 Array, 인쇄 p.71 / PDF p.64)
U02-005	C언어에서 배열 인덱스는 0부터 시작하며, [ ] 연산자로 값을 가져오거나 저장한다. 근거 대조 문구: 인덱스는 0부터 시작	(A 3.1 C언어에서의 1차원 배열, 인쇄 p.71 / PDF p.64)
U02-006	2차원 배열에서 가로줄은 행(row), 세로줄은 열(column)이며, C언어는 배열의 배열로 구현한다. 근거 대조 문구: 배열의 배열	(A 3.1 2차원 배열, 인쇄 p.72 / PDF p.65)
U02-007	배열이 같은 타입의 데이터 모임이라면 구조체(structure)는 타입이 다른 데이터를 묶는 방법이며 struct로 표기한다. 근거 대조 문구: 타입이 다른 데이터를 묶는	(A 3.2 구조체, 인쇄 p.73 / PDF p.66)
U02-008	typedef로 구조체의 별칭을 만들면 struct 키워드 없이 그 이름으로 변수나 포인터를 선언할 수 있다. 근거 대조 문구: typedef 키워드를 이용해서 별칭	(B 0.3.2 구조체 복습, 인쇄 p.10 / PDF p.21)
U02-009	구조체 멤버에는 .으로 접근하고, 포인터를 통해 멤버에 접근할 때에는 -> 연산자를 사용한다. 근거 대조 문구: 멤버 접근 연산자	(B 0.3.2 구조체 복습, 인쇄 p.9-10 / PDF p.20-21)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-010	다항식의 첫 번째 표현은 모든 차수의 계수를 coef 배열에 저장하고 차수를 degree에 저장한다. 근거 대조 문구: coef 배열	(A 3.3 배열의 응용: 다항식, 인쇄 p.76 / PDF p.69)
U02-011	두 번째 표현은 0이 아닌 항만 (계수, 차수) 형식의 구조체 배열에 저장하고 여러 다항식을 함께 보관한다. 근거 대조 문구: 0이 아닌 항	(A 3.3 두 번째 방법, 인쇄 p.79 / PDF p.72)
U02-012	두 번째 방식은 각 다항식의 시작·끝 인덱스를 관리해야 하고 차수도 저장하므로, 경우에 따라 첫 방식보다 공간이 더 필요할 수 있다. 근거 대조 문구: 인덱스 변수들을 관리	(A 3.3 두 번째 방법, 인쇄 p.79 / PDF p.72)
U02-013	희소행렬은 많은 항이 0이며, 2차원 배열로 표현하면 메모리 낭비가 커질 수 있다. 근거 대조 문구: 메모리의 낭비	(A 3.4 배열의 응용: 희소행렬, 인쇄 p.84 / PDF p.77)
U02-014	희소행렬의 다른 표현은 0이 아닌 요소만 (행, 열, 값)으로 나타내는 것이다. 근거 대조 문구: 열, 값)	(A 3.4 배열의 응용: 희소행렬, 인쇄 p.84 / PDF p.77)
U02-015	전치행렬은 원래 행렬의 행과 열을 바꾸며, 원문의 배열 코드는 B[c][r] = A[r][c]로 저장한다. 근거 대조 문구: 행릴 전치 함수	(A 3.4 전치 행렬 계산하기 #1, 인쇄 p.85 / PDF p.78)
U02-016	희소행렬의 전치 결과를 행 순서로 저장하기 위해 원본의 낮은 열부터 차례대로 처리한다. 근거 대조 문구: 0열을 먼저 처리	(A 3.4 알고리즘 3.1, 인쇄 p.87 / PDF p.80)
U02-017	포인터(pointer)는 다른 변수의 주소를 가지고 있는 변수이다. 근거 대조 문구: 다른 변수의 주소	(A 3.5 포인터, 인쇄 p.89 / PDF p.82)
U02-018	&는 변수의 주소를 추출하는 주소 연산자이고, *는 포인터를 통한 간접참조(역참조) 연산자이다. 근거 대조 문구: 주소 연산자	(A 3.5 포인터와 관련된 연산자, 인쇄 p.90 / PDF p.83)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-019	포인터 p가 a를 가리키면 *p와 a는 같은 실제 객체를 참조하므로 *p를 변경하면 a도 바뀐다. 근거 대조 문구: 동일한 실제적인 객체	(A 3.5 포인터와 관련된 연산자, 인쇄 p.90 / PDF p.83)
U02-020	널 포인터는 어떤 객체도 가리키지 않는 포인터이며, C언어에서는 일반적으로 NULL 매크로로 표시한다. 근거 대조 문구: 어떤 객체도 가리	(A 3.5 널 포인터, 인쇄 p.91 / PDF p.84)
U02-021	특정 변수를 가리키는 포인터를 함수 매개변수로 전달하면 그 포인터로 외부 변수의 값을 변경할 수 있다. 근거 대조 문구: 외부 변수의 값을 변경	(A 3.5 함수 매개변수로 포인터 사용하기, 인쇄 p.91 / PDF p.84)
U02-022	배열을 함수 매개변수로 전달하면 배열 시작 주소의 포인터가 전달되어, 함수 안의 변경이 원본 배열에 반영된다. 근거 대조 문구: 포인터가 전달되는	(A 3.5 배열과 포인터, 인쇄 p.92 / PDF p.85)
U02-023	원문의 int 배열에서 첫 요소를 가리키는 포인터 ptr에 ++를 적용하면 int의 크기만큼 주소가 증가하여 다음 요소를 가리킨다. 근거 대조 문구: ptr은 int의 크기만큼 더해진 주소값	(B 0.3.1 포인터 복습, 인쇄 p.8 / PDF p.19)
U02-024	동적 메모리가 할당되는 공간을 히프(heap)라고 한다. 근거 대조 문구: 동적 메모리가 할당되는 공간	(A 3.6 히프(heap), 인쇄 p.95 / PDF p.88)
U02-025	malloc(size)는 size 바이트의 메모리 블록을 할당하고 시작 주소를 반환하며, 확보하지 못하면 NULL을 반환한다. 근거 대조 문구: size 바이트	(A 3.6 동적 메모리 할당, 인쇄 p.95 / PDF p.88)
U02-026	malloc()의 반환값은 NULL인지 검사하고, free()로 할당된 메모리를 반환한다. 근거 대조 문구: 반환값은 항상 NULL인지	(A 3.6 동적 메모리 할당, 인쇄 p.95 / PDF p.88)
U02-027	malloc()이 반환한 포인터 값을 잊어버리면 동적 메모리를 반환할 수 없다. 근거 대조 문구: 포인터값을 잊어버리면	(A 3.6 동적 메모리 할당, 인쇄 p.95 / PDF p.88)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-028	텍스트 영역에는 실행할 코드가 적재되고, 데이터 영역에는 전역 변수나 정적 변수 등이 저장된다. 근거 대조 문구: 전역 변수나 정적 변수	(B 0.3.3 메모리 레이아웃 복습, 인쇄 p.11 / PDF p.22)
U02-029	스택 영역은 자동 메모리, 힙 영역은 자유 저장소라는 별명을 갖는다. 근거 대조 문구: 자동 메모리	(B 0.3.3 메모리 레이아웃 복습, 인쇄 p.11 / PDF p.22)
U02-030	원문의 지역 변수 a, b, c는 선언 순서대로 스택에 쌓이고 블록이 끝나면 c, b, a의 역순으로 제거된다. 근거 대조 문구: 제거는 그 역순	(B 0.3.4 스택에서 데이터를 다루는 방법, 인쇄 p.12 / PDF p.23)
U02-031	힙 메모리는 프로그래머가 직접 관리하며, B의 메모리 힙 영역은 힙 ADT와 관련이 없다. 근거 대조 문구: ADT와 관련 없습니다	(B 0.3.5 힙에서 데이터를 다루는 방법, 인쇄 p.12 / PDF p.23)
U02-032	지역 포인터 ptr이 블록 끝에서 제거되어도 malloc()으로 할당한 공간은 남으므로, free()로 해제하지 않으면 메모리 누수가 생긴다. 근거 대조 문구: 메모리 누수	(B 0.3.5 힙에서 데이터를 다루는 방법, 인쇄 p.13-14 / PDF p.24-25)
U02-033	두 다항식의 현재 항 차수를 비교한다. 근거 대조 문구: 각 항의 차수를 비교	(A 3.3 두 번째 방법, 인쇄 p.79 / PDF p.72)
U02-034	차수가 같으면 계수를 더하고, 다르면 차수가 큰 항을 결과로 옮긴다. 근거 대조 문구: 지수가 큰 항	(A 3.3 두 번째 방법, 인쇄 p.79 / PDF p.72)
U02-035	한쪽이 끝날 때까지 반복한 후 남은 항들을 옮긴다. 근거 대조 문구: 나머지 항들을 이동	(A 3.3 프로그램 3.3, 인쇄 p.79,82 / PDF p.72,75)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U02-036	프로그램 3.6 swap 원문 발췌 (원문 코드 발췌)
	void swap(int *px, int *py)
	{
	int tmp;
	tmp = *px;
	*px = *py;
	*py = tmp;
	}
	근거 대조 문구: void swap
		(A 3.5 프로그램 3.6, 인쇄 p.91 / PDF p.84)

U02-037	list[i] 요소의 주소: base+i*sizeof(int); base는 list[0]의 주소이다. 근거 대조 문구: base+i*sizeof(int)	(A 3.1 C언어에서의 1차원 배열, 인쇄 p.72 / PDF p.65)
U02-038	배열 근거 대조 문구: 배열	(A 3.2 구조체, 인쇄 p.73 / PDF p.66)
U02-039	구조체 근거 대조 문구: 구조체	(A 3.2 구조체, 인쇄 p.73 / PDF p.66)
U02-040	list[0]의 주소가 base이면 list[i]의 주소는 base+i*sizeof(int)이다. 근거 대조 문구: base+i*sizeof(int)	(A 3.1 1차원 배열, 인쇄 p.72 / PDF p.65)

03 알고리즘 성능 분석

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-001	시간 복잡도(time complexity)는 알고리즘의 수행 시간을 분석하는 것이며, 공간 복잡도(space complexity)는 필요한 기억 공간을 분석하는 것이다. 근거 대조 문구: 공간 복잡도	(A 1.3 알고리즘의 복잡도 분석 방법, 인쇄 p.24 / PDF p.19)
U03-002	빅오 표기법은 시간 복잡도 함수에서 불필요한 정보를 제거하여 분석을 쉽게 하며, 함수의 상한값을 나타낸다. 근거 대조 문구: 함수의 상한값	(A 1.3 빅오 표기법, 인쇄 p.28 / PDF p.23)
U03-003	재귀 방정식은 자기 자신을 항으로 갖는 방정식이며, 재귀 알고리즘의 수행 시간을 나타내는 데 사용한다. 근거 대조 문구: 자기 자신을 항	(B 11.3.1 재귀 방정식과 재귀 알고리즘, 인쇄 p.508-509 / PDF p.508-509)
U03-004	효율적인 알고리즘은 실행 시간이 짧고 메모리와 같은 컴퓨터 자원을 적게 사용한다. 근거 대조 문구: 실행시간이 짧고	(A 1.3 알고리즘의 성능 분석, 인쇄 p.22 / PDF p.17)
U03-005	실행 시간 측정은 알고리즘을 실제로 구현하고 실행 시간을 측정하여 성능을 비교하는 방법이다. 근거 대조 문구: 실제로 구현하고	(A 1.3 수행 시간 측정, 인쇄 p.22 / PDF p.17)
U03-006	실행 시간으로 비교하려면 같은 하드웨어와 소프트웨어 환경을 사용해야 하며, 실험하지 않은 입력에 대한 수행 시간은 알 수 없다. 근거 대조 문구: 동일한 하드웨어	(A 1.3 수행 시간 측정, 인쇄 p.23 / PDF p.18)
U03-007	복잡도 분석은 알고리즘을 구현하지 않고도 효율성을 평가하며, 하드웨어나 소프트웨어에 관계없이 분석할 수 있다. 근거 대조 문구: 직접 구현하지 않고서도	(A 1.3 알고리즘의 복잡도 분석 방법, 인쇄 p.24 / PDF p.19)
U03-008	시간 복잡도는 절대적인 실행 시간보다 알고리즘의 연산들이 몇 번 수행되는지를 센다. 근거 대조 문구: 몇 번이나 수행	(A 1.3 시간 복잡도 분석, 인쇄 p.24 / PDF p.19)
U03-009	연산의 수행 횟수를 입력 개수 n의 함수로 나타낸 것을 시간 복잡도 함수 T(n)이라고 한다. 근거 대조 문구: 시간 복잡도 함수	(A 1.3 시간 복잡도 분석, 인쇄 p.25 / PDF p.20)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-010	B의 대표적인 성능 측정 기준은 정확성·작업량·메모리 사용량·단순성·최적성이다. 근거 대조 문구: 정확성	(B 11.1.1 알고리즘 성능 측정 기준, 인쇄 p.499 / PDF p.499)
U03-011	정확성은 정해진 절차를 수행하여 정확한 출력을 제공하는가를, 작업량은 요구 기능을 수행하는 작업의 양을 뜻한다. 근거 대조 문구: 정확한 출력을 제공	(B 11.1.1 알고리즘 성능 측정 기준, 인쇄 p.500 / PDF p.500)
U03-012	메모리 사용량은 작업할 때 사용하는 메모리의 양이고, 최적성은 더 이상 개선할 부분이 없을 정도로 최적화되어 있는가를 가리킨다. 근거 대조 문구: 더 이상 개선할 부분	(B 11.1.1 알고리즘 성능 측정 기준, 인쇄 p.500 / PDF p.500)
U03-013	모든 $n>n_0$에 대해 $\|f(n)\|\leq c\|g(n)\|$을 만족하는 상수 c와 n_0가 존재하면 $f(n)=O(g(n))$이다. 근거 대조 문구: 모든 n>n	(A 1.3 정의 1.2 빅오 표기법, 인쇄 p.28 / PDF p.23)
U03-014	다항식에서는 최고차항만 고려하고 계수와 낮은 차수 항을 무시하되, log n과 같은 항을 함부로 무시해서는 안 된다. 근거 대조 문구: logn 항을 무시하면	(A 1.3 빅오 표기법, 인쇄 p.29 / PDF p.24)
U03-015	빅오 표기법의 계수 차이는 n이 작을 때 수행 시간에 영향을 줄 수 있다. 근거 대조 문구: 상수항도 중요	(A 1.3 빅오 표기법, 인쇄 p.30 / PDF p.25)
U03-016	상한은 여러 개일 수 있으므로 가능한 한 차수가 작은 빅오 표기법을 사용한다. 근거 대조 문구: 차수가 가장 낮은	(A 1.3 빅오메가와 빅세타 표기법, 인쇄 p.32 / PDF p.27)
U03-017	빅오메가 표기법은 함수의 하한을 나타내며, 모든 $n>n_0$에 대해 $\|f(n)\|\geq c\|g(n)\|$을 만족하는 상수 c와 n_0가 존재하면 $f(n)=\Omega(g(n))$이다. 근거 대조 문구: 하한	(A 1.3 정의 1.3 빅오메가 표기법, 인쇄 p.32 / PDF p.27)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-018	빅세타 표기법은 함수의 상한과 하한을 동시에 나타내며, 모든 $n>n_0$에 대해 $c_1\|g(n)\|\le\|f(n)\|\le c_2\|g(n)\|$을 만족하는 상수 c_1, c_2, n_0가 존재하면 $f(n)=\Theta(g(n))$이다. 근거 대조 문구: 상한과 하한	(A 1.3 정의 1.4 빅세타 표기법, 인쇄 p.32 / PDF p.27)
U03-019	Θ 표기법은 O 표기법과 Ω 표기법을 모두 만족하는 증가 함수를 나타내며, 두 집합의 교집합으로 설명된다. 근거 대조 문구: 교집합	(B 11.2.3 Θ 표기법, 인쇄 p.507 / PDF p.507)
U03-020	같은 알고리즘도 입력 자료 집합에 따라 실행 시간이 달라질 수 있어 최선·평균·최악의 경우를 구분한다. 근거 대조 문구: 입력 자료 집합	(A 1.3 최선, 평균, 최악의 경우, 인쇄 p.33 / PDF p.28)
U03-021	최선의 경우는 수행 시간이 가장 짧을 때, 최악의 경우는 수행 시간이 가장 오래 걸릴 때이다. 근거 대조 문구: 가장 오래	(A 1.3 최선, 평균, 최악의 경우, 인쇄 p.33 / PDF p.28)
U03-022	평균의 경우는 가능한 입력을 고려하여 각 경우의 실행 시간을 발생 확률에 따라 평균한 것이다. 근거 대조 문구: 평균적인 경우	(A 1.3 최선, 평균, 최악의 경우, 인쇄 p.33 / PDF p.28)
U03-023	평균의 경우를 구하기 어려운 알고리즘이 많아 최악의 경우를 주로 사용한다. 근거 대조 문구: 최악의 경우	(A 1.3 최선, 평균, 최악의 경우, 인쇄 p.33 / PDF p.28)
U03-024	순차 탐색의 최선은 찾는 항목이 맨 앞에 있어 비교가 한 번인 경우이며 O(1)이다. 근거 대조 문구: 맨 처음에	(A 1.3 순차 탐색 알고리즘, 인쇄 p.34 / PDF p.29)
U03-025	순차 탐색의 최악은 찾는 항목이 맨 뒤에 있어 모든 항목과 비교하는 경우이며 O(n)이다. 근거 대조 문구: 맨 뒤	(A 1.3 순차 탐색 알고리즘, 인쇄 p.34 / PDF p.29)
U03-026	각 항목이 탐색될 확률이 1/n으로 같다고 가정하면 순차 탐색의 평균 비교 횟수는 (n+1)/2이며 O(n)이다. 근거 대조 문구: 확률이 동일	(A 1.3 순차 탐색 알고리즘, 인쇄 p.35 / PDF p.30)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-027	RecurrenceSum()의 수행 시간은 T(n)=T(n−1)+c로 나타내고, 이를 전개하여 T(n)≤cn이라는 상한과 O(n)을 얻는다. 근거 대조 문구: T(n)	(B 11.3.1 재귀 방정식과 재귀 알고리즘, 인쇄 p.510 / PDF p.510)
U03-028	RecurrenceSum() 분석에서 상수 c는 재귀 호출 비용 외에 알고리즘에서 요구하는 처리 비용이다. 근거 대조 문구: 재귀 호출 비용 외에	(B 11.3.1 재귀 방정식과 재귀 알고리즘, 인쇄 p.510 / PDF p.510)
U03-029	퀵 정렬의 수행 시간은 분할 시간과 분할 뒤 두 재귀 호출 시간의 합이며, B는 비교 작업량을 분석한다. 근거 대조 문구: 비교 작업량	(B 11.3.2 퀵 정렬의 성능 분석, 인쇄 p.511 / PDF p.511)
U03-030	퀵 정렬은 기준 데이터가 가장 크거나 작으면 최악이고, 데이터를 절반에 가깝게 분할하면 최선이다. 근거 대조 문구: 절반에 가깝게	(B 11.3.2 퀵 정렬의 성능 분석, 인쇄 p.512 / PDF p.512)
U03-031	B의 퀵 정렬 분석 결과는 최악 $O(n^2)$, 최선 $O(n \log_2 n)$, 평균 O(n log n)이다. 근거 대조 문구: 평균의 경우	(B 11.3.2 퀵 정렬의 성능 분석, 인쇄 p.514-515,517 / PDF p.514-515,517)
U03-032	마스터 정리는 T(n)=aT(n/b)+f(n) 형태의 재귀 방정식에 대한 점근적 한계를 구하는 방법이며, a≥1, b>1은 상수이다. 근거 대조 문구: a≥1	(B 11.3.3 마스터 정리, 인쇄 p.517 / PDF p.517)
U03-033	어떤 상수 ε>0에 대해 f(n)=O(n^(log_b a−ε))이면 T(n)=Θ(n^(log_b a))이다. 근거 대조 문구: 어떤 상수	(B 11.3.3 마스터 정리 ①, 인쇄 p.517 / PDF p.517)
U03-034	f(n)=Θ(n^(log_b a))이면 T(n)=Θ(n^(log_b a) log(n))이다. 근거 대조 문구: log(n)	(B 11.3.3 마스터 정리 ②, 인쇄 p.517 / PDF p.517)
U03-035	어떤 상수 ε>0에 대해 f(n)=Ω(n^(log_b a+ε))이고, 상수 C(<1)와 충분히 큰 n에 대해 af(n/b)≤cf(n)이 성립하면 T(n)=Θ(f(n))이다. 근거 대조 문구: 충분히 큰 n	(B 11.3.3 마스터 정리 ③, 인쇄 p.517 / PDF p.517)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-036	O·Ω·Θ의 경계: $f(n)=O(g(n))$: $\lvert f(n)\rvert \le c\lvert g(n)\rvert$ $f(n)=\Omega(g(n))$: $\lvert f(n)\rvert \ge c\lvert g(n)\rvert$ $f(n)=\Theta(g(n))$: $c_1\lvert g(n)\rvert \le \lvert f(n)\rvert \le c_2\lvert g(n)\rvert$; 각 정의에서 상수들이 존재하고 모든 $n>n_0$에 대해 성립한다. 근거 대조 문구: 모든 n>n	(A 1.3 정의 1.2~1.4, 인쇄 p.28,32 / PDF p.23,27)
U03-037	순차 탐색 평균 비교 횟수: $(1+2+\cdots+n)/n=(n+1)/2$; 각 항목이 탐색될 확률이 1/n으로 동일하다는 가정이다. 근거 대조 문구: 확률이 동일	(A 1.3 순차 탐색 알고리즘, 인쇄 p.35 / PDF p.30)
U03-038	마스터 정리 대상: $T(n)=aT(n/b)+f(n)$; $a\ge1$, $b>1$인 상수이며 $f(n)$은 함수이다. 적용 조건은 위의 세 경우를 따른다. 근거 대조 문구: a≥1	(B 11.3.3 마스터 정리, 인쇄 p.517 / PDF p.517)
U03-039	$O(g(n))$ 근거 대조 문구: O(g(n))	(A 1.3 정의 1.2, 인쇄 p.28 / PDF p.23)
U03-040	$\Omega(g(n))$ 근거 대조 문구: 빅오메가	(A 1.3 정의 1.3, 인쇄 p.32 / PDF p.27)
U03-041	$\Theta(g(n))$ 근거 대조 문구: 빅세타	(A 1.3 정의 1.4, 인쇄 p.32 / PDF p.27)

04 순환과 재귀

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-001	순환(recursion)은 어떤 알고리즘이나 함수가 자기 자신을 호출하여 문제를 해결하는 프로그래밍 기법이다. 근거 대조 문구: 자기 자신을 호출	(A 2.1 순환의 소개, 인쇄 p.40 / PDF p.34)
U04-002	순환 알고리즘은 자기 자신을 순환적으로 호출하는 부분과 순환 호출을 멈추는 부분으로 구성된다. 근거 대조 문구: 멈추는 부분	(A 2.1 순환 알고리즘의 구조, 인쇄 p.46 / PDF p.40)
U04-003	순환적인 거듭제곱 계산은 호출할 때마다 문제 크기가 약 절반으로 줄어 시간 복잡도가 $O(\log_2 n)$이다. 근거 대조 문구: 절반	(A 2.2 거듭제곱값 계산, 인쇄 p.52-53 / PDF p.46-47)
U04-004	순환은 본질적으로 순환적인 문제나 그러한 자료구조를 다루는 프로그램에 적합하다. 근거 대조 문구: 본질적으로 순환적인 문제	(A 2.1 순환의 소개, 인쇄 p.40 / PDF p.34)
U04-005	순환 호출을 멈추는 부분이 없으면 시스템 스택을 다 사용할 때까지 호출되다가 오류를 내며 멈춘다. 근거 대조 문구: 시스템 스택을 다 사용	(A 2.1 순환 알고리즘의 구조, 인쇄 p.46 / PDF p.40)
U04-006	반복(iteration)은 for나 while 등의 반복 구조로 되풀이하는 방법이다. 근거 대조 문구: for나 while	(A 2.1 순환 ↔ 반복, 인쇄 p.47 / PDF p.41)
U04-007	반복과 순환은 기본적으로 문제 해결 능력이 같으며, 많은 경우 서로 바꾸어 쓸 수 있다. 근거 대조 문구: 문제 해결 능력이 같으며	(A 2.1 순환 ↔ 반복, 인쇄 p.47 / PDF p.41)
U04-008	순환은 문제의 일부를 해결한 뒤 나머지 문제에 대해 순환 호출을 한다. 근거 대조 문구: 문제의 일부를 해결	(A 2.1 순환의 원리, 인쇄 p.48 / PDF p.42)
U04-009	주어진 문제를 더 작은 동일한 문제들로 분해하여 해결하는 방법을 분할정복(divide and conquer)이라 한다. 근거 대조 문구: 더 작은 동일한 문제	(A 2.1 순환의 원리, 인쇄 p.49 / PDF p.43)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-010	순환 호출이 일어날 때마다 문제의 크기가 작아지며, 결국 쉽게 풀 수 있는 문제가 된다. 근거 대조 문구: 문제의 크기가 작아진다	(A 2.1 순환의 원리, 인쇄 p.49 / PDF p.43)
U04-011	함수를 호출하면 복귀주소를 시스템 스택에 저장하고, 호출되는 함수의 매개변수와 지역 변수를 스택에서 할당받는다. 근거 대조 문구: 복귀주소가 시스템 스택	(A 2.1 순환 호출의 내부적인 구현, 인쇄 p.44 / PDF p.38)
U04-012	활성 레코드(activation record)는 함수를 위해 시스템 스택에서 사용하는 공간이다. 근거 대조 문구: 활성 레코드	(A 2.1 순환 호출의 내부적인 구현, 인쇄 p.44 / PDF p.38)
U04-013	호출된 함수가 끝나면 시스템 스택에서 복귀주소를 추출하여 호출한 함수로 돌아가며, 순환 호출이 중첩될수록 활성 레코드가 쌓인다. 근거 대조 문구: 활성 레코드들이 쌓이게	(A 2.1 순환 호출의 내부적인 구현, 인쇄 p.44 / PDF p.38)
U04-014	순환적인 factorial(n)은 n≤1이면 1을 반환하고, 그렇지 않으면 n*factorial(n−1)을 반환한다. 근거 대조 문구: n * factorial	(A 2.1 프로그램 2.1, 인쇄 p.41 / PDF p.35)
U04-015	factorial(3)은 factorial(2), factorial(1)을 차례로 호출하고 반환값 1, 2, 6을 차례로 전달한다. 근거 대조 문구: 6을 반환	(A 2.1 순환의 예, 인쇄 p.42 / PDF p.36)
U04-016	팩토리얼의 반복 알고리즘과 순환 알고리즘은 모두 시간 복잡도가 O(n)이다. 근거 대조 문구: 역시 O(n)	(A 2.1 순환 알고리즘의 성능, 인쇄 p.50 / PDF p.44)
U04-017	순환적인 팩토리얼은 호출 상태를 기억할 여분의 기억 공간과 함수 호출의 사전 작업이 필요하다. 근거 대조 문구: 여분의 기억공간	(A 2.1 순환 알고리즘의 성능, 인쇄 p.50 / PDF p.44)
U04-018	순환적인 power(x,n)은 n=0이면 1, n이 짝수이면 power(x^2,n/2), 홀수이면 x*power(x^2,(n−1)/2)를 반환한다. 근거 대조 문구: n이 짝수	(A 2.2 알고리즘 2.1, 인쇄 p.51 / PDF p.45)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-019	반복적인 slow_power()는 루프마다 한 번의 곱셈이 필요하고 루프 횟수가 n이므로 시간 복잡도는 O(n)이다. 근거 대조 문구: 루프의 개수는 정확히	(A 2.2 거듭제곱값 계산, 인쇄 p.53 / PDF p.47)
U04-020	피보나치 수열은 앞의 두 숫자를 더해 뒤의 숫자를 만들며, fib(0)=0, fib(1)=1이다. 근거 대조 문구: 앞의 두 개의 숫자를 더해서	(A 2.3 피보나치 수열의 계산, 인쇄 p.54 / PDF p.48)
U04-021	순환적인 fib(n)은 fib(n−1)+fib(n−2)를 반환하며, 같은 값을 여러 번 계산할 수 있다. 근거 대조 문구: fib(n-1)	(A 2.3 피보나치 수열의 계산, 인쇄 p.54-55 / PDF p.48-49)
U04-022	순환적인 피보나치 계산이 비효율적인 근본 이유는 중간에 계산한 값을 기억하지 않고 다시 계산하기 때문이다. 근거 대조 문구: 기억하지 않고 다시 계산	(A 2.3 피보나치 수열의 계산, 인쇄 p.55 / PDF p.49)
U04-023	A가 제시한 순환적인 피보나치 알고리즘의 대략적인 시간 복잡도 상한은 $O(2^n)$이다. 근거 대조 문구: 상한	(A 2.3 피보나치 수열의 계산, 인쇄 p.55 / PDF p.49)
U04-024	반복적인 fib_iter()는 pp와 p에 앞의 두 값을 두고 result=p+pp를 계산한 뒤 pp=p, p=result로 갱신한다. 근거 대조 문구: result = p + pp	(A 2.3 프로그램 2.7, 인쇄 p.56 / PDF p.50)
U04-025	하노이 탑은 한 번에 하나의 원판만, 맨 위 원판만 이동하며, 작은 원판 위에 큰 원판을 쌓을 수 없다. 근거 대조 문구: 한 번에 하나의 원판	(A 2.4 하노이탑 문제, 인쇄 p.57 / PDF p.51)
U04-026	하노이 탑은 중간 막대를 임시로 사용할 수 있으나 원판 이동 조건을 지켜야 한다. 근거 대조 문구: 중간의 막대를 임시적으로	(A 2.4 하노이탑 문제, 인쇄 p.57 / PDF p.51)
U04-027	n개 원판을 옮길 때 문제의 크기는 이동해야 하는 원판의 개수이며, 순환할수록 작아진다. 근거 대조 문구: 문제의 크기는 이동	(A 2.4 하노이탑 문제, 인쇄 p.58 / PDF p.52)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-028	꼬리 순환(tail recursion)은 순환 호출이 순환 함수의 맨 끝에서 이루어지는 형태이며, 쉽게 반복 형태로 변환할 수 있다. 근거 대조 문구: 꼬리 순환	(A 2.1·2.4 반복적인 형태로 바꾸기 어려운 순환, 인쇄 p.47,62 / PDF p.41,56)
U04-029	머리 순환(head recursion)이나 여러 군데에서 자기 자신을 호출하는 경우(multi recursion)는 쉽게 반복 코드로 바꿀 수 없지만, 명시적인 스택으로 순환을 시뮬레이션할 수 있다. 근거 대조 문구: 명시적인 스택	(A 2.4 반복적인 형태로 바꾸기 어려운 순환, 인쇄 p.62 / PDF p.56)
U04-030	factorial(3)이 factorial(2)를 호출한다. 근거 대조 문구: factorial(2)를 호출	(A 2.1 순환의 예, 인쇄 p.42 / PDF p.36)
U04-031	factorial(2)가 factorial(1)을 호출하고, factorial(1)은 더 호출하지 않고 1을 반환한다. 근거 대조 문구: 1을 반환	(A 2.1 순환의 예, 인쇄 p.42 / PDF p.36)
U04-032	factorial(2)가 2를 곱해 2를 반환하고, factorial(3)이 3을 곱해 6을 반환한다. 근거 대조 문구: 6을 반환	(A 2.1 순환의 예, 인쇄 p.42 / PDF p.36)
U04-033	A에 쌓인 n개 원판 중 위의 n−1개 원판을 B로 옮긴다. 근거 대조 문구: B로 옮긴	(A 2.4 그림 2-15, 인쇄 p.59 / PDF p.53)
U04-034	A의 제일 밑 원판을 C로 옮긴다. 근거 대조 문구: 제일 밑에 있는 원판	(A 2.4 그림 2-15, 인쇄 p.59 / PDF p.53)
U04-035	B의 n−1개 원판을 C로 옮긴다. 근거 대조 문구: C로 옮긴다	(A 2.4 그림 2-15, 인쇄 p.59 / PDF p.53)

U04-036	프로그램 2.1 순환적인 팩토리얼 (원문 코드 발췌)
	int factorial(int n)
	{
	if(n <= 1) return(1);
	else return (n * factorial(n-1));
	}
	근거 대조 문구: n * factorial
		(A 2.1 프로그램 2.1, 인쇄 p.41 / PDF p.35)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U04-037	프로그램 2.5 순환적인 거듭제곱 (원문 코드 발췌)
	double power(double x, int n)
	{
	if(n==0) return 1;
	else if ((n%2)==0)
	return power(x*x, n/2);
	else return x*power(x*x, (n-1)/2);
	}
	근거 대조 문구: double power
		(A 2.2 프로그램 2.5, 인쇄 p.52 / PDF p.46)

U04-038	팩토리얼의 순환적 정의: n!=1 (n=0) n!=n×(n−1)! (n≥1); A의 수학적 정의. 프로그램 2.1의 종료 조건은 n≤1이다. 근거 대조 문구: 팩토리얼	(A 2.1 순환의 소개, 인쇄 p.40 / PDF p.34)
U04-039	피보나치 수열의 정의: fib(n)=0 (n=0) fib(n)=1 (n=1) fib(n)=fib(n−2)+fib(n−1) (otherwise) 근거 대조 문구: 앞의 두 개의 숫자를 더해서	(A 2.3 피보나치 수열의 계산, 인쇄 p.54 / PDF p.48)
U04-040	순환적인 피보나치 계산의 상한: $O(2^n)$ 근거 대조 문구: 상한	(A 2.3 피보나치 수열의 계산, 인쇄 p.55 / PDF p.49)
U04-041	팩토리얼: 반복·순환 근거 대조 문구: 팩토리얼	(A 2.1 순환 알고리즘의 성능, 인쇄 p.50 / PDF p.44)
U04-042	거듭제곱: 반복 근거 대조 문구: 반복적인 함수	(A 2.2 표 2-1, 인쇄 p.53 / PDF p.47)
U04-043	거듭제곱: 순환 근거 대조 문구: 순환적인 함수	(A 2.2 표 2-1, 인쇄 p.53 / PDF p.47)

05 연결 리스트

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-001	리스트는 배열과 연결 리스트를 이용하여 구현할 수 있다. 근거 대조 문구: 배열과 연결 리스트를 이용하여 구현	(A 6.1 리스트 추상 데이터 타입, 인쇄 p.177 / PDF p.169)
U05-002	이중 연결 리스트(doubly linked list)는 각 노드에 앞 노드와 뒤 노드를 가리키는 2개의 링크가 존재한다. 근거 대조 문구: 각 노드마다 2개의 링크	(A 6.3 연결 리스트의 종류, 인쇄 p.186 / PDF p.178)
U05-003	리스트의 객체는 n개의 element형으로 구성된 순서 있는 모임이다. 근거 대조 문구: n개의 element형으로 구성된 순서 있는 모임	(A 6.1 리스트 추상 데이터 타입, 인쇄 p.177 / PDF p.169)
U05-004	첫 번째 노드를 헤드(Head), 마지막 노드를 테일(Tail)이라 부르며, 리스트의 길이는 헤드부터 테일까지의 노드 개수이다. 근거 대조 문구: 첫 번째 노드를 헤드	(B 1.1.1 리스트의 개념, 인쇄 p.21 / PDF p.31)
U05-005	insert(), delete(), get_entry(), get_length(), is_empty(), is_full() 등의 연산으로 리스트를 다룬다. 근거 대조 문구: get_entry(list, pos)	(A 6.1 리스트 추상 데이터 타입, 인쇄 p.177 / PDF p.169)
U05-006	배열을 이용한 리스트 구현은 순차적인 메모리 공간을 할당하므로 순차적 표현(sequential representation)이라고도 한다. 근거 대조 문구: 순차적 표현	(A 6.2 배열로 구현된 리스트, 인쇄 p.178 / PDF p.170)
U05-007	배열 리스트는 크기가 고정되고, 중간에 삽입하거나 삭제하려면 기존 데이터를 이동하여야 한다. 근거 대조 문구: 기존의 데이터들을 이동	(A 6.2 배열로 구현된 리스트, 인쇄 p.178 / PDF p.170)
U05-008	배열 리스트의 get_entry 연산은 인덱스로 항목에 바로 접근하므로 O(1)이다. 근거 대조 문구: get_entry	(A 6.2 실행 시간 분석, 인쇄 p.183 / PDF p.175)
U05-009	배열 리스트의 삽입이나 삭제는 최악의 경우 O(n)이다. 근거 대조 문구: 최악의 경우 O(n)	(A 6.2 실행 시간 분석, 인쇄 p.183 / PDF p.175)
U05-010	배열 리스트의 맨 끝에 삽입하는 경우는 O(1)이다. 근거 대조 문구: 맨 끝에 삽입하는 경우는 O(1)	(A 6.2 실행 시간 분석, 인쇄 p.183 / PDF p.175)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-011	연결 리스트의 노드는 데이터 필드(data field)와 링크 필드(link field)로 구성된다. 근거 대조 문구: 데이터 필드(data field)와 링크 필드(link field)	(A 6.3 연결 리스트, 인쇄 p.185 / PDF p.177)
U05-012	데이터 필드에는 저장할 데이터가, 링크 필드에는 다른 노드를 가리키는 포인터가 저장된다. 근거 대조 문구: 링크 필드에는 다른 노드를 가리키는 포인터	(A 6.3 연결 리스트, 인쇄 p.185 / PDF p.177)
U05-013	헤드 포인터(head pointer)는 연결 리스트의 첫 번째 노드를 가리키는 변수이다. 근거 대조 문구: 첫 번째 노드를 가리키고 있는 변수	(A 6.3 연결 리스트, 인쇄 p.185 / PDF p.177)
U05-014	연결 리스트는 포인터도 저장하여야 하므로 메모리 공간을 많이 사용하고, i번째 데이터는 앞에서부터 순차적으로 접근하여야 한다. 근거 대조 문구: i번째 데이터를 찾으려면 앞에서부터 순차적으로	(A 6.3 연결 리스트, 인쇄 p.185 / PDF p.177)
U05-015	자기 참조 구조체는 자기 자신을 참조하는 포인터를 포함하는 구조체이다. 근거 대조 문구: 자기 자신을 참조하는 포인터	(A 6.4 노드의 정의, 인쇄 p.188 / PDF p.180)
U05-016	공백인 단순 연결 리스트에서는 헤드 포인터의 값이 NULL이다. 근거 대조 문구: head의 값은 NULL	(A 6.4 공백 리스트의 생성, 인쇄 p.188 / PDF p.180)
U05-017	SLL_CreateNode()는 malloc()으로 노드를 생성하고 데이터를 저장한 뒤 NextNode를 NULL로 초기화한다. 근거 대조 문구: NextNode = NULL	(B 1.2.2 노드 생성/소멸 연산, 인쇄 p.28 / PDF p.38)
U05-018	SLL_DestroyNode()는 노드의 주소를 free()에 넘겨 노드를 소멸시킨다. 근거 대조 문구: free(Node)	(B 1.2.2 노드 생성/소멸 연산, 인쇄 p.28 / PDF p.38)
U05-019	노드 삭제는 리스트에서 노드를 제외하는 연산이다. 근거 대조 문구: 노드 삭제는 리스트에서 노드를 제외	(B 1.2.2 링크드 리스트의 주요 연산, 인쇄 p.25 / PDF p.35)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-020	노드 소멸은 노드를 메모리에서 없애는 연산이다. 근거 대조 문구: 노드 소멸은 노드를 메모리에서 없애는	(B 1.2.2 링크드 리스트의 주요 연산, 인쇄 p.25 / PDF p.35)
U05-021	insert_first()는 변경된 헤드 포인터를 반환하므로 반환값을 헤드 포인터에 저장하여야 한다. 근거 대조 문구: 반환된 값을 헤드포인트에 저장	(A 6.5 삽입 연산 insert_first(), 인쇄 p.191 / PDF p.183)
U05-022	중간 삽입 insert()는 삽입 위치의 선행 노드를 알아야 가능하다. 근거 대조 문구: 선행 노드를 알아야 삽입이 가능	(A 6.5 삽입 연산 insert(), 인쇄 p.192 / PDF p.184)
U05-023	단순 연결 리스트(singly linked list)는 하나의 방향으로만 연결되며 체인(chain)이라고도 한다. 근거 대조 문구: 체인(chain)	(A 6.3 연결 리스트의 종류, 인쇄 p.186 / PDF p.178)
U05-024	단순 연결 리스트의 마지막 노드의 링크는 NULL값을 가진다. 근거 대조 문구: 마지막 노드의 링크는 NULL값	(A 6.3 연결 리스트의 종류, 인쇄 p.186 / PDF p.178)
U05-025	원형 연결 리스트(circular linked list)는 마지막 노드의 링크가 첫 번째 노드를 가리킨다. 근거 대조 문구: 마지막 노드의 링크가 첫 번째 노드를 가리킨다	(A 6.3 연결 리스트의 종류, 인쇄 p.186 / PDF p.178)
U05-026	헤드 포인터가 마지막 노드를 가리키도록 구성한 원형 연결 리스트는 상수 시간 안에 처음과 끝에 노드를 삽입할 수 있다. 근거 대조 문구: 상수 시간 안에 리스트의 처음과 끝	(A 7.1 원형 연결 리스트, 인쇄 p.222 / PDF p.214)
U05-027	헤드 노드(head node)는 데이터를 가지고 있지 않은 특별한 노드로 삽입·삭제 알고리즘을 간편하게 한다. 근거 대조 문구: 데이터를 가지고 있지 않는 특별한 노드	(A 7.3 이중 연결 리스트, 인쇄 p.230 / PDF p.222)
U05-028	이중 연결 리스트는 양방향 검색이 가능하지만 공간을 많이 차지하고 코드가 복잡해진다. 근거 대조 문구: 공간을 많이 차지하고 코드가 복잡	(A 7.3 이중 연결 리스트, 인쇄 p.230 / PDF p.222)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-029	더블 링크드 리스트를 환형으로 구현하면 헤드의 이전 노드는 테일이고 테일의 다음 노드는 헤드가 된다. 근거 대조 문구: 헤드의 이전 노드가 테일	(B 1.4 환형 링크드 리스트, 인쇄 p.57 / PDF p.67)
U05-030	다항식의 연결 리스트 표현에서는 각 항을 하나의 노드로 표현하며 계수(coef), 지수(exp), 링크(link) 필드를 둔다. 근거 대조 문구: 각 항을 하나의 노드로 표현	(A 6.6 연결 리스트의 응용: 다항식, 인쇄 p.209 / PDF p.201)
U05-031	링크드 리스트는 레코드의 추가·삽입·삭제가 잦지만 조회는 드문 곳에서 사용하기 적합하다. 근거 대조 문구: 추가, 삽입, 삭제가 잦지만 조회는 드문	(B 1.2.4 링크드 리스트의 장단점, 인쇄 p.43 / PDF p.53)
U05-032	동적 메모리 할당으로 새로운 노드 p를 생성한다. 근거 대조 문구: 새로운 노드 p를 생성	(A 6.5 알고리즘 6.1, 인쇄 p.192 / PDF p.184)
U05-033	p->data에 value를 저장한다. 근거 대조 문구: p->data에 value	(A 6.5 알고리즘 6.1, 인쇄 p.192 / PDF p.184)
U05-034	p->link를 현재의 head값으로 변경한다. 근거 대조 문구: p->link를 현재의 head값	(A 6.5 알고리즘 6.1, 인쇄 p.192 / PDF p.184)
U05-035	head를 p값으로 변경한다. 근거 대조 문구: head를 p값	(A 6.5 알고리즘 6.1, 인쇄 p.192 / PDF p.184)
U05-036	변경된 헤드 포인터를 반환한다. 근거 대조 문구: 변경된 헤드 포인터 반환	(A 6.5 알고리즘 6.1, 인쇄 p.192 / PDF p.184)
U05-037	단순 연결 리스트 노드 정의 (원문 코드 발췌) typedef int element; typedef struct ListNode { // 노드 타입을 구조체로 정의한다. element data; struct ListNode *link; } ListNode; 근거 대조 문구: typedef struct ListNode	(A 6.4 노드의 정의, 인쇄 p.188 / PDF p.180)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-038	배열 리스트의 접근: get_entry: O(1); 배열의 인덱스로 항목에 바로 접근한다. 근거 대조 문구: get_entry	(A 6.2 실행 시간 분석, 인쇄 p.183 / PDF p.175)
U05-039	배열 리스트의 삽입·삭제: 최악의 경우 O(n); 맨 끝 삽입 O(1); 다른 항목을 이동하는 삽입·삭제의 최악 경우와 맨 끝 삽입을 구분한다. 근거 대조 문구: 최악의 경우 O(n)	(A 6.2 실행 시간 분석, 인쇄 p.183 / PDF p.175)
U05-040	단순 연결 리스트 근거 대조 문구: 단순 연결 리스트	(A 6.3 연결 리스트의 종류, 인쇄 p.186 / PDF p.178)
U05-041	원형 연결 리스트 근거 대조 문구: 원형 연결 리스트	(A 6.3 연결 리스트의 종류, 인쇄 p.186 / PDF p.178)
U05-042	이중 연결 리스트 근거 대조 문구: 이중 연결 리스트	(A 6.3 연결 리스트의 종류, 인쇄 p.186 / PDF p.178)

06 스택·수식·미로

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-001	스택의 입출력 형태는 후입선출(LIFO: Last-In First-Out)이다. 근거 대조 문구: 후입선출(LIFO: Last-In First-Out)	(A 4.1 스택이란?, 인쇄 p.102 / PDF p.94)
U06-002	후위(postfix)는 연산자가 피연산자 뒤에 있는 표기법이다. 근거 대조 문구: 피연산자 뒤에 있으면 후위	(A 4.5 수식의 3가지 표기법, 인쇄 p.124 / PDF p.116)
U06-003	스택의 입출력은 맨 위에서만 일어나며, 입출력이 이루어지는 부분을 스택 상단(stack top)이라 한다. 근거 대조 문구: 스택 상단(stack top)	(A 4.1 스택이란?, 인쇄 p.103 / PDF p.95)
U06-004	스택에 요소가 하나도 없을 때 공백 스택(empty stack)이라고 한다. 근거 대조 문구: 공백 스택(empty stack)	(A 4.1 스택이란?, 인쇄 p.103 / PDF p.95)
U06-005	함수 호출 시 시스템 스택에 활성 레코드(activation record)가 만들어지며 복귀주소·매개변수·지역 변수가 저장된다. 근거 대조 문구: 활성 레코드(activation record)	(A 4.1 시스템 스택을 이용한 함수 호출, 인쇄 p.103 / PDF p.95)
U06-006	추상 자료형 스택의 객체는 0개 이상의 원소를 가지는 유한 선형 리스트이다. 근거 대조 문구: 0개 이상의 원소를 가지는 유한 선형 리스트	(A 4.1 ADT 4.1 Stack, 인쇄 p.105 / PDF p.97)
U06-007	push는 스택의 맨 위에 item을 추가하는 연산이다. 근거 대조 문구: 스택의 맨 위에 item을 추가	(A 4.1 ADT 4.1 Stack, 인쇄 p.105 / PDF p.97)
U06-008	pop은 스택의 맨 위의 원소를 제거해서 반환하는 연산이다. 근거 대조 문구: 맨 위의 원소를 제거해서 반환	(A 4.1 ADT 4.1 Stack, 인쇄 p.105 / PDF p.97)
U06-009	peek는 스택의 맨 위의 원소를 제거하지 않고 반환하는 연산이다. 근거 대조 문구: 맨 위의 원소를 제거하지 않고 반환	(A 4.1 ADT 4.1 Stack, 인쇄 p.105 / PDF p.97)
U06-010	포화 스택의 push와 공백 스택의 pop에서는 오류가 발생한다. 근거 대조 문구: 스택이 가득차서 입력이 불가능	(A 4.1 추상 자료형 스택, 인쇄 p.105 / PDF p.97)
U06-011	배열 스택에서 top은 가장 최근에 입력된 자료를 가리키며, 공백일 때 -1이다. 근거 대조 문구: top 변수는 스택이 비어 있으면 -1	(A 4.2 스택의 구현, 인쇄 p.106 / PDF p.98)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-012	배열 스택에서 top이 MAX_STACK_SIZE-1이면 포화 상태이다. 근거 대조 문구: 포화 상태로 판정	(A 4.2 스택의 is_full 연산, 인쇄 p.107 / PDF p.99)
U06-013	배열 스택의 push는 top을 먼저 증가시킨 후 stack[top]에 요소를 삽입한다. 근거 대조 문구: 먼저 top의 값을 증가	(A 4.2 스택의 push 연산, 인쇄 p.108 / PDF p.100)
U06-014	배열 스택의 pop은 공백 여부를 검사한 뒤 top이 가리키는 값을 반환하고 top을 하나 감소시킨다. 근거 대조 문구: top을 하나 감소	(A 4.2 스택의 pop 연산, 인쇄 p.108 / PDF p.100)
U06-015	동적 배열 스택의 push는 공간이 부족하면 capacity를 2배로 늘리고 realloc()으로 메모리를 확보한다. 근거 대조 문구: 공간이 부족하면 메모리를 2배	(A 4.3 동적 배열 스택, 인쇄 p.117 / PDF p.109)
U06-016	연결된 스택은 동적 메모리 할당이 가능하면 새로운 요소를 삽입할 수 있으며, top은 노드를 가리키는 포인터이다. 근거 대조 문구: top은 더 이상 정수가 아니고	(A 7.5 연결 리스트로 구현한 스택, 인쇄 p.239 / PDF p.231)
U06-017	A의 연결 스택은 단순 연결 리스트의 맨 앞에 노드를 삽입한다. 근거 대조 문구: 맨 앞에 데이터를 삽입	(A 7.5 연결 리스트로 구현한 스택, 인쇄 p.240 / PDF p.232)
U06-018	B의 링크드 리스트 스택은 List가 헤드를 가리키고 Top이 테일을 가리키는 구현을 사용한다. 근거 대조 문구: Top 포인터는 이 링크드 리스트의 테일	(B 2.3.1 링크드 리스트 기반 스택과 스택의 노드 표현, 인쇄 p.83 / PDF p.92)
U06-019	올바른 괄호는 좌우 개수가 같고, 같은 종류의 왼쪽 괄호가 먼저 나오며, 서로 다른 종류의 괄호 쌍은 교차하지 않는다. 근거 대조 문구: 괄호의 검사 조건은 다음의 3가지	(A 4.4 스택의 응용: 괄호 검사 문제, 인쇄 p.119 / PDF p.111)
U06-020	중위(infix)는 연산자가 피연산자 사이에 있는 표기법이다. 근거 대조 문구: 연산자가 피연산자 사이	(A 4.5 수식의 3가지 표기법, 인쇄 p.124 / PDF p.116)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-021	전위(prefix)는 연산자가 피연산자 앞에 있는 표기법이다. 근거 대조 문구: 피연산자 앞에 있으면 전위	(A 4.5 수식의 3가지 표기법, 인쇄 p.124 / PDF p.116)
U06-022	A의 후위표기식 계산 프로그램은 피연산자가 한 문자로 된 숫자라고 가정한다. 근거 대조 문구: 피연산자가 한 문자로 된 숫자	(A 4.5 프로그램 4.7 후위표기식 계산, 인쇄 p.127 / PDF p.119)
U06-023	중위식을 후위식으로 바꿀 때 스택의 연산자 우선순위가 현재 연산자보다 높거나 같으면 먼저 꺼내어 출력한다. 근거 대조 문구: 우선순위가 같은 경우에도	(A 4.5 중위표기수식을 후위표기수식으로 변환, 인쇄 p.129 / PDF p.121)
U06-024	왼쪽 괄호는 무조건 스택에 삽입하고, 스택 안에서는 우선순위가 제일 낮은 연산자로 취급한다. 근거 대조 문구: 왼쪽괄호는 무조건 스택에 삽입	(A 4.5 중위표기수식을 후위표기수식으로 변환, 인쇄 p.130 / PDF p.122)
U06-025	오른쪽 괄호를 만나면 왼쪽 괄호가 삭제될 때까지 그 위에 쌓인 모든 연산자를 출력한다. 근거 대조 문구: 모든 연산자들을 출력	(A 4.5 중위표기수식을 후위표기수식으로 변환, 인쇄 p.130 / PDF p.122)
U06-026	텍스트 분석에서 토큰은 최소 단위의 기호 또는 단어를 말한다. 근거 대조 문구: 토큰은 최소 단위의 기호 또는 단어	(B 2.4.2 토큰이 알고 싶다, 인쇄 p.96 / PDF p.105)
U06-027	왼쪽 괄호를 만나면 스택에 삽입한다. 근거 대조 문구: 왼쪽 괄호를 만나면 스택에 삽입	(A 4.4 괄호 검사 알고리즘, 인쇄 p.120 / PDF p.112)
U06-028	오른쪽 괄호를 만나면 맨 위의 괄호를 꺼내 짝을 검사하며, 스택이 비었거나 짝이 맞지 않으면 오류이다. 근거 대조 문구: 짝이 맞는지를 검사	(A 4.4 괄호 검사 알고리즘, 인쇄 p.120 / PDF p.112)
U06-029	문자열을 모두 조사한 뒤 괄호가 남으면 FALSE, 남지 않으면 TRUE를 반환한다. 근거 대조 문구: 성공이므로 TRUE	(A 4.4 괄호 검사 알고리즘, 인쇄 p.120 / PDF p.112)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-030	스택을 생성·초기화하고 후위식을 순서대로 읽어 피연산자를 스택에 삽입한다. 근거 대조 문구: 스택 s를 생성하고 초기화	(A 4.5 알고리즘 4.6, 인쇄 p.126 / PDF p.118)
U06-031	연산자 op를 만나면 second, first 순서로 pop하고 first op second의 결과를 push한다. 근거 대조 문구: result ← first op second	(A 4.5 알고리즘 4.6, 인쇄 p.126 / PDF p.118)
U06-032	식의 처리가 끝나면 pop한 값을 최종 결과로 얻는다. 근거 대조 문구: final_result	(A 4.5 알고리즘 4.6, 인쇄 p.126 / PDF p.118)
U06-033	스택과 출구·현재 위치를 초기화한다. 근거 대조 문구: 현재 생쥐의 위치를 초기화	(A 4.6 알고리즘 4.8, 인쇄 p.137 / PDF p.129)
U06-034	현재 위치를 방문 표시하고, 위·아래·왼쪽·오른쪽의 미방문·이동 가능 위치를 스택에 push한다. 근거 대조 문구: 위, 아래, 왼쪽, 오른쪽	(A 4.6 알고리즘 4.8, 인쇄 p.137 / PDF p.129)
U06-035	스택이 비면 실패이고, 그렇지 않으면 하나를 꺼내 현재 위치로 정하여 출구에 이를 때까지 반복한다. 근거 대조 문구: 스택에서 하나의 위치를 꺼내어 현재 위치	(A 4.6 알고리즘 4.8, 인쇄 p.137 / PDF p.129)
U06-036	동적 배열 스택 구조체 (원문 코드 발췌) typedef int element; typedef struct { element *data; int capacity; int top; } StackType; 근거 대조 문구: element *data;	(A 4.3 동적 배열 스택, 인쇄 p.117 / PDF p.109)
U06-037	A의 배열 스택 상태 조건: 공백: top == -1; 포화: top >= (MAX_STACK_SIZE-1); 배열 인덱스는 0부터 시작한다. 근거 대조 문구: top >= (MAX_STACK_SIZE-1)	(A 4.2 알고리즘 4.1·4.2, 인쇄 p.107 / PDF p.99)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-038	push 근거 대조 문구: push(s, item)	(A 4.1 ADT 4.1 Stack, 인쇄 p.105 / PDF p.97)
U06-039	pop 근거 대조 문구: pop(s)	(A 4.1 ADT 4.1 Stack, 인쇄 p.105 / PDF p.97)
U06-040	peek 근거 대조 문구: peek(s)	(A 4.1 ADT 4.1 Stack, 인쇄 p.105 / PDF p.97)

07 큐·덱·시뮬레이션

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-001	큐(queue)는 먼저 들어온 데이터가 먼저 나가는 선입선출(FIFO: First-In First-Out) 구조이다. 근거 대조 문구: 선입선출(FIFO: First-In First-Out)	(A 5.1 큐 추상 데이터 타입, 인쇄 p.146 / PDF p.138)
U07-002	덱(deque)은 double-ended queue의 줄임말이며 전단과 후단에서 모두 삽입·삭제가 가능한 큐이다. 근거 대조 문구: double-ended queue	(A 5.5 덱이란?, 인쇄 p.162 / PDF p.154)
U07-003	큐에서 삽입이 일어나는 곳은 후단(rear), 삭제가 일어나는 곳은 전단(front)이다. 근거 대조 문구: 후단(rear)	(A 5.1 큐 추상 데이터 타입, 인쇄 p.146 / PDF p.138)
U07-004	큐의 객체는 0개 이상의 요소들로 구성된 선형 리스트이다. 근거 대조 문구: 0개 이상의 요소들로 구성된 선형 리스트	(A 5.1 ADT 5.1 큐, 인쇄 p.147 / PDF p.139)
U07-005	enqueue는 큐의 끝에 요소를 추가하며 포화 큐에서는 queue_full 오류를 낸다. 근거 대조 문구: queue_full 오류	(A 5.1 ADT 5.1 큐, 인쇄 p.147 / PDF p.139)
U07-006	dequeue는 큐의 맨 앞 요소를 제거하여 반환하며 공백 큐에서는 queue_empty 오류를 낸다. 근거 대조 문구: 맨 앞에 있는 e를 제거하여 반환	(A 5.1 ADT 5.1 큐, 인쇄 p.147 / PDF p.139)
U07-007	peek는 공백이 아닌 큐의 맨 앞 요소를 읽어서 반환한다. 근거 대조 문구: 맨 앞에 있는 e를 읽어서 반환	(A 5.1 ADT 5.1 큐, 인쇄 p.147 / PDF p.139)
U07-008	선형큐는 front와 rear가 계속 증가하므로 배열 앞부분이 비어 있어도 사용하지 못할 수 있다. 근거 대조 문구: 배열의 앞부분이 비어 있더라도	(A 5.3 원형큐, 인쇄 p.153 / PDF p.145)
U07-009	선형큐의 오른쪽에 삽입 공간을 만들려면 요소들을 왼쪽으로 이동시켜야 한다. 근거 대조 문구: 모든 요소들을 왼쪽으로 이동	(A 5.3 원형큐, 인쇄 p.153 / PDF p.145)
U07-010	원형큐는 인덱스가 배열 끝에 도달한 뒤 다시 0으로 돌아가도록 한다. 근거 대조 문구: 다음에 증가되는 값은 0	(A 5.3 원형큐, 인쇄 p.154 / PDF p.146)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-011	A의 원형큐에서 front는 첫 번째 요소의 하나 앞을, rear는 마지막 요소를 가리킨다. 근거 대조 문구: 첫 번째 요소의 하나 앞	(A 5.3 원형큐, 인쇄 p.154 / PDF p.146)
U07-012	A의 원형큐는 front와 rear를 모두 0으로 초기화한다. 근거 대조 문구: front, rear는 모두 0	(A 5.3 원형큐, 인쇄 p.154 / PDF p.146)
U07-013	A의 원형큐 삽입은 rear를 먼저 증가시키고 그 위치에 데이터를 저장한다. 근거 대조 문구: rear가 먼저 증가	(A 5.3 원형큐, 인쇄 p.154 / PDF p.146)
U07-014	A의 원형큐 삭제는 front를 먼저 증가시키고 그 위치의 데이터를 삭제한다. 근거 대조 문구: front가 먼저 증가	(A 5.3 원형큐, 인쇄 p.154 / PDF p.146)
U07-015	원형큐는 포화 상태와 공백 상태를 구분하기 위해 한 자리를 항상 비워 둔다. 근거 대조 문구: 하나의 자리는 항상 비워둔다	(A 5.3 원형큐, 인쇄 p.155 / PDF p.147)
U07-016	요소 개수를 저장하는 추가 변수 count를 사용하면 한 자리를 비워두지 않아도 된다. 근거 대조 문구: count 변수를 사용할 수 있다면	(A 5.3 원형큐, 인쇄 p.155 / PDF p.147)
U07-017	B의 순환 큐에서 Front는 제거할 전단의 위치를, Rear는 실제 후단 위치에 1을 더한 값을 나타낸다. 근거 대조 문구: 실제 후단의 위치 값에 1	(B 3.2.2 순환 큐의 기본 연산, 인쇄 p.124-125 / PDF p.132-133)
U07-018	B는 공백·포화 구별을 위해 실제 용량보다 1만큼 더 큰 큐 배열을 만든다. 근거 대조 문구: 실제 용량보다 1만큼 더 크게	(B 3.2.1 공백 상태와 포화 상태, 인쇄 p.120 / PDF p.128)
U07-019	덱에서도 중간에 삽입하거나 삭제하는 것은 허용하지 않는다. 근거 대조 문구: 중간에 삽입하거나 삭제	(A 5.5 덱이란?, 인쇄 p.162 / PDF p.154)
U07-020	덱의 전단 관련 연산만 사용하면 스택으로 동작한다. 근거 대조 문구: 전단과 관련된 연산들만	(A 5.5 덱의 추상 자료형, 인쇄 p.163 / PDF p.155)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-021	덱에서 후단 삽입과 전단 삭제만 사용하면 큐로 동작한다. 근거 대조 문구: 삽입은 후단, 삭제는 전단	(A 5.5 덱의 추상 자료형, 인쇄 p.163 / PDF p.155)
U07-022	get_front와 get_rear는 각각 앞·뒤의 요소를 삭제하지 않고 반환한다. 근거 대조 문구: 삭제하지 않고	(A 5.5 ADT 5.2 Deque, 인쇄 p.163 / PDF p.155)
U07-023	연결된 큐(linked queue)는 연결 리스트로 만든 큐이며 front는 맨 앞 요소, rear는 맨 뒤 요소를 가리킨다. 근거 대조 문구: 연결된 큐(linked queue)	(A 7.5 연결 리스트로 구현한 큐, 인쇄 p.243 / PDF p.235)
U07-024	연결된 큐에 요소가 없으면 front와 rear는 NULL이다. 근거 대조 문구: front와 rear는 NULL	(A 7.5 연결 리스트로 구현한 큐, 인쇄 p.243 / PDF p.235)
U07-025	연결된 큐는 크기가 제한되지 않는 장점이 있지만 코드가 더 복잡하고 링크 필드의 메모리가 필요하다. 근거 대조 문구: 링크 필드 때문에 메모리 공간	(A 7.5 연결 리스트로 구현한 큐, 인쇄 p.243 / PDF p.235)
U07-026	B의 LinkedQueue 구조체는 Front, Rear, 노드 개수를 나타내는 Count를 갖는다. 근거 대조 문구: 노드 개수를 나타내는 Count	(B 3.3.1 링크드 큐의 기본 연산, 인쇄 p.132 / PDF p.140)
U07-027	큐는 서로 다른 속도로 실행되는 두 프로세스의 상호 작용을 조화시키는 버퍼 역할을 담당한다. 근거 대조 문구: 버퍼 역할을 담당	(A 5.4 큐의 응용: 버퍼, 인쇄 p.160 / PDF p.152)
U07-028	큐잉모델은 서비스를 수행하는 서버와 고객으로 구성되며 대기 행렬을 큐로 구현한다. 근거 대조 문구: 이 대기 행렬이 큐로 구현	(A 5.6 큐의 응용: 시뮬레이션, 인쇄 p.168 / PDF p.160)
U07-029	원문의 은행 시뮬레이션은 행원 한 사람을 가정하고 고객 도착 간격과 서비스 시간을 랜덤하게 정한다. 근거 대조 문구: 행원은 한사람이라고 가정	(A 5.6 큐의 응용: 시뮬레이션, 인쇄 p.168 / PDF p.160)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-030	현재 시각을 나타내는 clock을 증가시킨다. 근거 대조 문구: clock이라는 변수를 하나 증가	(A 5.6 시뮬레이션 반복 루프, 인쇄 p.168 / PDF p.160)
U07-031	새 고객이 들어오면 아이디·도착시간·서비스 시간을 구조체에 저장하여 enqueue한다. 근거 대조 문구: 구조체를 enqueue()	(A 5.6 시뮬레이션 반복 루프, 인쇄 p.168 / PDF p.160)
U07-032	service_time이 0이 아니면 하나 감소시키고, 0이면 큐에서 고객을 꺼내 서비스를 시작한다. 근거 대조 문구: service_time이 0이면	(A 5.6 시뮬레이션 반복 루프, 인쇄 p.168-169 / PDF p.160-161)
U07-033	60분이 지나면 고객들이 기다린 시간을 모두 합하여 출력한다. 근거 대조 문구: 60분의 시간이 지나면	(A 5.6 시뮬레이션 반복 루프, 인쇄 p.169 / PDF p.161)

U07-034	원형큐 공백·포화 검출 함수 (원문 코드 발췌)
	int is_empty(QueueType *q)
	{
	return (q->front == q->rear);
	}
	int is_full(QueueType *q)
	{
	return ((q->rear + 1) % MAX_QUEUE_SIZE == q->front);
	}
	근거 대조 문구: int is_empty(QueueType *q)
		(A 5.3 프로그램 5.2, 인쇄 p.158 / PDF p.150)

U07-035	A의 원형 회전: front ← (front+1) % MAX_QUEUE_SIZE; rear ← (rear+1) % MAX_QUEUE_SIZE;; 배열 끝 다음 인덱스는 0이 된다. 근거 대조 문구: 나머지 연산자 %	(A 5.3 원형큐의 삽입, 삭제 알고리즘, 인쇄 p.156 / PDF p.148)
U07-036	A의 공백·포화 조건: 공백: front == rear; 포화: (rear+1) % MAX_QUEUE_SIZE == front; 한 자리를 비워 두는 원문 구현이다. 근거 대조 문구: (q->rear + 1) % MAX_QUEUE_SIZE == q->front	(A 5.3 프로그램 5.2, 인쇄 p.158 / PDF p.150)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-037	A 원형큐 근거 대조 문구: 원형큐	(A 5.3 원형큐, 인쇄 p.154 / PDF p.146)
U07-038	B 순환 큐 근거 대조 문구: 순환 큐	(B 3.2.2 순환 큐의 기본 연산, 인쇄 p.124 / PDF p.132)
U07-039	연결된 큐 근거 대조 문구: 연결된 큐	(A 7.5 연결 리스트로 구현한 큐, 인쇄 p.243 / PDF p.235)

08 트리·순회·트리 응용

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-001	이진트리는 공집합이거나 루트와 왼쪽·오른쪽 서브 트리로 구성된 유한 집합이며 서브 트리들도 이진트리여야 한다. 근거 대조 문구: 공집합이거나	(A 8.2 정의 8.1 이진트리, 인쇄 p.258 / PDF p.250)
U08-002	이진트리 순회(traversal)는 모든 노드를 한 번씩 방문하여 데이터를 목적에 맞게 처리하는 것이다. 근거 대조 문구: 모든 노드를 한 번씩 방문	(A 8.4 이진 트리의 순회, 인쇄 p.267 / PDF p.259)
U08-003	트리는 한 개 이상의 노드로 이루어진 유한 집합이다. 근거 대조 문구: 한 개 이상의 노드로 이루어진 유한 집합	(A 8.1 트리의 용어들, 인쇄 p.255 / PDF p.247)
U08-004	루트(root)는 계층적인 구조에서 가장 높은 곳에 있는 노드이다. 근거 대조 문구: 가장 높은 곳에 있는 노드	(A 8.1 트리의 용어들, 인쇄 p.255 / PDF p.247)
U08-005	트리에서 루트와 서브트리를 연결하는 선을 간선(edge)이라고 한다. 근거 대조 문구: 이 연결선을 간선(edge)	(A 8.1 트리의 용어들, 인쇄 p.255 / PDF p.247)
U08-006	자식 노드가 없는 노드를 단말노드(terminal node, 또는 leaf node)라고 한다. 근거 대조 문구: 자식 노드가 없는 노드	(A 8.1 트리의 용어들, 인쇄 p.256 / PDF p.248)
U08-007	후손 노드는 임의의 노드 하위에 연결된 모든 노드를 의미한다. 근거 대조 문구: 임의의 노드 하위에 연결된 모든 노드	(A 8.1 트리의 용어들, 인쇄 p.256 / PDF p.248)
U08-008	노드의 차수(degree)는 자식 노드의 개수이며 트리의 차수는 노드 차수 중 가장 큰 값이다. 근거 대조 문구: 자식 노드의 개수	(A 8.1 트리의 용어들, 인쇄 p.256 / PDF p.248)
U08-009	A에서 루트의 레벨은 1이고 한 층씩 내려갈수록 1씩 증가한다. 근거 대조 문구: 루트의 레벨은 1	(A 8.1 트리의 용어들, 인쇄 p.256 / PDF p.248)
U08-010	A에서 트리의 높이(height)는 최대 레벨이다. 근거 대조 문구: 최대 레벨	(A 8.1 트리의 용어들, 인쇄 p.256 / PDF p.248)
U08-011	포리스트(forest)는 트리들의 집합이다. 근거 대조 문구: 트리들의 집합	(A 8.1 트리의 용어들, 인쇄 p.256 / PDF p.248)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-012	B에서 뿌리 노드의 깊이는 0이다. 근거 대조 문구: 뿌리 노드의 깊이	(B 4.1.2 트리의 구성 요소, 인쇄 p.146 / PDF p.154)
U08-013	B에서 레벨은 깊이가 같은 노드의 집합이며, 높이는 가장 깊은 잎 노드까지의 깊이이다. 근거 대조 문구: 깊이가 같은 노드의 집합	(B 4.1.2 트리의 구성 요소, 인쇄 p.146 / PDF p.154)
U08-014	이진트리에는 서브 트리 간 순서가 있어 왼쪽과 오른쪽 서브 트리가 구별된다. 근거 대조 문구: 왼쪽 서브 트리와 오른쪽 서브 트리는 서로 구별	(A 8.2 이진 트리의 정의, 인쇄 p.258 / PDF p.250)
U08-015	포화 이진 트리(full binary tree)는 각 레벨에 노드가 꽉 차 있는 이진 트리이다. 근거 대조 문구: 각 레벨에 노드가 꽉 차있는	(A 8.2 이진트리의 분류, 인쇄 p.261 / PDF p.253)
U08-016	완전 이진 트리(complete binary tree)는 마지막 전 레벨까지 모두 채워지고 마지막 레벨은 왼쪽부터 순서대로 채워진다. 근거 대조 문구: 왼쪽부터 오른쪽으로 노드가 순서대로	(A 8.2 이진트리의 분류, 인쇄 p.262 / PDF p.254)
U08-017	포화 이진트리는 항상 완전 이진트리이지만 그 역은 항상 성립하지 않는다. 근거 대조 문구: 그 역은 항상 성립하지 않는다	(A 8.2 이진트리의 분류, 인쇄 p.262 / PDF p.254)
U08-018	링크 표현법의 이진트리 노드는 데이터 필드와 왼쪽·오른쪽 자식을 가리키는 2개의 포인터 필드를 갖는다. 근거 대조 문구: 2개의 포인터 필드	(A 8.3 링크 표현법, 인쇄 p.264 / PDF p.256)
U08-019	링크 표현의 이진트리는 루트 노드를 가리키는 포인터로 모든 노드에 접근할 수 있다. 근거 대조 문구: 루트 노드를 가리키는 포인터만	(A 8.3 링크 표현법, 인쇄 p.265 / PDF p.257)
U08-020	N-링크 표현법은 차수가 N인 노드에 N개의 자식 링크를 두며, 노드마다 차수가 다른 트리에는 적용하기 힘들다. 근거 대조 문구: 노드가 N개의 링크	(B 4.1.4 노드 표현 방법, 인쇄 p.149 / PDF p.157)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-021	왼쪽 자식-오른쪽 형제 표현법은 왼쪽 자식과 오른쪽 형제에 대한 2 개의 포인터로 노드를 구성한다. 근거 대조 문구: 왼쪽 자식과 오른쪽 형제	(B 4.1.4 노드 표현 방법, 인쇄 p.149 / PDF p.157)
U08-022	전위순회(preorder traversal)의 방문 순서는 VLR이다. 근거 대조 문구: 전위순회(preorder traversal) : VLR	(A 8.4 이진트리 순회방법, 인쇄 p.268 / PDF p.260)
U08-023	중위순회(inorder traversal)의 방문 순서는 LVR이다. 근거 대조 문구: 중위순회(inorder traversal) : LVR	(A 8.4 이진트리 순회방법, 인쇄 p.268 / PDF p.260)
U08-024	후위순회(postorder traversal)의 방문 순서는 LRV이다. 근거 대조 문구: 후위순회(postorder traversal) : LRV	(A 8.4 이진트리 순회방법, 인쇄 p.268 / PDF p.260)
U08-025	각 서브 트리에도 전체 이진트리와 같은 순회 방법을 적용한다. 근거 대조 문구: 똑같은 방법을 적용	(A 8.4 이진트리 순회방법, 인쇄 p.268 / PDF p.260)
U08-026	반복적 트리 순회는 별도의 스택에 자식 노드를 저장하고 꺼내면서 순환 호출을 흉내 낼 수 있다. 근거 대조 문구: 별도의 스택을 사용하면	(A 8.5 반복적 순회, 인쇄 p.277 / PDF p.269)
U08-027	레벨 순회(level order)는 레벨 순으로 방문하고 같은 레벨에서는 좌에서 우로 방문하며 큐를 사용한다. 근거 대조 문구: 레벨 순회는 큐를 사용	(A 8.6 레벨 순회, 인쇄 p.279 / PDF p.271)
U08-028	수식 트리에서 피연산자는 단말노드, 연산자는 비단말 노드가 된다. 근거 대조 문구: 피연산자들은 단말노드	(A 8.7 트리의 응용: 수식 트리 처리, 인쇄 p.283 / PDF p.275)
U08-029	수식 트리를 전위·중위·후위로 읽으면 각각 전위·중위·후위 표기 수식이 된다. 근거 대조 문구: 각각 전위 표기 수식	(A 8.7 트리의 응용: 수식 트리 처리, 인쇄 p.283 / PDF p.275)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-030	수식 트리의 값을 계산하려면 자식 노드를 루트보다 먼저 방문하는 후위 순회를 사용한다. 근거 대조 문구: 후위 순회	(A 8.7 트리의 응용: 수식 트리 처리, 인쇄 p.283 / PDF p.275)
U08-031	B의 수식 트리는 하나의 연산자가 2개의 피연산자를 취한다고 가정한다. 근거 대조 문구: 2개의 피연산자를 취한다는 가정	(B 4.3 수식 트리, 인쇄 p.175 / PDF p.183)
U08-032	A의 이진트리 디렉토리 용량 계산에서는 한 디렉토리 안의 다른 디렉토리가 2개를 초과하면 안 된다. 근거 대조 문구: 2개를 초과하면 안 된다	(A 8.8 트리의 응용: 디렉토리 용량 계산, 인쇄 p.286 / PDF p.278)
U08-033	디렉토리 용량은 서브 디렉토리부터 계산하여 반환하는 후위 순회로 구한다. 근거 대조 문구: 따라서 후위 순회	(A 8.8 트리의 응용: 디렉토리 용량 계산, 인쇄 p.286 / PDF p.278)
U08-034	노드 개수는 왼쪽·오른쪽 서브트리의 노드 개수에 현재 노드 1개를 더하여 구한다. 근거 대조 문구: 반환되는 값에 1을 더하여	(A 8.9 노드의 개수, 인쇄 p.288 / PDF p.280)
U08-035	단말노드 개수는 양쪽 자식이 NULL인 노드에서 1을 반환하고, 비단말 노드에서 양쪽 서브트리의 결과를 더한다. 근거 대조 문구: 반환되는 값을 서로 더하면	(A 8.9 단말 노드 개수 구하기, 인쇄 p.288 / PDF p.280)
U08-036	높이 계산은 각 서브트리의 높이를 구한 뒤 그 최댓값에 1을 더한다. 근거 대조 문구: return 1 + max	(A 8.9 알고리즘 8.8, 인쇄 p.289 / PDF p.281)
U08-037	스레드 이진 트리는 NULL 링크에 중위 선행자나 중위 후속자를 저장하여 순회 순서대로 연결한 트리이다. 근거 대조 문구: 중위 선행자	(A 8.10 스레드 이진 트리, 인쇄 p.290 / PDF p.282)
U08-038	스레드 이진트리에서는 링크가 자식 포인터인지 스레드인지 구별하는 태그 필드가 필요하다. 근거 대조 문구: 구별해주는 태그 필드	(A 8.10 스레드 이진 트리, 인쇄 p.291 / PDF p.283)
U08-039	루트를 큐에 입력한 상태에서 시작한다. 근거 대조 문구: 루트 노드인 +가 큐에 입력	(A 8.6 레벨 순회, 인쇄 p.279 / PDF p.271)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-040	큐의 노드를 꺼내어 방문한 뒤 그 노드의 자식 노드를 큐에 삽입한다. 근거 대조 문구: 그 노드의 자식 노드를 큐에 삽입	(A 8.6 레벨 순회, 인쇄 p.279 / PDF p.271)
U08-041	큐에 노드가 더 이상 없을 때까지 반복한다. 근거 대조 문구: 더 이상의 노드가 없을 때까지	(A 8.6 레벨 순회, 인쇄 p.279 / PDF p.271)
U08-042	수식을 뒤에서 앞쪽으로 읽으며 토큰을 취하고 마지막 토큰으로 뿌리 노드를 생성한다. 근거 대조 문구: 뒤에서부터 앞쪽으로 읽어나가며	(B 4.3.1 수식 트리 구축 방법, 인쇄 p.176 / PDF p.184)
U08-043	연산자 토큰이면 가지 노드를 생성하고 오른쪽 자식과 왼쪽 자식을 구성하며, 다음 토큰도 연산자이면 그 하위 트리를 먼저 완성한다. 근거 대조 문구: 하위 트리가 완성된 이후	(B 4.3.1 수식 트리 구축 방법, 인쇄 p.176 / PDF p.184)
U08-044	숫자 토큰이면 잎 노드를 생성한다. 근거 대조 문구: 숫자인 경우 잎 노드를 생성	(B 4.3.1 수식 트리 구축 방법, 인쇄 p.177 / PDF p.185)
U08-045	이진트리 노드의 링크 표현 (원문 코드 발췌) typedef struct TreeNode { int data; struct TreeNode *left, *right; } TreeNode; 근거 대조 문구: typedef struct TreeNode	(A 8.3 링크 표현법, 인쇄 p.265 / PDF p.257)
U08-046	노드 수와 간선 수: 간선의 개수 = n-1; 노드가 n개인 이진트리. 근거 대조 문구: n-1의 간선	(A 8.2 이진트리의 성질, 인쇄 p.259 / PDF p.251)
U08-047	A의 노드 수·높이 범위: 최소 노드 수 h; 최대 노드 수 2^h-1; 최소 높이 $\lceil \log_2(n+1) \rceil$; 최대 높이 n; h는 A의 높이 정의(루트 레벨 1), n은 노드 수. 근거 대조 문구: 최대 2	(A 8.2 이진트리의 성질, 인쇄 p.260 / PDF p.252)
U08-048	A의 배열 표현 인덱스: 부모: i/2; 왼쪽 자식: 2i; 오른쪽 자식: 2i+1; 루트 인덱스 1인 원문 배열 표현. 근거 대조 문구: 부모 노드 인덱스 = i/2	(A 8.3 배열 표현법, 인쇄 p.264 / PDF p.256)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-049	전위순회 근거 대조 문구: 전위순회	(A 8.4 이진트리 순회방법, 인쇄 p.268 / PDF p.260)
U08-050	중위순회 근거 대조 문구: 중위순회	(A 8.4 이진트리 순회방법, 인쇄 p.268 / PDF p.260)
U08-051	후위순회 근거 대조 문구: 후위순회	(A 8.4 이진트리 순회방법, 인쇄 p.268 / PDF p.260)
U08-052	A 근거 대조 문구: 레벨	(A 8.1 트리의 용어들, 인쇄 p.256 / PDF p.248)
U08-053	B 근거 대조 문구: 레벨	(B 4.1.2 트리의 구성 요소, 인쇄 p.146 / PDF p.154)

09 이진 탐색 트리·균형 트리

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-001	이진 탐색 트리는 모든 원소의 키가 유일한 이진 트리이다. 근거 대조 문구: 모든 원소의 키는 유일한 키를 가진다.	(A 8.11 이진 탐색 트리, 인쇄 p.295 / PDF p.287)
U09-002	AVL 트리는 각 노드에서 왼쪽과 오른쪽 서브 트리의 높이 차이가 1 이하인 이진 탐색 트리이다. 근거 대조 문구: 높이 차이가 1 이하인 이진 탐색 트리	(A 13.5 AVL 트리, 인쇄 p.511 / PDF p.502)
U09-003	레드 블랙 트리는 노드를 빨간색 또는 검은색으로 표시하여 트리 균형을 유지하는 이진 탐색 트리이다. 근거 대조 문구: 노드의 색은 트리 전체의 균형을 유지	(B 6.5 레드 블랙 트리, 인쇄 p.267 / PDF p.274)
U09-004	왼쪽 서브 트리의 키들은 루트 키보다 작고 오른쪽 서브 트리의 키들은 루트 키보다 크다. 근거 대조 문구: 왼쪽 서브 트리 키들은 루트 키보다 작다.	(A 8.11 이진 탐색 트리, 인쇄 p.295 / PDF p.287)
U09-005	왼쪽과 오른쪽 서브 트리도 이진 탐색 트리이다. 근거 대조 문구: 왼쪽과 오른쪽 서브 트리도 이진 탐색 트리이다.	(A 8.11 이진 탐색 트리, 인쇄 p.295 / PDF p.287)
U09-006	이진 탐색 트리를 중위 순회하면 키값이 크기순으로 나온다. 근거 대조 문구: 숫자들의 크기 순이 된다.	(A 8.11 이진 탐색 트리, 인쇄 p.296 / PDF p.288)
U09-007	탐색 키와 현재 노드 키가 같으면 성공하고, 작으면 왼쪽 자식에서, 크면 오른쪽 자식에서 탐색을 다시 시작한다. 근거 대조 문구: 왼쪽 자식을 기준으로 다시 시작	(A 8.11 이진 탐색 트리, 인쇄 p.297 / PDF p.289)
U09-008	이진 탐색 트리의 탐색은 순환적 방법과 반복적 방법으로 구현할 수 있다. 근거 대조 문구: 순환적 방법과 반복적인 방법	(A 8.11 이진 탐색 트리, 인쇄 p.298 / PDF p.290)
U09-009	이진 탐색 트리에 삽입하기 전에 탐색하며 같은 키가 없고 탐색이 실패한 위치가 삽입 위치이다. 근거 대조 문구: 탐색에 실패한 위치가 바로 새로운 노드를 삽입하는 위치	(A 8.11 이진 탐색 트리, 인쇄 p.299 / PDF p.291)
U09-010	새로운 노드는 항상 단말 노드에 추가된다. 근거 대조 문구: 항상 단말노드에 추가	(A 8.11 이진 탐색 트리, 인쇄 p.299 / PDF p.291)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-011	단말 노드를 삭제할 때는 부모의 링크를 NULL로 바꾸어 연결을 끊고 동적으로 생성한 노드라면 메모리를 해제한다. 근거 대조 문구: 링크필드를 NULL로 만들어서 연결을 끊으면	(A 8.11 이진 탐색 트리, 인쇄 p.301 / PDF p.293)
U09-012	삭제 노드가 한 서브 트리만 가지면 그 서브 트리를 삭제 노드의 부모에 붙인다. 근거 대조 문구: 부모 노드에 붙여주면	(A 8.11 이진 탐색 트리, 인쇄 p.302 / PDF p.294)
U09-013	두 서브 트리를 가진 노드의 후계자는 왼쪽 서브 트리의 가장 큰 값이나 오른쪽 서브 트리의 가장 작은 값으로 선택할 수 있다. 근거 대조 문구: 왼쪽 서브트리에서 가장 큰 값이나 오른쪽 서브트리에서 가장 작은 값	(A 8.11 이진 탐색 트리, 인쇄 p.302 / PDF p.294)
U09-014	A의 삭제 구현은 오른쪽 서브 트리의 가장 작은 값을 후계자로 선택한다. 근거 대조 문구: 오른쪽 서브 트리에서 제일 작은 값을 후계자	(A 8.11 이진 탐색 트리, 인쇄 p.303 / PDF p.295)
U09-015	B는 오른쪽 하위 트리의 최솟값 노드를 삭제된 노드 위치로 옮기며, 최솟값 노드의 오른쪽 자식은 원래 부모에 연결한다. 근거 대조 문구: 최솟값 노드의 오른쪽 자식을 최솟값 노드의 원래 부모에게 연결	(B 6.4.2 이진 탐색 트리의 기본 연산, 인쇄 p.257 / PDF p.264)
U09-016	트리 높이를 h라고 하면 탐색·삽입·삭제의 시간 복잡도는 O(h)이다. 근거 대조 문구: 트리의 높이를 h라고 했을 때 O(h)	(A 8.11 이진 탐색 트리, 인쇄 p.309 / PDF p.301)
U09-017	한쪽으로 치우친 경사 트리의 최악 경우에는 높이가 n이 되어 탐색·삽입·삭제 시간이 O(n)이 된다. 근거 대조 문구: 트리의 높이가 n	(A 8.11 이진 탐색 트리, 인쇄 p.309 / PDF p.301)
U09-018	A의 영어 사전 예제는 입력에 insert_node(), 탐색에 search(), 삭제에 delete_node()를 사용한다. 근거 대조 문구: 가장 중요한 메뉴는 입력과 탐색, 삭제	(A 8.12 이진 탐색 트리의 응용: 영어 사전, 인쇄 p.310 / PDF p.302)
U09-019	균형 인수(Balance Factor)는 왼쪽 서브 트리 높이에서 오른쪽 서브 트리 높이를 뺀 값이다. 근거 대조 문구: 왼쪽 서브 트리의 높이-오른쪽 서브 트리의 높이	(A 13.5 AVL 트리, 인쇄 p.511 / PDF p.502)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-020	AVL 트리는 비균형 상태에서 노드를 재배치하여 균형 상태로 만든다. 근거 대조 문구: 스스로 노드들을 재배치하여 균형 상태	(A 13.5 AVL 트리, 인쇄 p.511 / PDF p.502)
U09-021	A는 AVL 트리의 탐색과 삽입·삭제가 O(logn) 시간 안에 가능하다고 설명한다. 근거 대조 문구: 삽입과 삭제 연산도 O(logn)시간	(A 13.5 AVL 트리, 인쇄 p.511 / PDF p.502)
U09-022	AVL 탐색은 일반적인 이진 탐색 트리와 동일하다. 근거 대조 문구: 일반적인 이진 탐색 트리와 동일	(A 13.5 AVL 트리, 인쇄 p.512 / PDF p.503)
U09-023	AVL 삽입 후에는 새 노드에서 가장 가까우면서 균형 인수가 ±2가 된 조상 노드의 서브 트리를 회전하여 다시 균형을 잡는다. 근거 대조 문구: 가장 가까운 조상 노드	(A 13.5 AVL 트리, 인쇄 p.512 / PDF p.503)
U09-024	LL 타입은 왼쪽 자식의 왼쪽에 삽입한 경우이고 오른쪽 회전으로 처리한다. 근거 대조 문구: 노드들을 오른쪽으로 회전	(A 13.5 AVL 트리, 인쇄 p.514 / PDF p.505)
U09-025	RR 타입은 오른쪽 자식의 오른쪽에 삽입한 경우이고 왼쪽 회전으로 처리한다. 근거 대조 문구: 노드들을 왼쪽으로 회전	(A 13.5 AVL 트리, 인쇄 p.514 / PDF p.505)
U09-026	A의 LR 처리 코드는 왼쪽 자식을 왼쪽으로 회전한 뒤 현재 노드를 오른쪽으로 회전한다. 근거 대조 문구: node->left = rotate_left(node->left);	(A 13.5 AVL 트리, 인쇄 p.520 / PDF p.511)
U09-027	A의 RL 처리 코드는 오른쪽 자식을 오른쪽으로 회전한 뒤 현재 노드를 왼쪽으로 회전한다. 근거 대조 문구: node->right = rotate_right(node->right);	(A 13.5 AVL 트리, 인쇄 p.520 / PDF p.511)
U09-028	A의 AVL 구현은 이진 탐색 트리 방식으로 삽입한 뒤 균형 인수를 다시 계산하고 회전 유형을 처리한다. 근거 대조 문구: 노드들의 균형인수 재계산	(A 13.5 AVL 트리, 인쇄 p.519-520 / PDF p.510-511)
U09-029	2-3 트리는 차수가 2 또는 3인 노드를 가지는 트리이다. 근거 대조 문구: 차수가 2 또는 3인 노드	(A 13.6 2-3 트리, 인쇄 p.523 / PDF p.514)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-030	2-노드는 데이터 k1 하나와 자식 노드 둘을 가진다. 근거 대조 문구: 하나의 데이터 k1과 두 개의 자식 노드	(A 13.6 2-3 트리, 인쇄 p.523 / PDF p.514)
U09-031	3-노드는 데이터 k1, k2 둘과 자식 노드 셋을 가진다. 근거 대조 문구: 2개의 데이터 k1, k2와 3개의 자식	(A 13.6 2-3 트리, 인쇄 p.523-524 / PDF p.514-515)
U09-032	3-노드의 왼쪽 데이터는 k1보다 작고, 중간 데이터는 k1보다 크고 k2보다 작으며, 오른쪽 데이터는 k2보다 크다. 근거 대조 문구: 중간 서브 트리에 있는 값들은 모두 k1보다 크고 k2보다 작다.	(A 13.6 2-3 트리, 인쇄 p.524 / PDF p.515)
U09-033	2-3 트리는 저장할 장소가 없으면 중간값을 한 레벨 올리고 작은 값과 큰 값을 별개 노드로 분리한다. 근거 대조 문구: 중간값을 한레벨위로 올리고	(A 13.6 2-3 트리, 인쇄 p.525 / PDF p.516)
U09-034	부모도 추가 데이터를 받을 공간이 없으면 부모에 대해 분리 과정을 되풀이한다. 근거 대조 문구: 분리 과정이 부모 노드에 대하여 되풀이	(A 13.6 2-3 트리, 인쇄 p.526 / PDF p.517)
U09-035	2-3 트리에서 높이가 증가하는 경우는 루트를 분리하여 새 루트가 생길 때뿐이다. 근거 대조 문구: 높이가 증가하게 되는 것은 오직 이 경우	(A 13.6 2-3 트리, 인쇄 p.527 / PDF p.518)
U09-036	2-3-4 트리는 노드가 자식을 최대 4개 갖도록 2-3 트리를 확장한 것이다. 근거 대조 문구: 4개의 자식까지 가질 수 있도록	(A 13.7 2-3-4 트리, 인쇄 p.528 / PDF p.519)
U09-037	4-노드는 데이터 셋과 자식 넷을 가질 수 있다. 근거 대조 문구: 3개의 데이터를 가질 수 있다.	(A 13.7 2-3-4 트리, 인쇄 p.528 / PDF p.519)
U09-038	2-3-4 트리는 삽입 위치를 찾으러 루트에서 단말로 내려갈 때 4-노드를 만나면 미리 분할하여 후진 분할을 막는다. 근거 대조 문구: 4-노드를 만나면 미리 분할	(A 13.7 2-3-4 트리, 인쇄 p.529 / PDF p.520)
U09-039	단말 노드가 2-노드 또는 3-노드이면 단순히 삽입하면 된다. 근거 대조 문구: 2-노드 또는 3-노드이면 간단하게 삽입	(A 13.7 2-3-4 트리, 인쇄 p.529 / PDF p.520)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-040	A는 2-3-4 트리가 루트에서 단말로 한 번만 이동하면서 삽입이나 삭제할 수 있다고 설명한다. 근거 대조 문구: 한번만 이동하면서 삽입이나 삭제가 가능	(A 13.7 2-3-4 트리, 인쇄 p.529 / PDF p.520)
U09-041	B의 RBTNode는 부모를 가리키는 Parent와 Color 필드를 가지며 색 값은 RED 또는 BLACK이다. 근거 대조 문구: RED 아니면 BLACK	(B 6.5 레드 블랙 트리, 인쇄 p.267 / PDF p.274)
U09-042	레드 블랙 트리의 모든 노드는 빨간색이거나 검은색이다. 근거 대조 문구: 모든 노드는 빨간색이거나 검은색	(B 6.5.1 레드 블랙 트리의 구현 규칙, 인쇄 p.268 / PDF p.275)
U09-043	레드 블랙 트리의 뿌리 노드와 잎 노드는 검은색이다. 근거 대조 문구: 뿌리 노드는 검은색이다.	(B 6.5.1 레드 블랙 트리의 구현 규칙, 인쇄 p.268 / PDF p.275)
U09-044	빨간색 노드의 자식은 모두 검은색이다. 근거 대조 문구: 빨간색 노드의 자식은 모두 검은색	(B 6.5.1 레드 블랙 트리의 구현 규칙, 인쇄 p.268 / PDF p.275)
U09-045	뿌리와 모든 잎 노드 사이의 검은색 노드 수는 모두 동일하다. 근거 대조 문구: 검은색 노드의 수는 모두 동일	(B 6.5.1 레드 블랙 트리의 구현 규칙, 인쇄 p.268 / PDF p.275)
U09-046	NIL은 데이터 없이 검은색만 갖는 더미 노드이며 센티넬(Sentinel) 노드라고 한다. 근거 대조 문구: 센티넬Sentinel	(B 6.5.1 레드 블랙 트리의 구현 규칙, 인쇄 p.268-269 / PDF p.275-276)
U09-047	레드 블랙 트리의 탐색은 이진 탐색 트리의 탐색 알고리즘을 그대로 사용하며 삽입·삭제 후에는 규칙을 바로잡는 뒤처리가 필요하다. 근거 대조 문구: 삽입과 삭제 연산 후의 뒤처리	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.269 / PDF p.276)
U09-048	회전은 부모와 자식 노드의 위치를 바꾸는 연산이고 우회전은 왼쪽 자식, 좌회전은 오른쪽 자식과 부모 위치를 교환한다. 근거 대조 문구: 부모와 자식 노드의 위치를 서로 바꾸는 연산	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.269 / PDF p.276)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-049	우회전할 때는 왼쪽 자식의 오른쪽 자식을 부모의 왼쪽 자식으로 연결한다. 근거 대조 문구: 왼쪽 자식 노드의 오른쪽 자식 노드	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.270 / PDF p.277)
U09-050	좌회전할 때는 오른쪽 자식의 왼쪽 자식을 부모의 오른쪽 자식으로 연결한다. 근거 대조 문구: 오른쪽 자식 노드의 왼쪽 자식 노드	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.270 / PDF p.277)
U09-051	새 노드를 이진 탐색으로 삽입한 뒤 빨간색으로 칠하고 양쪽 자식에 NIL을 연결하여 규칙을 복구한다. 근거 대조 문구: 새 노드를 빨간색으로 칠한 다음 NIL	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.271 / PDF p.278)
U09-052	B의 구현은 저장 공간을 줄이기 위해 전역 NIL 노드 하나를 생성하여 여러 자식 링크에서 함께 사용한다. 근거 대조 문구: NIL 노드를 전역으로 한 개만 생성	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.272 / PDF p.279)
U09-053	삽입 복구에서는 뿌리 노드를 무조건 검은색으로 칠한다. 근거 대조 문구: 뿌리 노드를 무조건 검은색으로	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.273 / PDF p.280)
U09-054	삽입 노드의 부모가 할아버지의 왼쪽 자식이고 삼촌도 빨간색이면 부모·삼촌을 검은색, 할아버지를 빨간색으로 칠한다. 근거 대조 문구: 부모 노드와 삼촌 노드를 검은색으로	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.273 / PDF p.280)
U09-055	같은 부모 방향에서 삼촌이 검은색이고 새 노드가 오른쪽 자식이면 부모를 좌회전하여 왼쪽 자식인 경우로 바꾼다. 근거 대조 문구: 부모 노드를 좌회전시켜	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.274 / PDF p.281)
U09-056	같은 부모 방향에서 삼촌이 검은색이고 새 노드가 왼쪽 자식이면 부모를 검은색, 할아버지를 빨간색으로 칠하고 할아버지를 우회전한다. 근거 대조 문구: 할아버지 노드를 우회전	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.274 / PDF p.281)
U09-057	부모가 할아버지의 오른쪽 자식이면 삽입 복구 설명에서 왼쪽과 오른쪽을 바꾼다. 근거 대조 문구: 왼쪽과 오른쪽을 바꿔주면	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.273 / PDF p.280)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-058	빨간색 노드 삭제는 색 규칙을 무너뜨리지 않지만 이진 탐색 트리로서의 뒤처리는 진행한다. 근거 대조 문구: 이진 탐색 트리로서의 뒤처리는 진행	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.277 / PDF p.284)
U09-059	검은색 노드를 삭제하면 해당 경로의 검은색 노드 수가 하나 적어져 경로 수 규칙을 위반한다. 근거 대조 문구: 하나 더 적어지기 때문	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.277 / PDF p.284)
U09-060	이중 흑색은 대체 노드가 검은색을 두 개 가진다고 표현하는 개념이며 실제 코드에서 별도로 표시하지 않는다. 근거 대조 문구: 실제 코드에서 노드를 이중 흑색으로 표시하는 일은 없	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.278-279 / PDF p.285-286)
U09-061	이중 흑색 노드가 부모의 왼쪽 자식인 경우 형제·조카 색으로 네 경우를 나누며 오른쪽 자식인 경우는 좌우를 바꾼다. 근거 대조 문구: 네 가지 경우로 나뉩니다.	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.279 / PDF p.286)
U09-062	형제가 빨간색이면 형제를 검은색, 부모를 빨간색으로 칠하고 부모를 좌회전하여 검은색 형제의 경우로 바꾼다. 근거 대조 문구: 형제를 검은색, 부모를 빨간색	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.279-280 / PDF p.286-287)
U09-063	형제가 검은색이고 양쪽 자식도 검은색이면 형제를 빨간색으로 칠하고 이중 흑색 중 하나를 부모에게 넘긴다. 근거 대조 문구: 형제 노드만 빨간색으로 칠한 후	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.280 / PDF p.287)
U09-064	형제가 검은색이고 왼쪽 자식이 빨간색·오른쪽이 검은색이면 형제를 빨간색, 왼쪽 자식을 검은색으로 칠하고 형제를 우회전한다. 근거 대조 문구: 형제 노드를 기준으로 우회전	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.281 / PDF p.288)
U09-065	형제가 검은색이고 오른쪽 자식이 빨간색이면 형제에 부모 색을 칠하고 부모·형제의 오른쪽 자식을 검은색으로 칠한 뒤 부모를 좌회전한다. 근거 대조 문구: 부모 노드를 기준으로 좌회전	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.282 / PDF p.289)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-066	뿌리 노드가 이중 흑색이면 검은색으로 칠해 처리를 끝낸다. 근거 대조 문구: 검은색으로 칠해주기만 하면	(B 6.5.2 레드 블랙 트리의 기본 연산, 인쇄 p.282 / PDF p.289)
U09-067	구현 코드에서 LL·RR은 단일 회전, LR·RL은 두 회전 함수를 이어서 처리한다. 근거 대조 문구: // LR 타입 처리	(A 13.5 AVL 트리, 인쇄 p.520 / PDF p.511)

U09-068	A 프로그램 8.9 순환 탐색 함수 (원문 코드 발췌)
	TreeNode * search(TreeNode * node, int key)
	{
	if (node == NULL) return NULL;
	if (key == node->key) return node;
	else if (key < node->key)
	return search(node->left, key);
	else
	return search(node->right, key);
	}
	근거 대조 문구: TreeNode * search(TreeNode * node, int key)
		(A 8.11 이진 탐색 트리, 인쇄 p.298 / PDF p.290)

U09-069	A rotate_left() 원문 함수 (원문 코드 발췌)
	AVLNode *rotate_left(AVLNode *parent)
	{
	AVLNode *child = parent->right;
	parent->right = child->left;
	child->left = parent;
	
	// 새로운 루트 반환
	return child;
	}
	근거 대조 문구: AVLNode *rotate_left(AVLNode *parent)
		(A 13.5 AVL 트리, 인쇄 p.518 / PDF p.509)

U09-070	이진 탐색 트리 연산: O(h); h는 트리 높이이다. 경사 트리의 최악 경우는 h=n, O(n)이다. 근거 대조 문구: O(h)	(A 8.11 이진 탐색 트리, 인쇄 p.309 / PDF p.301)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-071	AVL 균형 인수: 균형 인수 = 왼쪽 서브 트리의 높이 – 오른쪽 서브 트리의 높이; AVL은 각 노드의 두 서브 트리 높이 차이가 1 이하이다. 근거 대조 문구: 왼쪽 서브 트리의 높이-오른쪽 서브 트리의 높이	(A 13.5 AVL 트리, 인쇄 p.511 / PDF p.502)
U09-072	AVL 본문의 연산 시간: O(logn); A가 설명한 균형 보장 AVL의 탐색·삽입·삭제 시간. 근거 대조 문구: 삽입과 삭제 연산도 O(logn)시간	(A 13.5 AVL 트리, 인쇄 p.511 / PDF p.502)
U09-073	이진 탐색 트리 근거 대조 문구: 이진 탐색 트리	(A 8.11, 인쇄 p.295 / PDF p.287)
U09-074	AVL 트리 근거 대조 문구: AVL	(A 13.5, 인쇄 p.511 / PDF p.502)
U09-075	2-3 트리 근거 대조 문구: 2-3	(A 13.6, 인쇄 p.523 / PDF p.514)
U09-076	2-3-4 트리 근거 대조 문구: 2-3-4	(A 13.7, 인쇄 p.528 / PDF p.519)
U09-077	레드 블랙 트리 근거 대조 문구: 레드 블랙	(B 6.5, 인쇄 p.267 / PDF p.274)
U09-078	2-노드 근거 대조 문구: 2-노드	(A 13.6, 인쇄 p.523 / PDF p.514)
U09-079	3-노드 근거 대조 문구: 3-노드	(A 13.6, 인쇄 p.523 / PDF p.514)
U09-080	4-노드 근거 대조 문구: 4-노드	(A 13.7, 인쇄 p.528 / PDF p.519)

10 우선순위 큐·힙·허프만

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-001	우선순위 큐는 우선순위 속성을 갖는 데이터의 삽입(Enqueue)과 제거(Dequeue) 연산을 지원하는 ADT이다. 근거 대조 문구: 삽입과 제거 연산을 지원하는 ADT	(B 7.1 우선순위 큐, 인쇄 p.305 / PDF p.310)
U10-002	히프(heap)는 여러 개의 값 중에서 가장 큰 값이나 가장 작은 값을 빠르게 찾아내도록 만들어진 자료구조이다. 근거 대조 문구: 가장 큰 값이나 가장 작은 값	(A 9.3 히프란?, 인쇄 p.327 / PDF p.319)
U10-003	우선순위 큐는 가장 높은 우선순위를 가진 요소부터 빠져나오게 한다. 근거 대조 문구: 가장 높은 우선순위를 가진 요소부터	(B 7.1 우선순위 큐, 인쇄 p.305 / PDF p.310)
U10-004	우선순위의 기준은 프로그래머가 결정한다. 근거 대조 문구: 전적으로 프로그래머에게 달렸습니다	(B 7.1 우선순위의 기준, 인쇄 p.305 / PDF p.310)
U10-005	B에서는 데이터의 값이 작을수록 우선순위가 높은 것으로 간주한다. 근거 대조 문구: 데이터의 값이 작을수록 높은	(B 7.1 우선순위의 기준, 인쇄 p.305 / PDF p.310)
U10-006	우선순위 큐는 시뮬레이션 시스템, 네트워크 트래픽 제어, 운영 체제의 작업 스케줄링, 수치 해석적인 계산에 사용된다. 근거 대조 문구: 네트워크 트래픽 제어	(A 9.1 우선순위 큐, 인쇄 p.323 / PDF p.315)
U10-007	히프는 완전 이진트리이다. 근거 대조 문구: 완전 이진트리	(A 9.3 히프란?, 인쇄 p.327 / PDF p.319)
U10-008	히프는 부모 노드의 키 값이 자식 노드의 키 값보다 큰 느슨한 정렬 상태를 유지한다. 근거 대조 문구: 느슨한 정렬 상태	(A 9.3 히프란?, 인쇄 p.327 / PDF p.319)
U10-009	히프 트리에서는 중복된 값을 허용한다. 근거 대조 문구: 중복된 값을 허용	(A 9.3 히프란?, 인쇄 p.327 / PDF p.319)
U10-010	최대 히프(max heap)는 부모 노드의 키 값이 자식 노드의 키 값보다 크거나 같은 완전 이진트리이다. 근거 대조 문구: 크거나 같은 완전 이진트리	(A 9.3 최대 히프, 인쇄 p.328 / PDF p.320)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-011	최소 히프(min heap)는 부모 노드의 키 값이 자식 노드의 키 값보다 작거나 같은 완전 이진트리이다. 근거 대조 문구: 작거나 같은 완전 이진트리	(A 9.3 최소 히프, 인쇄 p.328 / PDF p.320)
U10-012	B의 힙에서 가장 작은 데이터를 갖는 노드는 뿌리 노드이다. 근거 대조 문구: 가장 작은 데이터를 갖는 노드는 뿌리	(B 7.2.2 힙의 최솟값 삭제 연산, 인쇄 p.310 / PDF p.315)
U10-013	B의 힙은 이진 탐색을 할 수 없으며 어떤 노드를 찾으려면 모든 노드를 순회해야 한다. 근거 대조 문구: 모든 노드를 순회	(B 7.2 힙, 인쇄 p.307 / PDF p.312)
U10-014	A의 배열 표현은 인덱스 0을 사용하지 않고 루트 노드를 인덱스 1에 저장한다. 근거 대조 문구: 인덱스는 0이 아닌 1부터	(A 9.3 히프의 구현 방법, 인쇄 p.328 / PDF p.320)
U10-015	B의 배열 표현은 깊이 0의 뿌리 노드를 배열의 0번 요소에 저장한다. 근거 대조 문구: 배열의 0번 요소에 저장	(B 7.2.3 힙의 구현, 인쇄 p.312 / PDF p.317)
U10-016	B의 Heap 구조체에서 Capacity는 Nodes 배열의 크기, UsedSize는 실제 배열에 들어 있는 큐 요소의 수이다. 근거 대조 문구: Capacity 필드는 힙의 용량	(B 7.2.3 자료구조 선언, 인쇄 p.313 / PDF p.318)
U10-017	B의 HEAP_Insert()는 UsedSize가 Capacity에 도달하면 Capacity를 2배로 늘리고 realloc()으로 배열을 다시 할당한다. 근거 대조 문구: Capacity *= 2	(B 7.2.3 삽입 연산 구현, 인쇄 p.314 / PDF p.319)
U10-018	B의 최솟값 삭제 구현은 UsedSize가 Capacity/2보다 작으면 Capacity를 반으로 줄이고 realloc()을 호출한다. 근거 대조 문구: H->Capacity /= 2	(B 7.2.3 최솟값 삭제 연산 구현, 인쇄 p.316 / PDF p.321)
U10-019	B의 힙 기반 우선순위 큐는 노드 구조체에 Priority 필드를 추가하고 Data 필드를 void*로 바꾸어 다양한 자료형을 담는다. 근거 대조 문구: Priority 필드 추가	(B 7.3 힙 기반 우선순위 큐의 구현, 인쇄 p.322 / PDF p.327)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-020	최대 히프 삽입에서는 마지막 노드에 새 노드를 추가한 뒤 부모 노드와 비교하여 교환하며 올라간다. 근거 대조 문구: PARENT(i)	(A 9.4 알고리즘 9.1, 인쇄 p.333 / PDF p.325)
U10-021	최대 히프 삭제에서는 루트를 삭제하고 마지막 노드를 루트로 옮긴 뒤 자식과 비교하여 교환하며 내려간다. 근거 대조 문구: 마지막 노드를 루트 노드로	(A 9.4 삭제 알고리즘, 인쇄 p.335 / PDF p.327)
U10-022	히프의 삽입과 삭제 연산은 최악의 경우 히프의 높이만큼 비교하므로 $O(\log_2 n)$이다. 근거 대조 문구: O(log	(A 9.4 삽입과 삭제 연산의 복잡도 분석, 인쇄 p.340 / PDF p.332)
U10-023	히프 정렬은 히프의 삽입과 삭제 연산을 이용하여 정렬하며 전체 시간 복잡도는 $O(n\log_2 n)$이다. 근거 대조 문구: O(nlog	(A 9.5 히프 정렬, 인쇄 p.342 / PDF p.334)
U10-024	히프 정렬은 전체 자료를 정렬하지 않고 가장 큰 값 몇 개만 필요할 때 유용하다. 근거 대조 문구: 가장 큰 값 몇 개만 필요	(A 9.5 히프 정렬, 인쇄 p.342 / PDF p.334)
U10-025	기계 스케줄링 문제는 동일한 기계 m개에 서로 다른 작업 시간의 작업 n개를 할당하여 전체 작업을 가장 빨리 끝내도록 하는 문제이다. 근거 대조 문구: 동일한 기계 m개	(A 9.6 히프의 응용: 기계 스케줄링, 인쇄 p.343 / PDF p.335)
U10-026	LPT(longest processing time first)는 작업 시간이 가장 긴 작업을 먼저 할당하는 근사해 방법이다. 근거 대조 문구: 근사해	(A 9.6 히프의 응용: 기계 스케줄링, 인쇄 p.343 / PDF p.335)
U10-027	LPT에서는 기계가 사용 가능해질 때마다 남은 작업 중 시간이 가장 긴 작업을 할당한다. 근거 대조 문구: 기계가 사용 가능하게 될 때마다	(A 9.6 히프의 응용: 기계 스케줄링, 인쇄 p.343 / PDF p.335)
U10-028	기계의 종료 시간을 최소 히프에 저장하여 가장 먼저 사용 가능해지는 기계를 선택한다. 근거 대조 문구: 종료 시간을 최소 히프에 저장	(A 9.6 히프의 응용: 기계 스케줄링, 인쇄 p.344 / PDF p.336)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-029	빈도수(frequencies)는 영문 텍스트에서 해당 글자가 나타나는 횟수이다. 근거 대조 문구: 해당 글자가 나타나는 횟수	(A 9.7 히프의 응용: 허프만 코드, 인쇄 p.348 / PDF p.340)
U10-030	빈도수에 따라 자주 등장하는 글자에는 짧은 비트열을, 잘 나오지 않는 글자에는 긴 비트열을 사용하는 가변 길이 코드로 전체 크기를 줄인다. 근거 대조 문구: 가변 길이의 코드를 사용	(A 9.7 히프의 응용: 허프만 코드, 인쇄 p.348 / PDF p.340)
U10-031	허프만 코드(Huffman codes)에서는 모든 코드가 다른 코드의 첫 부분이 아니므로 왼쪽에서 오른쪽으로 조사하면 정확히 하나의 코드만 일치한다. 근거 대조 문구: 모든 코드가 다른 코드의 첫 부분이 아니라는	(A 9.7 히프의 응용: 허프만 코드, 인쇄 p.349 / PDF p.341)
U10-032	A의 허프만 트리에서 왼쪽 간선은 비트 1을, 오른쪽 간선은 비트 0을 나타낸다. 근거 대조 문구: 왼쪽 간선은 비트 1	(A 9.7 히프의 응용: 허프만 코드, 인쇄 p.350 / PDF p.342)
U10-033	히프의 크기를 1 증가시키고 새 원소를 마지막 위치에 둔다. 근거 대조 문구: heap_size	(A 9.4 알고리즘 9.1, 인쇄 p.333 / PDF p.325)
U10-034	새 원소가 루트가 아니며 부모의 키보다 크면 부모와 교환한다. 근거 대조 문구: A[i] > A[PARENT(i)]	(A 9.4 알고리즘 9.1, 인쇄 p.333 / PDF p.325)
U10-035	비교 위치를 부모 위치로 옮겨 같은 과정을 반복한다. 근거 대조 문구: i ← PARENT(i)	(A 9.4 알고리즘 9.1, 인쇄 p.333 / PDF p.325)
U10-036	뿌리 노드를 삭제한 후 최고 깊이의 가장 우측 노드를 뿌리로 옮긴다. 근거 대조 문구: 최고 깊이 가장 우측	(B 7.2.2 힙의 최솟값 삭제 연산, 인쇄 p.310 / PDF p.315)
U10-037	옮겨온 노드의 양쪽 자식을 비교하여 작은 쪽 자식과 위치를 교환한다. 근거 대조 문구: 작은 쪽 자식과 위치를 교환	(B 7.2.2 힙의 최솟값 삭제 연산, 인쇄 p.310 / PDF p.315)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-038	옮겨온 노드가 잎 노드가 되거나 양쪽 자식보다 작으면 종료하고, 그렇지 않으면 교환을 반복한다. 근거 대조 문구: 그렇지 않으면 단계	(B 7.2.2 힙의 최솟값 삭제 연산, 인쇄 p.310 / PDF p.315)
U10-039	빈도수에 따라 글자를 나열하고 가장 작은 빈도수의 글자 2개를 추출한다. 근거 대조 문구: 가장 작은 빈도수를 가지는 글자 2개	(A 9.7 히프의 응용: 허프만 코드, 인쇄 p.349 / PDF p.341)
U10-040	두 글자를 단말노드로 하는 이진트리를 만들고 루트에 두 자식의 값의 합을 저장한다. 근거 대조 문구: 루트의 값은 각 자식노드의 값을 합한 값	(A 9.7 히프의 응용: 허프만 코드, 인쇄 p.349 / PDF p.341)
U10-041	합친 값을 글자들의 정렬된 리스트에 다시 삽입하고 가장 작은 값 2개를 합치는 과정을 최종 트리가 될 때까지 반복한다. 근거 대조 문구: 합쳐진 값을 글자들의 리스트에 삽입	(A 9.7 히프의 응용: 허프만 코드, 인쇄 p.350 / PDF p.342)
U10-042	B의 Heap 구조체 (원문 코드 발췌) typedef struct tagHeap { HeapNode* Nodes; int Capacity; int UsedSize; } Heap; 근거 대조 문구: typedef struct tagHeap	(B 7.2.3 자료구조 선언, 인쇄 p.313 / PDF p.318)
U10-043	최대·최소 히프의 키 관계: 최대 히프: key(부모노드) ≥ key(자식노드); 최소 히프: key(부모노드) ≤ key(자식노드); 두 히프 모두 완전 이진트리. 근거 대조 문구: key(부모노드)	(A 9.3 히프란?, 인쇄 p.328 / PDF p.320)
U10-044	A의 배열 인덱스 관계: 왼쪽 자식 = 부모 인덱스*2; 오른쪽 자식 = 부모 인덱스*2+1; 부모 = 자식 인덱스/2; 루트 인덱스 1을 사용하는 배열 표현. 근거 대조 문구: 부모 노드의 인덱스	(A 9.3 히프의 구현 방법, 인쇄 p.329 / PDF p.321)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-045	B의 배열 인덱스 관계: 왼쪽 자식: 2k+1; 오른쪽 자식: 2k+2; 부모: (k-1)÷2의 몫; k는 노드의 인덱스이며, 루트 인덱스는 0. 근거 대조 문구: (k-1)÷2의 몫	(B 7.2.3 힙의 구현, 인쇄 p.313 / PDF p.318)
U10-046	순서 없는 배열 근거 대조 문구: 순서 없는 배열	(A 9.2 표 9-2 우선순위 큐의 구현 방법 비교, 인쇄 p.326 / PDF p.318)
U10-047	O(1) 근거 대조 문구: O(1)	(A 9.2 표 9-2 우선순위 큐의 구현 방법 비교, 인쇄 p.326 / PDF p.318)
U10-048	O(n) 근거 대조 문구: O(n)	(A 9.2 표 9-2 우선순위 큐의 구현 방법 비교, 인쇄 p.326 / PDF p.318)
U10-049	순서 없는 연결 리스트 근거 대조 문구: 순서 없는 연결 리스트	(A 9.2 표 9-2 우선순위 큐의 구현 방법 비교, 인쇄 p.326 / PDF p.318)
U10-050	정렬된 배열 근거 대조 문구: 정렬된 배열	(A 9.2 표 9-2 우선순위 큐의 구현 방법 비교, 인쇄 p.326 / PDF p.318)
U10-051	정렬된 연결 리스트 근거 대조 문구: 정렬된 연결 리스트	(A 9.2 표 9-2 우선순위 큐의 구현 방법 비교, 인쇄 p.326 / PDF p.318)
U10-052	히프 근거 대조 문구: 히프	(A 9.2 표 9-2 우선순위 큐의 구현 방법 비교, 인쇄 p.326 / PDF p.318)
U10-053	O(logn) 근거 대조 문구: O(logn)	(A 9.2 표 9-2 우선순위 큐의 구현 방법 비교, 인쇄 p.326 / PDF p.318)
U10-054	A 근거 대조 문구: 인덱스	(A 9.3 히프의 구현 방법, 인쇄 p.328 / PDF p.320)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-055	B 근거 대조 문구: 0번 요소	(B 7.2.3 힙의 구현, 인쇄 p.312 / PDF p.317)

11 그래프·순회·위상 정렬·분리 집합

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-001	그래프는 정점(vertex)과 간선(edge)들의 유한 집합이며 G = (V, E)로 표시한다. 근거 대조 문구: 그래프는 정점(vertex)와 간선(edge)들의 유한 집합	(A 10.2 그래프의 정의와 용어, 인쇄 p.363-364 / PDF p.355-356)
U11-002	방향 그래프의 각 정점들의 선행 순서를 위배하지 않으면서 모든 정점을 나열하는 것을 위상 정렬(topological sort)이라 한다. 근거 대조 문구: 선행 순서를 위배하지 않으면서	(A 11.7 위상 정렬, 인쇄 p.431 / PDF p.423)
U11-003	정점은 여러 가지 특성을 가질 수 있는 객체이며 간선은 정점들 간의 관계를 의미한다. 근거 대조 문구: 간선은 이러한 정점들 간의 관계를 의미한다	(A 10.2 그래프의 정의와 용어, 인쇄 p.364 / PDF p.356)
U11-004	무방향 그래프의 간선 (A, B)와 (B, A)는 동일한 간선이다. 근거 대조 문구: (A, B)와 (B, A)는 동일한 간선	(A 10.2 그래프의 정의와 용어, 인쇄 p.364 / PDF p.356)
U11-005	방향 그래프의 간선 <A, B>와 <B, A>는 서로 다른 간선이다. 근거 대조 문구: 서로 다른 간선이다	(A 10.2 그래프의 정의와 용어, 인쇄 p.364 / PDF p.356)
U11-006	간선에 비용이나 가중치가 할당된 그래프를 가중치 그래프(weighted graph) 또는 네트워크(network)라 한다. 근거 대조 문구: 가중치 그래프	(A 10.2 그래프의 정의와 용어, 인쇄 p.365 / PDF p.357)
U11-007	어떤 그래프의 정점의 일부와 간선의 일부로 이루어진 그래프를 부분 그래프(subgraph)라 한다. 근거 대조 문구: 정점의 일부와 간선의 일부	(A 10.2 그래프의 정의와 용어, 인쇄 p.365 / PDF p.357)
U11-008	인접 정점(adjacent vertex)은 간선에 의해 직접 연결된 정점이다. 근거 대조 문구: 간선에 의해 직접 연결된 정점	(A 10.2 그래프의 정의와 용어, 인쇄 p.366 / PDF p.358)
U11-009	무방향 그래프에서 정점의 차수(degree)는 그 정점에 인접한 정점의 수이다. 근거 대조 문구: 그 정점에 인접한 정점의 수	(A 10.2 그래프의 정의와 용어, 인쇄 p.366 / PDF p.358)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-010	방향 그래프에서 외부에서 오는 간선의 개수는 진입 차수, 외부로 향하는 간선의 개수는 진출 차수이다. 근거 대조 문구: 진입 차수	(A 10.2 그래프의 정의와 용어, 인쇄 p.366 / PDF p.358)
U11-011	A는 반복되는 간선이 없는 경로를 단순 경로(simple path)라 하고, 단순 경로의 시작 정점과 종료 정점이 같으면 사이클(cycle)이라 한다. 근거 대조 문구: 반복되는 간선이 없을 경우	(A 10.2 그래프의 정의와 용어, 인쇄 p.366 / PDF p.358)
U11-012	무방향 그래프의 모든 정점쌍 사이에 항상 경로가 존재하면 연결 그래프(connected graph)라 한다. 근거 대조 문구: 모든 정점쌍에 대하여 항상 경로가 존재	(A 10.2 그래프의 정의와 용어, 인쇄 p.367 / PDF p.359)
U11-013	트리는 사이클을 가지지 않는 연결 그래프이다. 근거 대조 문구: 사이클을 가지지 않는 연결 그래프	(A 10.2 그래프의 정의와 용어, 인쇄 p.367 / PDF p.359)
U11-014	모든 정점이 서로 연결되어 있는 그래프를 완전 그래프(complete graph)라 한다. 근거 대조 문구: 모든 정점이 서로 연결되어 있는 그래프	(A 10.2 그래프의 정의와 용어, 인쇄 p.367 / PDF p.359)
U11-015	인접 행렬(adjacency matrix)은 2차원 배열을 사용하여 그래프를 표현한다. 근거 대조 문구: 2차원 배열을 사용하여 그래프를 표현한다	(A 10.3 그래프의 표현 방법, 인쇄 p.369 / PDF p.361)
U11-016	자체 간선을 허용하지 않는 이 책의 그래프에서는 인접 행렬의 대각선 성분이 모두 0이다. 근거 대조 문구: 자체 간선을 허용하지 않으므로	(A 10.3 그래프의 표현 방법, 인쇄 p.370 / PDF p.362)
U11-017	무방향 그래프의 인접 행렬은 대칭 행렬이다. 근거 대조 문구: 무방향 그래프의 인접 행렬은 대칭 행렬	(A 10.3 그래프의 표현 방법, 인쇄 p.370 / PDF p.362)
U11-018	n개 정점의 인접 행렬에는 간선 수에 무관하게 항상 n^2개의 메모리 공간이 필요하다. 근거 대조 문구: 간선의 수에 무관하게 항상	(A 10.3 그래프의 표현 방법, 인쇄 p.370 / PDF p.362)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-019	인접 행렬에서는 두 정점을 연결하는 간선의 존재 여부를 O(1) 시간에 알 수 있다. 근거 대조 문구: O(1)시간 안에 즉시 알 수	(A 10.3 그래프의 표현 방법, 인쇄 p.370 / PDF p.362)
U11-020	인접 리스트(adjacency list)는 각 정점에 인접한 정점들을 연결 리스트로 표시한다. 근거 대조 문구: 각각의 정점에 인접한 정점들을 연결 리스트	(A 10.3 그래프의 표현 방법, 인쇄 p.373 / PDF p.365)
U11-021	n개 정점과 e개 간선의 무방향 그래프를 인접 리스트로 표시하면 n개의 헤더 노드와 2e개의 노드가 필요하다. 근거 대조 문구: n개의 헤더 노드와 2e개의 노드	(A 10.3 그래프의 표현 방법, 인쇄 p.374 / PDF p.366)
U11-022	인접 리스트에서 간선의 존재 여부나 정점의 차수를 알려면 해당 정점의 연결 리스트를 탐색해야 한다. 근거 대조 문구: 정점 i의 연결 리스트를 탐색	(A 10.3 그래프의 표현 방법, 인쇄 p.374 / PDF p.366)
U11-023	깊이 우선 탐색은 시작 정점을 방문 표시하고 아직 방문하지 않은 인접 정점을 시작 정점으로 하여 탐색을 다시 시작한다. 근거 대조 문구: 깊이 우선 탐색을 다시 시작한다	(A 10.5 깊이 우선 탐색, 인쇄 p.379 / PDF p.371)
U11-024	깊이 우선 탐색은 자기 자신을 다시 호출하는 순환 알고리즘의 형태를 가진다. 근거 대조 문구: 순환 알고리즘의 형태	(A 10.5 깊이 우선 탐색, 인쇄 p.379 / PDF p.371)
U11-025	정점 수 n, 간선 수 e인 그래프에서 깊이 우선 탐색의 시간 복잡도는 인접 리스트이면 O(n+e), 인접 행렬이면 $O(n^2)$이다. 근거 대조 문구: 인접 리스트로 표현되어 있다면	(A 10.5 깊이 우선 탐색, 인쇄 p.385 / PDF p.377)
U11-026	너비 우선 탐색은 시작 정점으로부터 가까운 정점을 먼저 방문하고 멀리 떨어진 정점을 나중에 방문한다. 근거 대조 문구: 가까운 정점을 먼저 방문	(A 10.6 너비 우선 탐색, 인쇄 p.385 / PDF p.377)
U11-027	너비 우선 탐색에는 가까운 정점들을 차례로 저장한 후 꺼낼 수 있는 큐(queue)가 필요하다. 근거 대조 문구: 큐(queue)가 필요하다	(A 10.6 너비 우선 탐색, 인쇄 p.385-386 / PDF p.377-378)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-028	진입 차수가 0인 정점이 여러 개이면 어느 정점을 선택해도 무방하며 하나의 그래프에 복수의 위상 순서가 있을 수 있다. 근거 대조 문구: 복수의 위상순서가 있을 수 있다	(A 11.7 위상 정렬, 인쇄 p.431 / PDF p.423)
U11-029	남은 정점 중 진입 차수가 0인 정점이 없으면 위상 정렬 알고리즘은 중단된다. 근거 대조 문구: 진입 차수 0인 정점이 없다면	(A 11.7 위상 정렬, 인쇄 p.431 / PDF p.423)
U11-030	B의 분리 집합 표현에서는 자식 노드가 부모 노드를 가리킨다. 근거 대조 문구: 자식이 부모를 가리킵니다	(B 4.4.1 분리 집합 표현, 인쇄 p.189 / PDF p.197)
U11-031	분리 집합의 목적은 원소가 어느 집합에 귀속되어 있는지 쉽게 알아내는 것이다. 근거 대조 문구: 원소가 어느 집합에 귀속되어 있는지	(B 4.4.2 분리 집합의 기본 연산, 인쇄 p.190 / PDF p.198)
U11-032	분리 집합의 두 기본 연산은 합집합과 집합 탐색 연산이다. 근거 대조 문구: 합집합	(B 4.4.2 분리 집합의 기본 연산, 인쇄 p.190 / PDF p.198)
U11-033	집합 탐색은 집합에서 원소를 찾는 연산이 아니라 원소가 속한 집합을 찾는 연산이다. 근거 대조 문구: 원소가 속한 집합을 찾는 연산	(B 4.4.2 분리 집합의 기본 연산, 인쇄 p.191-192 / PDF p.199-200)
U11-034	v를 방문되었다고 표시한다. 근거 대조 문구: v를 방문되었다고 표시	(A 10.5 알고리즘 10.1, 인쇄 p.379 / PDF p.371)
U11-035	v에 인접한 모든 정점 u에 대하여 아직 방문되지 않았으면 depth_first_search(u)를 수행한다. 근거 대조 문구: depth_first_search(u)	(A 10.5 알고리즘 10.1, 인쇄 p.379 / PDF p.371)
U11-036	v를 방문 표시하고 큐에 정점 v를 삽입한다. 근거 대조 문구: 큐에 정점 v를 삽입	(A 10.6 알고리즘 10.2, 인쇄 p.386 / PDF p.378)
U11-037	큐가 공백이 아니면 큐에서 정점 w를 삭제한다. 근거 대조 문구: 큐에서 정점 w를 삭제	(A 10.6 알고리즘 10.2, 인쇄 p.386 / PDF p.378)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-038	w에 인접한 정점 u가 아직 방문되지 않았으면 큐에 삽입하고 방문 표시한다. 근거 대조 문구: u를 큐에 삽입	(A 10.6 알고리즘 10.2, 인쇄 p.386 / PDF p.378)
U11-039	리스트를 하나 준비한다. 근거 대조 문구: 리스트를 하나 준비	(B 9.4.1 위상 정렬의 동작 방식, 인쇄 p.408 / PDF p.410)
U11-040	진입 간선이 없는 정점을 리스트에 추가하고 해당 정점과 진출 간선을 제거한다. 근거 대조 문구: 진입 간선이 없는 정점을 리스트에 추가	(B 9.4.1 위상 정렬의 동작 방식, 인쇄 p.408 / PDF p.410)
U11-041	모든 정점에 대해 반복하고 정점이 남아 있지 않으면 종료한다. 근거 대조 문구: 정점이 남아 있지 않으면 정렬을 종료	(B 9.4.1 위상 정렬의 동작 방식, 인쇄 p.408 / PDF p.410)
U11-042	부모 포인터를 따라 올라가 부모가 NULL인 뿌리 노드를 찾는다. 근거 대조 문구: 부모가 NULL인 뿌리 노드	(B 4.4.2 집합 탐색 연산, 인쇄 p.192 / PDF p.200)
U11-043	뿌리 노드를 반환하여 해당 노드가 소속된 집합을 반환한다. 근거 대조 문구: 이것을 반환하면	(B 4.4.2 집합 탐색 연산, 인쇄 p.192 / PDF p.200)

U11-044	DS_FindSet() 원문 발췌 (원문 코드 발췌)
	DisjointSet* DS_FindSet(DisjointSet* Set)
	{
	while (Set->Parent != NULL)
	{
	Set = Set->Parent;
	}
	return Set;
	}
	근거 대조 문구: DisjointSet* DS_FindSet
		(B 4.4.2 집합 탐색 연산, 인쇄 p.192 / PDF p.200)

U11-045	무방향 완전 그래프의 간선 수: n×(n-1)/2; n은 정점 수이다. 근거 대조 문구: n×(n-1)/2	(A 10.2 완전 그래프, 인쇄 p.367 / PDF p.359)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-046	깊이 우선 탐색의 시간 복잡도: 인접 리스트: O(n+e), 인접 행렬: O(n²); n은 정점 수, e는 간선 수이다. 근거 대조 문구: 시간 복잡도	(A 10.5 깊이 우선 탐색의 분석, 인쇄 p.385 / PDF p.377)
U11-047	인접 행렬 근거 대조 문구: 인접 행렬	(A 10.3 그래프의 표현 방법, 인쇄 p.369 / PDF p.361)
U11-048	인접 리스트 근거 대조 문구: 인접 리스트	(A 10.3 그래프의 표현 방법, 인쇄 p.373 / PDF p.365)
U11-049	깊이 우선 탐색 근거 대조 문구: 깊이 우선 탐색	(A 10.5 깊이 우선 탐색, 인쇄 p.379 / PDF p.371)
U11-050	너비 우선 탐색 근거 대조 문구: 너비 우선 탐색	(A 10.6 너비 우선 탐색, 인쇄 p.385-386 / PDF p.377-378)

12 최소 비용 신장 트리·최단 경로

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-001	최소 비용 신장 트리는 신장 트리 중 사용된 간선들의 가중치 합이 최소인 신장 트리이다. 근거 대조 문구: 가중치 합이 최소	(A 11.1 최소 비용 신장 트리, 인쇄 p.397 / PDF p.389)
U12-002	최단 경로 문제는 두 정점을 연결하는 경로 중 간선의 가중치 합이 최소가 되는 경로를 찾는 문제이다. 근거 대조 문구: 가중치 합이 최소	(A 11.4 최단 경로, 인쇄 p.414 / PDF p.406)
U12-003	신장 트리(spanning tree)는 그래프 내의 모든 정점을 포함하는 트리이다. 근거 대조 문구: 모든 정점을 포함하는 트리	(A 11.1 최소 비용 신장 트리, 인쇄 p.396 / PDF p.388)
U12-004	신장 트리는 모든 정점이 연결되어 있어야 하며 사이클을 포함해서는 안 된다. 근거 대조 문구: 사이클을 포함해서는 안 된다	(A 11.1 최소 비용 신장 트리, 인쇄 p.396 / PDF p.388)
U12-005	신장 트리는 n개의 정점을 정확히 (n-1)개의 간선으로 연결한다. 근거 대조 문구: 정확히 (n-1)개의 간선	(A 11.1 최소 비용 신장 트리, 인쇄 p.396 / PDF p.388)
U12-006	깊이 우선 또는 너비 우선 탐색 도중 사용된 간선만 모으면 신장 트리를 만들 수 있다. 근거 대조 문구: 사용된 간선만 모으면	(A 11.1 최소 비용 신장 트리, 인쇄 p.396 / PDF p.388)
U12-007	B는 최소 신장 트리가 연결된 가중치 그래프(Connected Weighted Graph)로부터 만들어진다고 설명한다. 근거 대조 문구: 연결된 가중치 그래프	(B 9.5.1 최소 신장 트리를 만들어낼 수 있는 그래프의 조건, 인쇄 p.420 / PDF p.422)
U12-008	탐욕적인 방법은 선택할 때마다 그 순간 가장 좋다고 생각되는 것을 선택하여 최종 해답에 도달하는 방법이다. 근거 대조 문구: 그 순간 가장 좋다고 생각되는 것	(A 11.2 Kruskal의 MST 알고리즘, 인쇄 p.398 / PDF p.390)
U12-009	탐욕적인 방법의 지역적인 최적 선택을 모아도 최종 해답의 전역적인 최적은 보장되지 않으므로 검증해야 한다. 근거 대조 문구: 전역적으로 최적이라는 보장은 없다	(A 11.2 Kruskal의 MST 알고리즘, 인쇄 p.399 / PDF p.391)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-010	Kruskal의 알고리즘은 각 단계에서 사이클을 이루지 않는 최소 비용 간선을 선택한다. 근거 대조 문구: 사이클을 이루지 않는 최소 비용 간선	(A 11.2 Kruskal의 MST 알고리즘, 인쇄 p.399 / PDF p.391)
U12-011	새 간선이 다른 경로로 이미 연결된 정점들을 연결하면 사이클이 형성된다. 근거 대조 문구: 새로운 간선이 이미 다른 경로에 의하여 연결	(A 11.2 Kruskal의 MST 알고리즘, 인쇄 p.401 / PDF p.393)
U12-012	A의 union-find를 이용한 Kruskal 알고리즘은 간선 정렬 시간에 좌우된다. 근거 대조 문구: 간선들을 정렬하는 시간에 좌우	(A 11.2 시간 복잡도 분석, 인쇄 p.407 / PDF p.399)
U12-013	Prim의 알고리즘은 시작 정점에서 출발하여 신장 트리 집합을 단계적으로 확장한다. 근거 대조 문구: 신장 트리 집합을 단계적으로 확장	(A 11.3 Prim의 MST 알고리즘, 인쇄 p.408 / PDF p.400)
U12-014	Prim의 방법은 신장 트리 집합에 인접한 정점 중 최저 간선으로 연결된 정점을 선택한다. 근거 대조 문구: 최저 간선으로 연결된 정점	(A 11.3 Prim의 MST 알고리즘, 인쇄 p.408 / PDF p.400)
U12-015	Kruskal의 알고리즘은 간선을 기반으로 하고 Prim의 알고리즘은 정점을 기반으로 한다. 근거 대조 문구: 간선을 기반으로 하는 알고리즘	(A 11.3 Prim의 MST 알고리즘, 인쇄 p.409 / PDF p.401)
U12-016	A의 Prim 구현은 주 반복문과 내부 반복문이 각각 n번 반복하여 $O(n^2)$의 복잡도를 가진다. 근거 대조 문구: $O(n^2)$의 복잡도	(A 11.3 Prim의 알고리즘의 분석, 인쇄 p.413 / PDF p.405)
U12-017	A는 희소 그래프에는 Kruskal이 적합하고 밀집 그래프에는 Prim이 유리하다고 비교한다. 근거 대조 문구: 희소 그래프를 대상으로 할 경우	(A 11.3 Prim의 알고리즘의 분석, 인쇄 p.413 / PDF p.405)
U12-018	Dijkstra 알고리즘은 하나의 시작 정점으로부터 모든 다른 정점까지의 최단 경로를 찾는다. 근거 대조 문구: 하나의 시작 정점으로부터	(A 11.5 Dijkstra의 최단 경로 알고리즘, 인쇄 p.415 / PDF p.407)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-019	A의 최단 거리 알고리즘 입력은 가중치가 음수가 아닌 가중치 그래프이다. 근거 대조 문구: 가중치는 음수가 아님	(A 11.5 알고리즘 11.6, 인쇄 p.417 / PDF p.409)
U12-020	A는 가중치 인접 행렬에서 간선이 없으면 상당히 큰 정수를 무한대로 간주하여 저장한다. 근거 대조 문구: 상당히 큰 값을 무한대라고 생각	(A 11.5 Dijkstra의 최단 경로 알고리즘, 인쇄 p.415 / PDF p.407)
U12-021	각 단계에서 집합 S에 속하지 않은 정점 중 distance 값이 가장 작은 정점을 S에 추가한다. 근거 대조 문구: distance값이 가장 작은 정점	(A 11.5 Dijkstra의 최단 경로 알고리즘, 인쇄 p.416 / PDF p.408)
U12-022	새 정점 u가 S에 추가되면 u를 거치는 거리와 기존 거리를 비교하여 더 작은 거리로 distance를 수정한다. 근거 대조 문구: 더 작은 거리로	(A 11.5 Dijkstra의 최단 경로 알고리즘, 인쇄 p.417 / PDF p.409)
U12-023	A의 distance 배열은 시작 정점부터 다른 정점까지의 최단 경로 거리 정보만 제공한다. 근거 대조 문구: 거리 정보만을 제공	(A 11.5 Dijkstra의 분석, 인쇄 p.424 / PDF p.416)
U12-024	A의 Dijkstra 구현은 $O(n^2)$의 복잡도를 가진다. 근거 대조 문구: $O(n^2)$의 복잡도	(A 11.5 Dijkstra의 분석, 인쇄 p.424 / PDF p.416)
U12-025	B는 데이크스트라 알고리즘을 사이클이 없는 방향성 그래프에 한해서 사용할 수 있다고 기술한다. 근거 대조 문구: 사이클이 없는 방향성 그래프에 한해서만	(B 9.6.1 데이크스트라 알고리즘의 개념, 인쇄 p.442 / PDF p.444)
U12-026	B는 프림이 간선의 길이를 이용하는 데 비해 데이크스트라는 경로의 길이에 따라 간선을 연결한다고 비교한다. 근거 대조 문구: 경로의 길이에 따라서 간선을 연결	(B 9.6.1 데이크스트라 알고리즘의 개념, 인쇄 p.442 / PDF p.444)
U12-027	Floyd 알고리즘은 그래프의 모든 정점 사이의 최단 경로를 한 번에 모두 찾는다. 근거 대조 문구: 모든 정점 사이의 최단 경로	(A 11.6 Floyd의 최단 경로 알고리즘, 인쇄 p.425 / PDF p.417)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-028	Floyd 알고리즘은 2차원 배열 A와 삼중 반복문으로 구성된다. 근거 대조 문구: 2차원 배열 A를 이용하여	(A 11.6 Floyd의 최단 경로 알고리즘, 인쇄 p.425 / PDF p.417)
U12-029	Floyd 알고리즘의 A 초기값은 가중치 행렬 weight이다. 근거 대조 문구: A의 초기값은 가중치 행렬	(A 11.6 Floyd의 최단 경로 알고리즘, 인쇄 p.425 / PDF p.417)
U12-030	Floyd 알고리즘은 삼중 반복문이 실행되므로 시간 복잡도가 O(n³)이다. 근거 대조 문구: 3중 반복문이 실행	(A 11.6 Floyd 최단 경로 알고리즘의 분석, 인쇄 p.430 / PDF p.422)
U12-031	그래프의 간선들을 가중치의 오름차순으로 정렬한다. 근거 대조 문구: 가중치의 오름차순으로 정렬	(A 11.2 Kruskal의 MST 알고리즘, 인쇄 p.399 / PDF p.391)
U12-032	정렬된 간선 중 사이클을 형성하지 않는 간선을 신장 트리 집합에 추가하고 사이클을 형성하는 간선은 제외한다. 근거 대조 문구: 사이클을 형성하면 그 간선은 제외	(A 11.2 Kruskal의 MST 알고리즘, 인쇄 p.399 / PDF p.391)
U12-033	선택된 간선 수가 (n-1)이 될 때까지 반복한다. 근거 대조 문구: ecounter<(n-1)	(A 11.2 알고리즘 11.2, 인쇄 p.399 / PDF p.391)
U12-034	그래프와 정점이 하나도 없는 최소 신장 트리를 준비한다. 근거 대조 문구: 노드가 하나도 없는 상태	(B 9.5.1 프림 알고리즘, 인쇄 p.420 / PDF p.422)
U12-035	임의의 정점을 시작 정점으로 선택하여 최소 신장 트리의 뿌리 노드로 삽입한다. 근거 대조 문구: 뿌리 노드로 삽입	(B 9.5.1 프림 알고리즘, 인쇄 p.420 / PDF p.422)
U12-036	트리의 정점과 인접 정점 사이에서 최소 가중치 간선을 골라 사이클이 생기지 않는 인접 정점을 삽입한다. 근거 대조 문구: 사이클을 형성해서는 안 됩니다	(B 9.5.1 프림 알고리즘, 인쇄 p.420 / PDF p.422)
U12-037	모든 정점이 연결될 때까지 반복하고 종료한다. 근거 대조 문구: 그래프의 모든 정점을 연결	(B 9.5.1 프림 알고리즘, 인쇄 p.420 / PDF p.422)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-038	시작 정점 v에 대해 distance[v]=0, 다른 정점의 distance는 시작 정점과의 가중치 값으로 둔다. 근거 대조 문구: distance[v]=0	(A 11.5 Dijkstra의 최단 경로 알고리즘, 인쇄 p.415 / PDF p.407)
U12-039	S에 속하지 않은 정점 중 distance 값이 가장 작은 정점을 S에 추가한다. 근거 대조 문구: 가장 distance값이 작은 정점	(A 11.5 Dijkstra의 최단 경로 알고리즘, 인쇄 p.416 / PDF p.408)

U12-040	floyd()의 핵심 반복문 발췌 (원문 코드 발췌)
	for (k = 0; k<g->n; k++) {
	for (i = 0; i<g->n; i++)
	for (j = 0; j<g->n; j++)
	if (A[i][k] + A[k][j] < A[i][j])
	A[i][j] = A[i][k] + A[k][j];
	printA(g);
	}
	근거 대조 문구: A[i][k] + A[k][j] < A[i][j]
		(A 11.6 프로그램 11.11, 인쇄 p.429 / PDF p.421)

U12-041	distance 갱신: distance[w] = min(distance[w], distance[u]+weight[u][w]); u는 새로 추가된 정점이며 S에 속하지 않는 정점 w의 값을 갱신한다. 근거 대조 문구: distance[w]	(A 11.5 Dijkstra의 최단 경로 알고리즘, 인쇄 p.417 / PDF p.409)
U12-042	Floyd 갱신: A[i][j] = min(A[i][j], A[i][k] + A[k][j]); k를 0부터 n-1까지 고려한다. 근거 대조 문구: A[i][j]	(A 11.6 알고리즘 11.8, 인쇄 p.425 / PDF p.417)
U12-043	A의 MST 구현 복잡도: Kruskal: O(e $\log_2 e$), Prim: O(n^2); e는 간선 수, n은 정점 수이며 A의 구현과 효율적인 정렬 알고리즘을 기준으로 한다. 근거 대조 문구: O(e $\log_2 e$)	(A 11.3 Prim의 알고리즘의 분석, 인쇄 p.413 / PDF p.405)
U12-044	Kruskal 근거 대조 문구: Kruskal	(A 11.2 Kruskal의 MST 알고리즘, 인쇄 p.399 / PDF p.391)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-045	Prim 근거 대조 문구: Prim	(A 11.3 Prim의 MST 알고리즘, 인쇄 p.408 / PDF p.400)
U12-046	Dijkstra 근거 대조 문구: Dijkstra	(A 11.5 Dijkstra의 최단 경로 알고리즘, 인쇄 p.415 / PDF p.407)
U12-047	Floyd 근거 대조 문구: Floyd	(A 11.6 Floyd의 최단 경로 알고리즘, 인쇄 p.425 / PDF p.417)

13 정렬

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-001	정렬(sorting)은 크기순으로 오름차순(ascending order)이나 내림차순(descending order)으로 나열하는 것이다. 근거 대조 문구: 오름차순	(A 12.1 정렬이란?, 인쇄 p.442 / PDF p.433)
U13-002	안정성(stability)은 동일 키값 레코드들의 상대적 위치가 정렬 후에도 바뀌지 않는 성질이다. 근거 대조 문구: 상대적인 위치가 정렬 후에도 바뀌지	(A 12.1 정렬이란?, 인쇄 p.445 / PDF p.436)
U13-003	정렬의 대상은 레코드(record)이며 레코드는 필드(field)로 나누어진다. 근거 대조 문구: 레코드는 다시 필드	(A 12.1 정렬이란?, 인쇄 p.443 / PDF p.434)
U13-004	키(key)는 레코드와 레코드를 식별하는 필드이며 정렬은 레코드들을 키값 순서로 재배열하는 것이다. 근거 대조 문구: 키값의 순서로 재배열	(A 12.1 정렬이란?, 인쇄 p.443 / PDF p.434)
U13-005	정렬 알고리즘의 효율성 기준은 비교 연산 횟수와 이동 연산 횟수이다. 근거 대조 문구: 비교 연산의 횟수와 이동 연산의 횟수	(A 12.1 정렬이란?, 인쇄 p.443 / PDF p.434)
U13-006	내부 정렬은 모든 데이터가 메인 메모리에 올라와 있는 상태의 정렬이다. 근거 대조 문구: 모든 데이터가 메인 메모리에	(A 12.1 정렬이란?, 인쇄 p.445 / PDF p.436)
U13-007	외부 정렬은 대부분의 데이터가 외부 기억 장치에 있고 일부만 메모리에 올려놓은 상태의 정렬이다. 근거 대조 문구: 외부 기억 장치에 대부분의 데이터	(A 12.1 정렬이란?, 인쇄 p.445 / PDF p.436)
U13-008	A는 안정성을 충족하는 정렬로 삽입 정렬·버블 정렬·합병 정렬을 든다. 근거 대조 문구: 삽입정렬, 버블정렬, 합병정렬	(A 12.1 정렬이란?, 인쇄 p.445 / PDF p.436)
U13-009	선택 정렬은 정렬되지 않은 리스트에서 가장 작은 숫자를 선택하여 정렬된 리스트로 이동하는 작업을 반복한다. 근거 대조 문구: 가장 작은 숫자를 선택	(A 12.2 선택 정렬, 인쇄 p.445 / PDF p.436)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-010	A의 선택 정렬은 안정성을 만족하지 않는다. 근거 대조 문구: 안정성을 만족하지는 않는다	(A 12.2 선택 정렬, 인쇄 p.450 / PDF p.441)
U13-011	삽입 정렬은 새 레코드를 이미 정렬된 레코드 사이의 올바른 자리에 삽입하여 정렬을 유지한다. 근거 대조 문구: 올바른 자리를 찾아 삽입	(A 12.3 삽입 정렬, 인쇄 p.450 / PDF p.441)
U13-012	삽입 정렬은 이미 정렬된 입력에서 O(n), 역순 입력에서 O(n²)의 시간 복잡도를 가진다. 근거 대조 문구: 입력 자료가 역순일 경우	(A 12.3 삽입 정렬의 복잡도 분석, 인쇄 p.454 / PDF p.445)
U13-013	삽입 정렬은 레코드 수가 적거나 대부분 이미 정렬된 경우에 유리할 수 있다. 근거 대조 문구: 레코드의 수가 적을 경우	(A 12.3 삽입 정렬의 복잡도 분석, 인쇄 p.454 / PDF p.445)
U13-014	버블 정렬은 인접한 두 레코드를 비교하여 순서가 맞지 않으면 교환하는 비교-교환 과정을 반복한다. 근거 대조 문구: 인접한 2개의 레코드를 비교	(A 12.4 버블 정렬, 인쇄 p.455 / PDF p.446)
U13-015	버블 정렬의 한 번 스캔이 끝나면 가장 큰 레코드가 리스트 오른쪽 끝으로 이동한다. 근거 대조 문구: 가장 큰 레코드가 리스트의 오른쪽 끝	(A 12.4 버블 정렬, 인쇄 p.455 / PDF p.446)
U13-016	A의 버블 정렬 비교 횟수는 최선·평균·최악 모두 n(n-1)/2로 일정하다. 근거 대조 문구: 최선, 평균, 최악의 어떠한 경우에도 항상 일정	(A 12.4 버블 정렬의 복잡도 분석, 인쇄 p.458 / PDF p.449)
U13-017	쉘 정렬은 일정 간격의 요소들로 만든 연속적이지 않은 부분 리스트를 삽입 정렬한다. 근거 대조 문구: 연속적이지 않은 여러 개의 부분 리스트	(A 12.5 쉘 정렬, 인쇄 p.459 / PDF p.450)
U13-018	쉘 정렬은 간격(gap)을 줄여가며 부분 리스트를 다시 만들고 마지막 간격은 1이다. 근거 대조 문구: 마지막 스텝에서는 간격의 값이 1	(A 12.5 쉘 정렬, 인쇄 p.459 / PDF p.450)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-019	쉘 정렬에서는 일정 간격으로 삽입 정렬하므로 실제 부분 리스트를 위한 추가 공간이 필요 없다. 근거 대조 문구: 추가적인 공간은 필요 없다	(A 12.5 쉘 정렬, 인쇄 p.459 / PDF p.450)
U13-020	A 본문은 실험적 연구의 쉘 정렬 복잡도를 최악 O(n²), 평균 O(n^1.5)로 기술한다. 근거 대조 문구: 실험적인 연구를 통하여	(A 12.5 쉘 정렬의 분석, 인쇄 p.461 / PDF p.452)
U13-021	합병 정렬은 리스트를 균등한 두 부분으로 분할하고 각 부분을 정렬한 다음 합하여 전체 정렬 리스트를 얻는다. 근거 대조 문구: 두 개의 균등한 크기로 분할	(A 12.6 합병 정렬, 인쇄 p.462 / PDF p.453)
U13-022	합병 정렬은 분할 정복(divide and conquer) 기법에 바탕을 둔다. 근거 대조 문구: 분할 정복	(A 12.6 합병 정렬, 인쇄 p.462 / PDF p.453)
U13-023	A의 합병 정렬 구현은 부분 배열에 숫자 하나가 남을 때까지 분할한 뒤 merge 함수로 합병한다. 근거 대조 문구: 숫자가 하나 남을 때까지	(A 12.6 합병 정렬의 C언어 구현, 인쇄 p.467 / PDF p.458)
U13-024	merge는 임시 배열 sorted에 정렬한 숫자들을 저장한 후 원래 배열로 복사한다. 근거 대조 문구: 임시 배열	(A 12.6 합병 정렬의 C언어 구현, 인쇄 p.467 / PDF p.458)
U13-025	퀵 정렬은 피벗(pivot)보다 작은 요소를 왼쪽, 큰 요소를 오른쪽으로 옮겨 비균등하게 분할한다. 근거 대조 문구: 비균등하게 분할	(A 12.7 퀵 정렬, 인쇄 p.470 / PDF p.461)
U13-026	퀵 정렬은 피벗을 제외한 왼쪽·오른쪽 부분 리스트에 다시 퀵 정렬을 적용한다. 근거 대조 문구: 피벗을 제외한	(A 12.7 퀵 정렬, 인쇄 p.470 / PDF p.461)
U13-027	A의 퀵 정렬은 계속 불균형하게 분할되면 최악 O(n²)의 시간 복잡도가 된다. 근거 대조 문구: 계속 불균형하게 나누어지는	(A 12.7 퀵 정렬의 복잡도 분석, 인쇄 p.476 / PDF p.467)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-028	A의 퀵 정렬 평균 시간 복잡도는 $O(n\log_2 n)$이다. 근거 대조 문구: 평균적인 경우의 시간 복잡도	(A 12.7 퀵 정렬의 복잡도 분석, 인쇄 p.477 / PDF p.468)
U13-029	A는 왼쪽·오른쪽·중간의 세 데이터 중 중간값을 피벗으로 고르는 median of three를 설명한다. 근거 대조 문구: median of three	(A 12.7 퀵 정렬의 복잡도 분석, 인쇄 p.477 / PDF p.468)
U13-030	B는 배열의 양끝에서 기준보다 큰 요소와 작은 요소를 찾아 맞교환하고 마지막에 기준 요소의 위치를 정하는 분할을 설명한다. 근거 대조 문구: 두 요소를 맞교환	(B 5.4.1 퀵 정렬 사용 전 해결해야 하는 2가지 문제, 인쇄 p.219 / PDF p.226)
U13-031	A의 히프 정렬은 최소 히프로 변환한 배열에서 가장 작은 원소부터 차례로 추출한다. 근거 대조 문구: 최소 히프로 변환	(A 12.8 히프 정렬, 인쇄 p.479 / PDF p.470)
U13-032	기수 정렬(radix sort)은 레코드 비교 연산 없이 자리수의 값에 따라 정렬한다. 근거 대조 문구: 어떤 비교 연산도 실행하지 않고	(A 12.9 기수 정렬, 인쇄 p.480 / PDF p.471)
U13-033	A는 기수 정렬의 키가 동일한 길이의 숫자나 문자열로 구성되어야 한다고 설명한다. 근거 대조 문구: 동일한 길이를 가지는 숫자나 문자열	(A 12.9 기수 정렬, 인쇄 p.480 / PDF p.471)
U13-034	여러 자리 정수의 기수 정렬은 낮은 자리수부터 정렬하고 차츰 높은 자리수를 사용한다. 근거 대조 문구: 먼저 낮은 자리수로 정렬	(A 12.9 기수 정렬, 인쇄 p.481 / PDF p.472)
U13-035	n개 정수가 각각 d개 자리수를 가지면 A의 기수 정렬 시간 복잡도는 $O(d\cdot n)$이다. 근거 대조 문구: O(d·n)	(A 12.9 기수 정렬의 분석, 인쇄 p.485 / PDF p.476)
U13-036	A의 영어 사전 정렬 예제는 단어와 의미를 저장한 구조체 배열을 정렬한다. 근거 대조 문구: 구조체의 배열	(A 12.11 정렬의 응용: 영어 사전을 위한 정렬, 인쇄 p.487 / PDF p.478)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-037	qsort()의 base는 정렬 대상 배열 주소, num은 요소 수, width는 요소 하나의 바이트 크기이다. 근거 대조 문구: 개별 데이터 요소의 크기	(B 5.5 C 언어 표준 라이브러리의 퀵 정렬 함수: qsort(), 인쇄 p.227 / PDF p.234)
U13-038	qsort()의 마지막 매개 변수는 비교 결과를 반환하는 함수의 포인터이다. 근거 대조 문구: 비교를 수행한 결과를 반환하는 함수	(B 5.5 C 언어 표준 라이브러리의 퀵 정렬 함수: qsort(), 인쇄 p.227 / PDF p.234)
U13-039	비교 함수는 첫 요소가 둘째보다 크면 양수, 같으면 0, 작으면 음수를 반환한다. 근거 대조 문구: 0보다 큰 수를, 같으면 0을	(B 5.5 C 언어 표준 라이브러리의 퀵 정렬 함수: qsort(), 인쇄 p.228 / PDF p.235)
U13-040	인덱스 1부터 시작하여 현재 i번째 정수를 key에 복사한다. 근거 대조 문구: 인덱스 1부터 시작	(A 12.3 알고리즘 설명, 인쇄 p.453 / PDF p.444)
U13-041	i-1부터 역순으로 조사하고 key보다 큰 정수를 한 칸씩 오른쪽으로 이동한다. 근거 대조 문구: 역순으로 조사	(A 12.3 알고리즘 설명, 인쇄 p.453 / PDF p.444)
U13-042	key보다 작은 정수 다음 위치인 j+1에 key를 넣는다. 근거 대조 문구: j+1번째가 key값이 들어갈 위치	(A 12.3 알고리즘 설명, 인쇄 p.453 / PDF p.444)
U13-043	분할(Divide): 입력 배열을 같은 크기의 두 부분 배열로 분할한다. 근거 대조 문구: 분할(Divide)	(A 12.6 합병 정렬, 인쇄 p.462 / PDF p.453)
U13-044	정복(Conquer): 부분 배열을 정렬하고 충분히 작지 않으면 순환 호출로 다시 분할 정복한다. 근거 대조 문구: 정복(Conquer)	(A 12.6 합병 정렬, 인쇄 p.462 / PDF p.453)
U13-045	결합(Combine): 정렬된 부분 배열들을 하나의 배열에 통합한다. 근거 대조 문구: 결합(Combine)	(A 12.6 합병 정렬, 인쇄 p.462 / PDF p.453)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-046	정렬 범위가 두 개 이상의 데이터이면 partition 함수로 피벗 기준 두 리스트로 분할한다. 근거 대조 문구: partition 함수를 호출	(A 12.7 프로그램 설명, 인쇄 p.472 / PDF p.463)
U13-047	left부터 피벗 바로 앞까지를 순환 호출한다. 근거 대조 문구: 피봇은 제외된다	(A 12.7 프로그램 설명, 인쇄 p.472 / PDF p.463)
U13-048	피벗 바로 다음부터 right까지를 순환 호출한다. 근거 대조 문구: right까지를 대상으로 순환호출	(A 12.7 프로그램 설명, 인쇄 p.472 / PDF p.463)

U13-049	quick_sort() 원문 발췌 (원문 코드 발췌)
	void quick_sort(int list[], int left, int right)
	{
	if (left<right) {
	int q = partition(list, left, right);
	quick_sort(list, left, q - 1);
	quick_sort(list, q + 1, right);
	}
	}
	근거 대조 문구: void quick_sort
		(A 12.7 프로그램 12.6, 인쇄 p.471 / PDF p.462)

U13-050	선택 정렬의 비교 횟수: (n-1)+(n-2)+...+1 = n(n-1)/2 = O(n²); n은 레코드 수이다. 근거 대조 문구: n(n-1)/2	(A 12.2 선택 정렬의 복잡도, 인쇄 p.450 / PDF p.441)
U13-051	기수 정렬의 시간 복잡도: O(d·n); n은 정수 수, d는 각 정수의 자리수이다. 근거 대조 문구: O(d·n)	(A 12.9 기수 정렬의 분석, 인쇄 p.485 / PDF p.476)
U13-052	삽입 정렬 근거 대조 문구: 삽입 정렬	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-053	O(n) 근거 대조 문구: O(n)	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-054	O(n²) 근거 대조 문구: O(n²)	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-055	선택 정렬 근거 대조 문구: 선택 정렬	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-056	버블 정렬 근거 대조 문구: 버블 정렬	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-057	퀵 정렬 근거 대조 문구: 퀵 정렬	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-058	O(nlog₂n) 근거 대조 문구: O(nlog₂n)	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-059	쉘 정렬 근거 대조 문구: 쉘 정렬	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-060	O(n^1.5) 근거 대조 문구: O(n^1.5)	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-061	히프 정렬 근거 대조 문구: 히프 정렬	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-062	합병 정렬 근거 대조 문구: 합병 정렬	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-063	기수 정렬 근거 대조 문구: 기수 정렬	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)
U13-064	O(dn) 근거 대조 문구: O(dn)	(A 12.10 표 12-1, 인쇄 p.486 / PDF p.477)

14 탐색

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-001	탐색(search)은 여러 자료 중에서 원하는 자료를 찾는 작업이다. 근거 대조 문구: 원하는 자료를 찾는 작업	(A 13.1 탐색이란?, 인쇄 p.496 / PDF p.487)
U14-002	이진 탐색을 적용하기 전에 배열이 반드시 정렬되어 있어야 한다. 근거 대조 문구: 배열이 반드시 정렬되어 있어야	(A 13.3 정렬된 배열에서의 탐색, 인쇄 p.501 / PDF p.492)
U14-003	탐색의 단위는 항목이며 항목 안에는 항목을 구별하는 탐색키(search key)가 있다. 근거 대조 문구: 탐색키(search key)	(A 13.1 탐색이란?, 인쇄 p.496 / PDF p.487)
U14-004	탐색은 탐색키와 데이터로 이루어진 항목들 중 원하는 탐색키를 가진 항목을 찾는 것이다. 근거 대조 문구: 원하는 탐색키를 가지고 있는 항목	(A 13.1 탐색이란?, 인쇄 p.496 / PDF p.487)
U14-005	순차 탐색은 정렬되지 않은 배열의 처음부터 마지막까지 하나씩 검사하여 원하는 항목을 찾는다. 근거 대조 문구: 처음부터 마지막까지 하나씩 검사	(A 13.2 정렬되지 않은 배열에서의 탐색, 인쇄 p.497 / PDF p.488)
U14-006	A의 순차 탐색 함수는 성공하면 발견 위치를, 실패하면 -1을 반환한다. 근거 대조 문구: 그 항목이 발견된 위치를 반환	(A 13.2 프로그램 13.1, 인쇄 p.497 / PDF p.488)
U14-007	A의 개선된 순차 탐색은 리스트 끝에 탐색키를 저장하여 끝 도달 여부를 매번 비교하지 않는다. 근거 대조 문구: 리스트의 끝에 찾고자 하는 키 값을 저장	(A 13.2 개선된 순차 탐색, 인쇄 p.498 / PDF p.489)
U14-008	개선된 순차 탐색에서 발견 인덱스가 high+1이면 탐색 실패이다. 근거 대조 문구: high+1	(A 13.2 프로그램 13.2, 인쇄 p.499 / PDF p.490)
U14-009	각 키의 탐색 확률이 같으면 순차 탐색 성공 시 평균 비교 횟수는 (n+1)/2이다. 근거 대조 문구: 모든 키가 탐색될 확률이 동일	(A 13.2 순차 탐색의 시간 복잡도, 인쇄 p.500 / PDF p.491)
U14-010	순차 탐색은 실패 시 n번 비교하고 시간 복잡도는 O(n)이다. 근거 대조 문구: 탐색이 실패한 경우 n번	(A 13.2 순차 탐색의 시간 복잡도, 인쇄 p.500 / PDF p.491)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-011	자기 구성법은 자주 사용되는 항목을 데이터 앞쪽에 배치하여 순차 검색 효율을 높인다. 근거 대조 문구: 자주 사용되는 항목을 데이터 앞쪽에 배치	(B 6.2 자기 구성법, 인쇄 p.237 / PDF p.244)
U14-012	전진 이동법(Move To Front Method)은 탐색된 항목을 데이터의 가장 앞으로 옮긴다. 근거 대조 문구: 데이터의 가장 앞	(B 6.2.1 전진 이동법, 인쇄 p.237 / PDF p.244)
U14-013	전위법(Transpose Method)은 탐색된 항목을 바로 이전 항목과 교환한다. 근거 대조 문구: 바로 이전 항목과 교환	(B 6.2.2 전위법, 인쇄 p.239 / PDF p.246)
U14-014	계수법(Frequency Count Method)은 각 요소의 탐색 횟수를 별도 공간에 저장하고 횟수가 높은 순으로 재구성한다. 근거 대조 문구: 탐색된 횟수가 높은 순	(B 6.2.3 계수법, 인쇄 p.241 / PDF p.248)
U14-015	계수법에는 계수 결과를 저장하는 별도 공간과 결과에 따른 데이터 재배치 비용이 든다. 근거 대조 문구: 별도의 공간을 유지	(B 6.2.3 계수법, 인쇄 p.241 / PDF p.248)
U14-016	이진 탐색(binary search)은 중앙값을 조사하여 탐색 범위를 왼쪽 또는 오른쪽 절반으로 줄인다. 근거 대조 문구: 탐색의 범위를 반으로 줄인다	(A 13.3 정렬된 배열에서의 탐색, 인쇄 p.500 / PDF p.491)
U14-017	A는 이진 탐색이 삽입·삭제가 빈번한 데이터보다 주로 고정된 데이터에 적합하다고 설명한다. 근거 대조 문구: 주로 고정된 데이터	(A 13.3 정렬된 배열에서의 탐색, 인쇄 p.501 / PDF p.492)
U14-018	이진 탐색은 순환 호출 또는 반복문으로 구현할 수 있다. 근거 대조 문구: 반복문을 사용하여 구현	(A 13.3 이진 탐색 구현, 인쇄 p.503 / PDF p.494)
U14-019	이진 탐색의 시간 복잡도는 $O(\log_2 n)$이다. 근거 대조 문구: $O(\log_2 n)$	(A 13.3 이진 탐색의 시간 복잡도, 인쇄 p.505 / PDF p.496)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-020	색인 순차 탐색은 주 자료 리스트에서 일정 간격으로 발췌한 자료를 가진 인덱스 테이블을 이용한다. 근거 대조 문구: 일정 간격으로 발췌	(A 13.3 정렬된 배열에서의 색인 순차 탐색, 인쇄 p.506 / PDF p.497)
U14-021	색인 순차 탐색에서는 주 자료 리스트와 인덱스 테이블이 모두 정렬되어 있어야 한다. 근거 대조 문구: 모두 정렬되어 있어야	(A 13.3 정렬된 배열에서의 색인 순차 탐색, 인쇄 p.506 / PDF p.497)
U14-022	인덱스 크기를 줄이면 주 자료 리스트 탐색 시간이 늘고 크게 하면 인덱스 탐색 시간이 는다. 근거 대조 문구: 인덱스 테이블의 크기를 줄이면	(A 13.3 색인 순차 탐색, 인쇄 p.507 / PDF p.498)
U14-023	데이터 수 증가로 1차 인덱스가 커지면 2차 인덱스를 사용할 수 있다. 근거 대조 문구: 2차 인덱스 테이블	(A 13.3 색인 순차 탐색, 인쇄 p.507 / PDF p.498)
U14-024	보간 탐색(interpolation search)은 탐색키가 존재할 위치를 예측하여 탐색하는 방법이다. 근거 대조 문구: 존재할 위치를 예측	(A 13.3 보간 탐색, 인쇄 p.507 / PDF p.498)
U14-025	보간 탐색은 리스트를 불균등하게 분할하고 값과 위치가 비례한다고 가정한다. 근거 대조 문구: 값과 위치는 비례한다는 가정	(A 13.3 보간 탐색, 인쇄 p.508 / PDF p.499)
U14-026	보간 탐색으로 계산한 실수 위치는 보통 소수점 이하를 버려 정수로 변환한다. 근거 대조 문구: 소수점 이하를 버리는 방법	(A 13.3 보간 탐색, 인쇄 p.509 / PDF p.500)
U14-027	A는 비교적 균등하게 분포한 많은 데이터에서 보간 탐색이 이진 탐색보다 우수할 수 있다고 설명한다. 근거 대조 문구: 비교적 균등하게 분포	(A 13.3 보간 탐색, 인쇄 p.509 / PDF p.500)
U14-028	bsearch()는 C 언어 표준 라이브러리의 이진 탐색 함수이다. 근거 대조 문구: 이진 탐색을 구현한	(B 6.3.4 C 언어 표준 라이브러리의 이진 탐색 함수: bsearch(), 인쇄 p.248 / PDF p.255)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-029	bsearch()의 매개 변수는 목표값 주소 key, 배열 주소 base, 요소 수 num, 요소 바이트 크기 width, 비교 함수 포인터이다. 근거 대조 문구: 찾고자 하는 목표값 데이터의 주소	(B 6.3.4 bsearch(), 인쇄 p.248 / PDF p.255)
U14-030	데이터 중앙의 요소를 고른다. 근거 대조 문구: 데이터 중앙에 있는 요소	(B 6.3 이진 탐색, 인쇄 p.242 / PDF p.249)
U14-031	중앙 요소값과 목표값을 비교한다. 근거 대조 문구: 목표값을 비교	(B 6.3 이진 탐색, 인쇄 p.242 / PDF p.249)
U14-032	목표값이 작으면 왼편, 크면 오른편 데이터에 이진 탐색을 새로 수행한다. 근거 대조 문구: 데이터 왼편	(B 6.3 이진 탐색, 인쇄 p.242 / PDF p.249)
U14-033	찾고자 하는 값을 찾을 때까지 앞 단계를 반복한다. 근거 대조 문구: 단계를 반복	(B 6.3 이진 탐색, 인쇄 p.242 / PDF p.249)
U14-034	인덱스에서 index[i] <= key < index[i+1]을 만족하는 항목을 찾는다. 근거 대조 문구: index[i] <= key	(A 13.3 색인 순차 탐색, 인쇄 p.506 / PDF p.497)
U14-035	해당 인덱스가 가리키는 주 자료 리스트의 구간을 순차 탐색한다. 근거 대조 문구: 주 자료 리스트에서 순차 탐색	(A 13.3 색인 순차 탐색, 인쇄 p.506 / PDF p.497)

U14-036	seq_search() 원문 발췌 (원문 코드 발췌)
	int seq_search(int key, int low, int high)
	{
	int i;
	for (i = low; i <= high; i++)
	if (list[i] == key)
	return i; // 탐색에 성공하면 키 값의 인덱스 반환
	return -1; // 탐색에 실패하면 -1 반환
	}
	근거 대조 문구: int seq_search
		(A 13.2 프로그램 13.1, 인쇄 p.497 / PDF p.488)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-037	성공 시 순차 탐색 평균 비교 횟수: (1+2+3+...+n)/n = (n+1)/2; 모든 키가 탐색될 확률이 같다고 가정한다. 근거 대조 문구: (n+1)/2	(A 13.2 순차 탐색의 시간 복잡도, 인쇄 p.500 / PDF p.491)
U14-038	색인 순차 탐색의 복잡도: O(m+n/m); m은 인덱스 테이블 크기, n은 주 자료 리스트 크기이다. 근거 대조 문구: O(m+n/m)	(A 13.3 색인 순차 탐색, 인쇄 p.507 / PDF p.498)
U14-039	보간 탐색 위치: (k-list[low])/(list[high]-list[low]) × (high-low) + low; k는 탐색키, low와 high는 탐색 범위의 최소·최대 인덱스이다. 근거 대조 문구: 탐색 위치	(A 13.3 보간 탐색, 인쇄 p.508 / PDF p.499)
U14-040	순차 탐색 근거 대조 문구: 순차 탐색	(A 13.2 정렬되지 않은 배열에서의 탐색, 인쇄 p.497 / PDF p.488)
U14-041	이진 탐색 근거 대조 문구: 이진 탐색	(A 13.3 정렬된 배열에서의 탐색, 인쇄 p.500 / PDF p.491)
U14-042	색인 순차 탐색 근거 대조 문구: 색인 순차 탐색	(A 13.3 정렬된 배열에서의 색인 순차 탐색, 인쇄 p.506 / PDF p.497)
U14-043	보간 탐색 근거 대조 문구: 보간 탐색	(A 13.3 보간 탐색, 인쇄 p.507 / PDF p.498)
U14-044	전진 이동법 근거 대조 문구: 전진 이동법	(B 6.2.1 전진 이동법, 인쇄 p.237 / PDF p.244)
U14-045	전위법 근거 대조 문구: 전위법	(B 6.2.2 전위법, 인쇄 p.239 / PDF p.246)
U14-046	계수법 근거 대조 문구: 계수법	(B 6.2.3 계수법, 인쇄 p.241 / PDF p.248)

15 해싱·해시 테이블

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-001	해싱(hashing)은 키에 산술 연산을 적용하여 항목이 저장된 테이블 주소를 계산하고 접근하는 탐색이다. 근거 대조 문구: 키(key)에 산술적인 연산	(A 14.1 해싱이란?, 인쇄 p.534 / PDF p.525)
U15-002	적재 밀도(loading density) 또는 적재 비율(loading factor)은 저장 항목 수 n과 테이블 크기 M의 비율이다. 근거 대조 문구: 저장되는 항목의 개수 n	(A 14.7 해싱의 성능 분석, 인쇄 p.564 / PDF p.555)
U15-003	키 연산으로 직접 접근 가능한 구조를 해시 테이블(hash table)이라 한다. 근거 대조 문구: 직접 접근이 가능한 구조	(A 14.1 해싱이란?, 인쇄 p.534 / PDF p.525)
U15-004	사전(dictionary)은 (키, 값) 쌍의 집합이다. 근거 대조 문구: (키, 값) 쌍의 집합	(A 14.2 추상 자료형 사전, 인쇄 p.535 / PDF p.526)
U15-005	사전은 항목 위치가 아니라 키에 의하여 관리되며 정렬 여부는 구현할 때 결정한다. 근거 대조 문구: 사전은 오직 키에 의해서만 관리	(A 14.2 추상 자료형 사전, 인쇄 p.536 / PDF p.527)
U15-006	사전의 핵심 연산은 add(key, value), delete(key), search(key)이다. 근거 대조 문구: add(key, value)	(A 14.2 ADT 14.1, 인쇄 p.536 / PDF p.527)
U15-007	search(key)는 관련 value를 반환하며 탐색 실패 시 NULL을 반환한다. 근거 대조 문구: 탐색이 실패하면 NULL	(A 14.2 ADT 14.1, 인쇄 p.536 / PDF p.527)
U15-008	해시 함수(hash function)는 키를 입력받아 해시 주소(hash address)를 생성하고 이를 테이블 인덱스로 사용한다. 근거 대조 문구: 해시 주소(hash address)	(A 14.3 해싱의 구조, 인쇄 p.537 / PDF p.528)
U15-009	해시 테이블 ht는 M개 버킷(bucket)으로 이루어지며 하나의 버킷은 여러 슬롯(slot)을 가질 수 있다. 근거 대조 문구: M개의 버킷	(A 14.3 해싱의 구조, 인쇄 p.537 / PDF p.528)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-010	하나의 슬롯에는 하나의 항목이 저장된다. 근거 대조 문구: 하나의 슬롯에는 하나의 항목	(A 14.3 해싱의 구조, 인쇄 p.537 / PDF p.528)
U15-011	서로 다른 키 k1, k2에 대해 h(k1)=h(k2)이면 충돌(collision)이며 두 키는 동의어(synonym)이다. 근거 대조 문구: 동의어(synonym)	(A 14.3 해싱의 구조, 인쇄 p.538 / PDF p.529)
U15-012	충돌이 버킷의 슬롯 수보다 많이 발생하여 항목을 더 저장할 수 없으면 오버플로우(overflow)이다. 근거 대조 문구: 오버플로우(overflow)	(A 14.3 해싱의 구조, 인쇄 p.538 / PDF p.529)
U15-013	버킷당 슬롯이 하나이면 충돌이 곧 오버플로우를 의미한다. 근거 대조 문구: 충돌이 곧 오버플로우	(A 14.3 해싱의 구조, 인쇄 p.538 / PDF p.529)
U15-014	좋은 해시 함수는 충돌이 적고 주소 영역에 고르게 분포하며 계산이 빨라야 한다. 근거 대조 문구: 계산이 빨라야 한다	(A 14.4 해시함수, 인쇄 p.542 / PDF p.533)
U15-015	제산 함수는 키를 테이블 크기로 나눈 나머지를 해시 주소로 사용한다. 근거 대조 문구: 나눈 나머지	(A 14.4 제산 함수, 인쇄 p.543 / PDF p.534)
U15-016	폴딩은 키를 여러 부분으로 나누어 더하거나 비트별 XOR와 같은 연산으로 해시 주소를 만드는 방법이다. 근거 대조 문구: 키를 몇 개의 부분으로 나누어	(A 14.4 폴딩 함수, 인쇄 p.544 / PDF p.535)
U15-017	이동 폴딩은 나눈 부분의 값을 더하고 경계 폴딩은 이웃한 부분을 거꾸로 더한다. 근거 대조 문구: 경계 폴딩	(A 14.4 폴딩 함수, 인쇄 p.544 / PDF p.535)
U15-018	중간 제곱 함수는 키를 제곱한 다음 중간의 몇 비트를 해시 주소로 사용한다. 근거 대조 문구: 키를 제곱한 다음	(A 14.4 중간 제곱 함수, 인쇄 p.545 / PDF p.536)
U15-019	비트 추출은 테이블 크기 M=2^k일 때 키를 이진수로 보고 임의 위치의 k개 비트를 해시 주소로 사용한다. 근거 대조 문구: 임의의 위치의 k개의 비트	(A 14.4 비트 추출 방법, 인쇄 p.545 / PDF p.536)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-020	숫자 분석 방법은 키의 각 위치의 수 특징을 알고 있을 때 편중되지 않은 숫자를 조합하여 해시 주소로 사용한다. 근거 대조 문구: 편중되지 않는 수들을	(A 14.4 숫자 분석 방법, 인쇄 p.545 / PDF p.536)
U15-021	A는 문자열 문자들의 코드값에 위치에 기초한 값을 곱하고 호너의 방법(Horner's method)으로 계산량을 줄이는 방법을 설명한다. 근거 대조 문구: 호너의 방법	(A 14.4 탐색키가 문자열일 경우 주의할 점, 인쇄 p.545-546 / PDF p.536-537)
U15-022	B의 자릿수 접기(Folding)는 숫자를 자릿수별로 더하여 해시값을 만드는 방법이다. 근거 대조 문구: 숫자를 각 자릿수별로 더해	(B 8.2.2 자릿수 접기, 인쇄 p.343 / PDF p.347)
U15-023	개방 주소법(open addressing)은 충돌이 일어난 항목을 테이블의 다른 위치에 저장한다. 근거 대조 문구: 해시 테이블의 다른 위치	(A 14.5 개방 주소법, 인쇄 p.547 / PDF p.538)
U15-024	선형 조사법은 빈 공간이 나올 때까지 다음 버킷을 차례로 조사하고 끝에서는 처음으로 돌아간다. 근거 대조 문구: 테이블의 처음으로 간다	(A 14.5 선형 조사법, 인쇄 p.547 / PDF p.538)
U15-025	조사 시작 위치로 되돌아오면 테이블이 가득 찬 것으로 판단한다. 근거 대조 문구: 조사를 시작했던 곳으로 되돌아오게	(A 14.5 선형 조사법, 인쇄 p.547 / PDF p.538)
U15-026	선형 조사법에서 삭제를 지원하려면 미사용·사용 후 공백·현재 사용 중인 버킷을 구분해야 한다. 근거 대조 문구: 현재 사용 중인 버킷	(A 14.5 선형 조사법, 인쇄 p.554 / PDF p.545)
U15-027	삭제가 있는 선형 조사법의 탐색은 한 번도 사용하지 않은 버킷을 만나야 중단한다. 근거 대조 문구: 한 번도 사용이 안 된 버킷	(A 14.5 선형 조사법, 인쇄 p.554 / PDF p.545)
U15-028	이차 조사법은 다음 위치를 (h(k)+inc*inc) mod M으로 결정한다. 근거 대조 문구: (h(k)+inc*inc) mod M	(A 14.5 이차 조사법, 인쇄 p.554 / PDF p.545)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-029	A의 이중 해싱법(double hashing) 또는 재해싱(rehashing)은 다음 저장 위치를 별개의 해시 함수로 결정한다. 근거 대조 문구: 재해싱(rehashing)	(A 14.5 이중 해싱법, 인쇄 p.555 / PDF p.546)
U15-030	B의 이중 해싱은 최초 주소용 해시 함수와 충돌 시 이동폭을 얻는 두 번째 해시 함수를 사용한다. 근거 대조 문구: 2개의 해시 함수를 준비	(B 8.3.2 이중 해싱, 인쇄 p.363 / PDF p.367)
U15-031	B는 같은 해시값을 가진 데이터가 제곱 탐사에서 2차 클러스터를 형성할 수 있다고 설명한다. 근거 대조 문구: 2차 클러스터	(B 8.3.2 제곱 탐사, 인쇄 p.362 / PDF p.366)
U15-032	B의 OAHT_Rehash()는 두 배 크기의 새 테이블을 만들고 점유 항목을 옮긴 뒤 이전 테이블을 소멸시킨다. 근거 대조 문구: TableSize * 2	(B 8.3.2 OAHT_Rehash(), 인쇄 p.371 / PDF p.375)
U15-033	체이닝(chaining)은 각 버킷에 연결 리스트를 할당하여 오버플로우를 해결한다. 근거 대조 문구: 각 버킷에	(A 14.6 체이닝, 인쇄 p.559 / PDF p.550)
U15-034	체이닝에서 원하는 항목은 버킷의 연결 리스트를 순차 탐색하여 찾는다. 근거 대조 문구: 연결 리스트를 순차 탐색	(A 14.6 체이닝, 인쇄 p.559 / PDF p.550)
U15-035	중복 키를 허용하면 리스트 처음에 삽입하는 것이 능률적이며 허용하지 않으면 먼저 리스트를 탐색한다. 근거 대조 문구: 키들의 중복을 허용	(A 14.6 체이닝, 인쇄 p.559 / PDF p.550)
U15-036	B는 테이블 주소 바깥에 공간을 할당하는 것을 개방 해싱, 테이블 내부 공간에서 해결하는 것을 폐쇄 해싱이라 부른다. 근거 대조 문구: 개방 해싱	(B 8.3 충돌 해결 기법, 인쇄 p.346 / PDF p.350)
U15-037	B의 체이닝은 각 테이블 요소가 데이터 대신 링크드 리스트의 포인터를 관리한다. 근거 대조 문구: 링크드 리스트에 대한 포인터	(B 8.3.1 체이닝, 인쇄 p.347 / PDF p.351)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-038	이상적인 해싱의 시간 복잡도 O(1)은 충돌이 전혀 없다는 가정에서 가능하다. 근거 대조 문구: 충돌은 전혀 일어나지 않는다는 가정	(A 14.7 해싱의 성능 분석, 인쇄 p.564 / PDF p.555)
U15-039	선형 조사법에서 적재 밀도 최댓값은 1이며 체이닝에서는 항목 수가 테이블 크기를 넘을 수 있어 최댓값이 없다. 근거 대조 문구: 최대값을 가지지 않는다	(A 14.7 해싱의 성능 분석, 인쇄 p.564-565 / PDF p.555-556)
U15-040	체이닝에서 적재 밀도 α는 평균 연결 리스트당 항목 수이며 실패 탐색의 평균 비교 수는 α, 성공 탐색은 1+α/2이다. 근거 대조 문구: 1+α/2	(A 14.7 해싱의 성능 분석, 인쇄 p.565 / PDF p.556)
U15-041	A는 선형 조사법 적재 밀도를 0.5 이하, 이차 조사법·이중 해싱법은 0.7 이하로 유지하는 것이 좋다고 정리한다. 근거 대조 문구: 0.7이하	(A 14.7 정리 문단, 인쇄 p.566 / PDF p.557)
U15-042	모든 값이 하나의 버킷으로 집중되는 최악의 해싱 시간 복잡도는 O(n)이다. 근거 대조 문구: 시간 복잡도는 O(n)	(A 14.7 해싱의 성능 분석, 인쇄 p.567 / PDF p.558)
U15-043	A가 제시하는 해싱 응용은 데이터베이스 인덱싱·컴파일러 심볼 테이블·인터넷 검색 엔진이다. 근거 대조 문구: 심볼 테이블	(A 14.8 해싱의 응용 분야, 인쇄 p.568 / PDF p.559)
U15-044	충돌 위치 ht[k]의 다음 버킷 ht[k+1]을 조사한다. 근거 대조 문구: ht[k+1]	(A 14.5 선형 조사법, 인쇄 p.547 / PDF p.538)
U15-045	빈 버킷이 없으면 ht[k+2] 등으로 계속 조사하며 테이블 끝에서는 처음으로 간다. 근거 대조 문구: ht[k+2]	(A 14.5 선형 조사법, 인쇄 p.547 / PDF p.538)
U15-046	기존 테이블 크기의 두 배인 새 해시 테이블을 만든다. 근거 대조 문구: TableSize * 2	(B 8.3.2 OAHT_Rehash() , 인쇄 p.371 / PDF p.375)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-047	이전 테이블의 OCCUPIED 항목을 OAHT_Set()으로 새 테이블에 넣는다. 근거 대조 문구: OCCUPIED	(B 8.3.2 OAHT_Rehash(), 인쇄 p.371 / PDF p.375)
U15-048	이전 테이블을 소멸시키고 HT 포인터에 새 테이블 주소를 대입한다. 근거 대조 문구: (*HT) = NewHT	(B 8.3.2 OAHT_Rehash(), 인쇄 p.371 / PDF p.375)

U15-049	음수 결과를 처리하는 hash_function() 발췌 (원문 코드 발췌)
	int hash_function(int key)
	{
	int hash_index = key % M;
	if (hash_index < 0)
	hash_index += M;
	return hash_index;
	}
	근거 대조 문구: int hash_function
		(A 14.4 제산 함수, 인쇄 p.543 / PDF p.534)

U15-050	제산 해시 함수: h(k) = k mod M; M은 해시 테이블 크기이다. 근거 대조 문구: h(k)=k mod M	(A 14.4 제산 함수, 인쇄 p.543 / PDF p.534)
U15-051	적재 밀도: α = n/M; n은 저장된 항목 수, M은 해시 테이블의 버킷 수이다. 근거 대조 문구: n/M	(A 14.7 해싱의 성능 분석, 인쇄 p.564 / PDF p.555)
U15-052	선형 조사법의 탐색 비교 연산 수: 실패: ½{1+1/(1-α)²}, 성공: ½{1+1/(1-α)}; α는 적재 밀도이다. 근거 대조 문구: 실패한 탐색	(A 14.7 선형 조사법의 비교 연산 수, 인쇄 p.565 / PDF p.556)
U15-053	개방 주소법 근거 대조 문구: 개방 주소법	(A 14.5 개방 주소법, 인쇄 p.547 / PDF p.538)
U15-054	선형 조사법 근거 대조 문구: 선형 조사법	(A 14.5 선형 조사법, 인쇄 p.547 / PDF p.538)
U15-055	이차 조사법 근거 대조 문구: 이차 조사법	(A 14.5 이차 조사법, 인쇄 p.554 / PDF p.545)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-056	이중 해싱법 근거 대조 문구: 이중 해싱법	(A 14.5 이중 해싱법, 인쇄 p.555 / PDF p.546)
U15-057	체이닝 근거 대조 문구: 체이닝	(A 14.6 체이닝, 인쇄 p.559 / PDF p.550)
U15-058	적재 밀도가 작은 경우에는 선형 조사법이 이차 조사법이나 이중 해싱보다 효율적일 수 있다. 근거 대조 문구: 적재 밀도가 작은 경우	(A 14.7 정리 문단, 인쇄 p.566 / PDF p.557)
U15-059	체인법은 적재 밀도에 비례하는 성능을 보이며 링크용 메모리 낭비는 저장 자료 크기에 따라 달라진다. 근거 대조 문구: 적재 밀도에 비례하는 성능	(A 14.7 정리 문단, 인쇄 p.567 / PDF p.558)

16 문자열 탐색

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-001	본문(Text)은 탐색 대상이 되는 문자열이다. 근거 대조 문구: 본문Text: 탐색 대상이 되는 문자열을 의미합니다.	(B 10.1 문자열 탐색 알고리즘의 개요, 인쇄 p.457 / PDF p.458)
U16-002	나쁜 문자 이동 테이블의 모든 요소를 -1로 초기화하고 패턴을 왼쪽에서 오른쪽으로 읽으며 각 문자의 위치를 기록한다. 근거 대조 문구: 모든 요소를 -1로 초기화	(B 10.5.3 보이어-무어 알고리즘의 전처리 과정, 인쇄 p.484 / PDF p.485)
U16-003	패턴(Pattern)은 탐색어를 의미한다. 근거 대조 문구: 패턴Pattern: 탐색어를 의미합니다.	(B 10.1 문자열 탐색 알고리즘의 개요, 인쇄 p.457 / PDF p.458)
U16-004	이동(Shift)은 본문에서 탐색 위치를 옮기는 일이다. 근거 대조 문구: 이동Shift: 본문에서 탐색 위치를 옮기는 일입니다.	(B 10.1 문자열 탐색 알고리즘의 개요, 인쇄 p.457 / PDF p.458)
U16-005	고지식한 탐색은 본문의 앞에서부터 끝까지 차근차근 탐색하는 알고리즘이다. 근거 대조 문구: 본문의 앞에서부터 끝까지 차근차근 탐색하는 알고리즘	(B 10.2 고지식한 탐색 알고리즘, 인쇄 p.458 / PDF p.459)
U16-006	본문 문자열을 처음부터 끝까지 차례대로 순회하면서 패턴 내 문자들을 일일이 비교한다. 근거 대조 문구: 패턴 내 문자들을 일일이 비교하는 방식	(B 10.2.1 고지식한 탐색의 동작 방식, 인쇄 p.458 / PDF p.459)
U16-007	본문과 패턴의 문자가 다르면 본문의 다음 문자로 이동한다. 근거 대조 문구: 본문의 다음 문자로 이동합니다.	(B 10.2.1 고지식한 탐색의 동작 방식, 인쇄 p.458 / PDF p.459)
U16-008	본문 길이가 N, 패턴 길이가 M일 때 최악의 경우 N×M번의 비교를 수행한다. 근거 대조 문구: 최악의 경우 N×M번의 비교를 수행합니다.	(B 10.2.1 고지식한 탐색의 동작 방식, 인쇄 p.459 / PDF p.460)
U16-009	카프-라빈 알고리즘은 문자열 탐색을 위해 해시 함수를 이용한다. 근거 대조 문구: 문자열 탐색을 위해 해시 함수를 이용합니다.	(B 10.3.1 카프-라빈 알고리즘의 동작 방식, 인쇄 p.463 / PDF p.464)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-010	패턴의 해시값과 본문 내 하위 문자열의 해시값을 비교한다. 근거 대조 문구: 패턴의 해시값과 본문 내에 있는 하위 문자열의 해시값만 비교	(B 10.3.1 카프-라빈 알고리즘의 동작 방식, 인쇄 p.463 / PDF p.464)
U16-011	새 해시 함수는 앞 단계의 해시값과 제거된 문자, 새로 추가된 문자를 이용한다. 근거 대조 문구: 앞 단계에서 계산해놓은	(B 10.3.1 카프-라빈 알고리즘의 동작 방식, 인쇄 p.464 / PDF p.465)
U16-012	이전 해시값이 없는 최초의 해시값은 기존의 해시 함수로 구한다. 근거 대조 문구: 최초의 해시값을 구할 때는 기존의 해시 함수를 사용	(B 10.3.1 카프-라빈 알고리즘의 동작 방식, 인쇄 p.464 / PDF p.465)
U16-013	해시 함수 결과를 임의의 값 q로 나눈 나머지를 해시값으로 사용한다. 근거 대조 문구: 임의의 값으로 나눈 나머지를 해시값으로 사용	(B 10.3.1 카프-라빈 알고리즘의 동작 방식, 인쇄 p.465 / PDF p.466)
U16-014	패턴과 일치하는 해시값을 가진 문자열을 찾으면 실제로 각 문자가 패턴의 문자와 일치하는지 한 차례 더 확인한다. 근거 대조 문구: 실제로 각 문자가 패턴의 문자와 일치하는지 한 차례 더 확인	(B 10.3.1 카프-라빈 알고리즘의 동작 방식, 인쇄 p.466 / PDF p.467)
U16-015	서로 다른 문자열에서 얻은 해시값이 동일한 충돌이 생길 수 있다. 근거 대조 문구: 서로 다른 문자열에서 얻어낸 해시값이 동일한 경우(충돌)	(B 10.3.1 카프-라빈 알고리즘의 동작 방식, 인쇄 p.466 / PDF p.467)
U16-016	KMP 알고리즘은 탐색 위치를 본문 왼쪽부터 오른쪽으로 옮기며 문자를 직접 비교한다. 근거 대조 문구: 탐색 위치를 본문 왼쪽부터 시작해서 오른쪽으로 옮겨가며 문자를 직접 비교	(B 10.4 KMP 알고리즘, 인쇄 p.471 / PDF p.472)
U16-017	접두부는 문자열의 머리 부분이고 접미부는 문자열의 꼬리 부분이다. 근거 대조 문구: 접두부는 문자열의 머리 부분을 뜻하고 접미부는 문자열의 꼬리 부분	(B 10.4.1 KMP 알고리즘의 동작 방식, 인쇄 p.471 / PDF p.472)
U16-018	빈 문자열도 접두부와 접미부가 될 수 있다. 근거 대조 문구: 빈 문자열도 접두부와 접미부가 될 수 있습니다.	(B 10.4.1 KMP 알고리즘의 동작 방식, 인쇄 p.471 / PDF p.472)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-019	경계(Border)는 문자열에서 일치하는 접두부와 접미부 쌍이다. 근거 대조 문구: 문자열에서 일치하는 접두부와 접미부 쌍을 가리켜 경계	(B 10.4.1 KMP 알고리즘의 동작 방식, 인쇄 p.472 / PDF p.473)
U16-020	KMP 알고리즘은 경계를 활용해서 불필요한 문자 비교를 피한다. 근거 대조 문구: 이 경계를 활용해서 불필요한 문자 비교를 피합니다.	(B 10.4.1 KMP 알고리즘의 동작 방식, 인쇄 p.472 / PDF p.473)
U16-021	탐색을 수행하기 전에 패턴으로부터 경계 정보를 가진 테이블을 만든다. 근거 대조 문구: 탐색을 수행하기 전에 미리 패턴으로부터 경계의 정보를 가진 테이블	(B 10.4.2 경계 정보 사전 계산 방법, 인쇄 p.473 / PDF p.474)
U16-022	경계 테이블의 크기는 패턴 크기보다 1만큼 더 크다. 근거 대조 문구: 경계 테이블의 크기가 패턴 크기보다 1만큼 더 큽니다.	(B 10.4.2 경계 정보 사전 계산 방법, 인쇄 p.475 / PDF p.476)
U16-023	보이어-무어 알고리즘은 문자열을 오른쪽에서 왼쪽으로 비교하지만 이동은 왼쪽에서 오른쪽으로 이루어진다. 근거 대조 문구: 문자열을 오른쪽에서 왼쪽으로 비교해나갑니다.	(B 10.5 보이어-무어 알고리즘, 인쇄 p.480 / PDF p.481)
U16-024	불일치가 발생하면 나쁜 문자 이동과 착한 접미부 이동을 모두 검토해서 더 멀리 옮겨가는 방법을 선택한다. 근거 대조 문구: 더 멀리 옮겨가는 이동 방법을 선택	(B 10.5 보이어-무어 알고리즘, 인쇄 p.480 / PDF p.481)
U16-025	나쁜 문자는 본문과 패턴을 불일치하게 만드는 본문의 문자이다. 근거 대조 문구: 본문의 문자	(B 10.5.1 나쁜 문자 이동, 인쇄 p.480 / PDF p.481)
U16-026	불일치 문자와 동일한 문자가 패턴 내에서 2개 이상 나오면 가장 오른쪽에 있는 것을 본문의 불일치 문자에 맞춘다. 근거 대조 문구: 발견된 문자 중 가장 오른쪽에 있는 것	(B 10.5.1 나쁜 문자 이동, 인쇄 p.481 / PDF p.482)
U16-027	착한 접미부는 본문과 일치하는 패턴의 접미부이다. 근거 대조 문구: 일치하는 접미부를 착한 접미부	(B 10.5.2 착한 접미부 이동, 인쇄 p.482 / PDF p.483)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-028	착한 접미부와 동일한 문자열이 패턴의 나머지 부분에 존재하면 그 부분을 본문의 착한 접미부 위치에 맞춘다. 근거 대조 문구: 본문의 착한 접미부 위치와 일치하도록 패턴을 이동	(B 10.5.2 착한 접미부 이동, 인쇄 p.482 / PDF p.483)
U16-029	첫 번째 경우가 아니면 착한 접미부의 접미부와 일치하는 패턴의 접두부가 같은 위치에 놓이도록 이동한다. 근거 대조 문구: 착한 접미부의 접미부	(B 10.5.2 착한 접미부 이동, 인쇄 p.482-483 / PDF p.483-484)
U16-030	착한 접미부 이동의 두 경우에 모두 해당하지 않으면 패턴의 전체 길이만큼 오른쪽으로 이동한다. 근거 대조 문구: 패턴의 전체 길이만큼 탐색 위치를 오른쪽으로 이동	(B 10.5.2 착한 접미부 이동, 인쇄 p.483 / PDF p.484)
U16-031	같은 문자가 여러 번 나타나도 나쁜 문자 이동 테이블에는 가장 마지막에 등장한 문자의 위치만 남는다. 근거 대조 문구: 가장 마지막에 등장한 문자의 위치만 테이블에 남습니다.	(B 10.5.3 보이어-무어 알고리즘의 전처리 과정, 인쇄 p.484 / PDF p.485)
U16-032	착한 접미부 이동 테이블은 패턴으로부터 접미부의 이동 거리를 미리 계산해 기록한다. 근거 대조 문구: 접미부의 이동 거리를 미리 계산해서 테이블에 기록	(B 10.5.3 보이어-무어 알고리즘의 전처리 과정, 인쇄 p.484 / PDF p.485)
U16-033	본문과 패턴의 불일치가 발생하면 본문의 불일치한 문자와 같은 문자를 패턴에서 찾는다. 근거 대조 문구: 패턴에서 찾습니다.	(B 10.5.1 나쁜 문자 이동, 인쇄 p.480 / PDF p.481)
U16-034	찾은 패턴의 불일치 문자 위치를 본문의 불일치 문자 위치와 일치하도록 패턴을 이동한다. 근거 대조 문구: 패턴을 이동시킵니다.	(B 10.5.1 나쁜 문자 이동, 인쇄 p.480 / PDF p.481)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U16-035	ReHash() 원문 발췌 (원문 코드 발췌)
	int ReHash(char* String, int Start, int Size, int HashPrev, int Coefficient)
	{
	if (Start == 0)
	return HashPrev;
	
	return String[Start + Size - 1] +
	((HashPrev - Coefficient * String [Start-1]) * 2);
	}
	근거 대조 문구: int ReHash(char* String, int Start, int Size, int HashPrev, int Coefficient)
		(B 10.3.2 카프-라빈 알고리즘 예제 프로그램, 인쇄 p.469 / PDF p.470)

U16-036	카프-라빈의 최초 해시값: H_i = (S[i]×2^(m−1) + S[i+1]×2^(m−2) + ⋯ + S[i+m−2]×2¹ + S[i+m−1]×2⁰) mod q; i=0. S는 문자열, S[n]은 n번째 문자, m은 문자열 길이, q는 해시값 범위를 제한하는 값. 근거 대조 문구: i=0	(B 10.3.1 카프-라빈 알고리즘의 동작 방식, 인쇄 p.465 / PDF p.466)
U16-037	카프-라빈의 다음 해시값: H_i = (2(H_{i-1} − S[i−1]×2^(m−1)) + S[i+m−1]) mod q; i>0. 앞 단계 해시값 H_{i-1}을 이용한다. 근거 대조 문구: i>0	(B 10.3.1 카프-라빈 알고리즘의 동작 방식, 인쇄 p.465 / PDF p.466)
U16-038	KMP 이동 거리: 이동 거리 = 일치 접두부의 길이 − 최대 경계 너비 근거 대조 문구: 이동 거리 = 일치 접두부의 길이 - 최대 경계 너비	(B 10.4.2 경계 정보 사전 계산 방법, 인쇄 p.475 / PDF p.476)
U16-039	고지식한 탐색 근거 대조 문구: 고지식한 탐색	(B 10.2, 인쇄 p.458 / PDF p.459)
U16-040	카프-라빈 알고리즘 근거 대조 문구: 카프-라빈 알고리즘	(B 10.3, 인쇄 p.463 / PDF p.464)
U16-041	KMP 알고리즘 근거 대조 문구: KMP 알고리즘	(B 10.4, 인쇄 p.472 / PDF p.473)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-042	보이어-무어 알고리즘 근거 대조 문구: 보이어-무어 알고리즘	(B 10.5, 인쇄 p.480 / PDF p.481)

17 분할 정복

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-001	분할 정복 알고리즘은 문제를 더 이상 나눌 수 없을 때까지 나누고, 나뉜 문제들을 각각 풀어 전체 문제의 답을 얻는 기법이다. 근거 대조 문구: 문제를 더 이상 나눌 수 없을 때까지 나누고	(B 12.1.2 분할 정복 알고리즘의 개념, 인쇄 p.524 / PDF p.524)
U17-002	피보나치 재귀 방정식을 그대로 옮긴 구현은 Fibonacci(n−1)과 Fibonacci(n−2)를 재귀 호출한다. 근거 대조 문구: 2개의 재귀 호출	(B 12.4.1 피보나치 수를 구하는 방법, 인쇄 p.538 / PDF p.538)
U17-003	분할 정복 알고리즘의 핵심은 문제를 잘게 나누는 것이다. 근거 대조 문구: 문제를 잘게 나누는 것	(B 12.1.2 분할 정복 알고리즘의 개념, 인쇄 p.524 / PDF p.524)
U17-004	분할은 문제가 분할 가능한 경우 2개 이상의 하위 문제로 나누는 단계이다. 근거 대조 문구: 2개 이상의 하위 문제로 나눕니다.	(B 12.1.2 분할 정복 알고리즘의 개념, 인쇄 p.525 / PDF p.525)
U17-005	정복은 하위 문제가 여전히 분할 가능하면 분할을 수행하고, 그렇지 않으면 하위 문제를 푸는 단계이다. 근거 대조 문구: 그렇지 않다면 하위 문제를 풉니다.	(B 12.1.2 분할 정복 알고리즘의 개념, 인쇄 p.525 / PDF p.525)
U17-006	결합은 정복을 통해 구한 답을 취합하는 단계이다. 근거 대조 문구: 구한 답을 취합합니다.	(B 12.1.2 분할 정복 알고리즘의 개념, 인쇄 p.525 / PDF p.525)
U17-007	분할 정복 기법으로 문제를 나눌 때 생기는 각 하위 문제는 완전히 독립적이다. 근거 대조 문구: 각 하위 문제는 완전히 독립적입니다.	(B 12.1.2 분할 정복 알고리즘의 개념, 인쇄 p.525 / PDF p.525)
U17-008	분할 정복 구현에는 재귀 호출이 많이 사용되며 재귀 호출 비용이 알고리즘의 효율성을 깎아내리기도 한다. 근거 대조 문구: 효율성을 재귀 호출 비용이 깎아내리기도	(B 12.1.2 분할 정복 알고리즘의 개념, 인쇄 p.525 / PDF p.525)
U17-009	병합 정렬은 퀵 정렬과 같이 분할 정복에 기반한 알고리즘이다. 근거 대조 문구: 분할 정복에 기반한 알고리즘	(B 12.2 병합 정렬, 인쇄 p.525 / PDF p.525)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-010	정렬할 데이터를 반으로 나눈다. 근거 대조 문구: 정렬할 데이터를 반으로 나눕니다.	(B 12.2.1 병합 정렬 동작 방식, 인쇄 p.526 / PDF p.526)
U17-011	나뉜 하위 데이터의 크기가 2 이상이면 그 하위 데이터를 다시 반으로 나눈다. 근거 대조 문구: 하위 데이터의 크기가 2 이상이면	(B 12.2.1 병합 정렬 동작 방식, 인쇄 p.526 / PDF p.526)
U17-012	동일 데이터에서 나뉜 하위 데이터 둘을 병합하며 원소를 정렬 기준에 따라 정렬한다. 근거 대조 문구: 병합할 때 데이터의 원소는 정렬 기준에 따라 정렬	(B 12.2.1 병합 정렬 동작 방식, 인쇄 p.526 / PDF p.526)
U17-013	데이터가 원래대로 모두 하나가 될 때까지 병합을 반복한다. 근거 대조 문구: 모두 하나가 될 때까지	(B 12.2.1 병합 정렬 동작 방식, 인쇄 p.526 / PDF p.526)
U17-014	두 데이터를 병합할 때는 두 데이터를 합한 것만큼 빈 공간을 마련한다. 근거 대조 문구: 두 데이터를 합한 것만큼 비어 있는 데이터(공간)를 마련	(B 12.2.1 병합 정렬 동작 방식, 인쇄 p.527 / PDF p.527)
U17-015	두 데이터의 첫 번째 요소들을 비교하여 작은 요소를 새 데이터에 추가하고 원래 데이터에서 삭제한다. 근거 대조 문구: 첫 번째 요소들을 비교하여 작은 요소를 새 데이터에 추가	(B 12.2.1 병합 정렬 동작 방식, 인쇄 p.527 / PDF p.527)
U17-016	양쪽 데이터가 빌 때까지 비교와 이동을 반복한다. 근거 대조 문구: 양쪽 데이터가 빌 때까지	(B 12.2.1 병합 정렬 동작 방식, 인쇄 p.527 / PDF p.527)
U17-017	MergeSort()는 데이터를 반으로 나누고, 나눈 데이터를 매개 변수로 재귀 호출한 뒤 다시 병합한다. 근거 대조 문구: 크게 세 가지 작업을 수행	(B 12.2.2 병합 정렬 알고리즘의 구현, 인쇄 p.529 / PDF p.529)
U17-018	원문의 MergeSort()는 하위 데이터용 메모리와 배열을 따로 할당하지 않고 시작·중간·마지막 인덱스로 작업 대상을 식별한다. 근거 대조 문구: 하위 데이터용 메모리와 배열을 따로 할당하지 않습니다.	(B 12.2.2 병합 정렬 알고리즘의 구현, 인쇄 p.530 / PDF p.530)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-019	처음 거듭 제곱 알고리즘은 Exponent 크기만큼 곱셈을 수행하여 Exponent에 대해 O(n)의 수행 시간이 소요된다. 근거 대조 문구: Exponent에 대해 O(n)의 수행 시간	(B 12.3.1 거듭 제곱 계산법, 인쇄 p.533 / PDF p.533)
U17-020	C^8은 C^2을 구한 뒤 제곱을 두 번 더 반복하여 세 번의 곱셈으로 구할 수 있다. 근거 대조 문구: 세 번의 곱셈만 수행	(B 12.3.1 거듭 제곱 계산법, 인쇄 p.533 / PDF p.533)
U17-021	지수가 홀수이면 지수를 2로 나누어떨어지게 할 수 없으므로 C를 한 번 더 곱하는 정리를 이용한다. 근거 대조 문구: 지수가 홀수인 경우	(B 12.3.1 거듭 제곱 계산법, 인쇄 p.534 / PDF p.534)
U17-022	지수를 반으로 나눠가는 거듭 제곱 알고리즘의 수행 시간은 $O(\log_2 n)$이다. 근거 대조 문구: 알고리즘의 수행 시간은 $O(\log_2 n)$	(B 12.3.1 거듭 제곱 계산법, 인쇄 p.534 / PDF p.534)
U17-023	같은 피보나치 호출이 새롭게 생성되는 반복 때문에 원문의 직접 재귀 알고리즘 수행 시간은 $\Theta(2^n)$이다. 근거 대조 문구: 수행 시간)은 $\Theta(2^n)$	(B 12.4.1 피보나치 수를 구하는 방법, 인쇄 p.539 / PDF p.539)
U17-024	분할 정복 기법으로 피보나치 수를 구하는 수행 시간을 $\Theta(\log_2 n)$ 수준으로 줄일 수 있다. 근거 대조 문구: $\Theta(\log_2 n)$ 수준으로 줄일 수 있습니다.	(B 12.4.1 피보나치 수를 구하는 방법, 인쇄 p.539 / PDF p.539)
U17-025	정사각 행렬 A에 대해 $A^nA^m=A^{n+m}$이 성립하고 n=m이면 $A^nA^n=A^{2n}$이다. 근거 대조 문구: 정사각 행렬 A	(B 12.4.2 분할 정복으로 피보나치 수를 구하는 방법, 인쇄 p.539 / PDF p.539)
U17-026	피보나치 행렬의 제곱을 반으로 나누면 n번째 피보나치 수를 구할 때 최초의 행렬을 $\log_2 n$회 제곱하게 된다. 근거 대조 문구: 최초의 행렬을 $\log_2 n$회 제곱	(B 12.4.2 분할 정복으로 피보나치 수를 구하는 방법, 인쇄 p.540 / PDF p.540)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-027	원문의 Fibonacci()는 Matrix2x2 구조체로 행렬을 초기화하고 Matrix2x2_Power()로 N제곱한 뒤 A.Data[0][1]을 반환한다. 근거 대조 문구: return A.Data[0][1];	(B 12.4.3 분할 정복 기반 피보나치 수 구하기 알고리즘의 구현, 인쇄 p.542 / PDF p.542)
U17-028	MergeSort() 분할·재귀·병합 부분 (원문 코드 발췌) MiddleIndex = (StartIndex + EndIndex) / 2; MergeSort(DataSet, StartIndex, MiddleIndex); MergeSort(DataSet, MiddleIndex+1, EndIndex); Merge(DataSet, StartIndex, MiddleIndex, EndIndex); 근거 대조 문구: MiddleIndex = (StartIndex + EndIndex) / 2;	(B 12.2.2 병합 정렬 알고리즘의 구현, 인쇄 p.530 / PDF p.530)
U17-029	거듭 제곱의 분할: $C^n = C^{(n/2)} \cdot C^{(n/2)}$ (n은 짝수) $C^n = C^{((n-1)/2)} \cdot C^{((n-1)/2)} \cdot C$ (n은 홀수); 원문 두 경우의 재귀 방정식. 근거 대조 문구: n은 짝수	(B 12.3.1 거듭 제곱 계산법, 인쇄 p.534 / PDF p.534)
U17-030	피보나치 수의 정의: $F_n = 0$ (n=0), 1 (n=1), $F_{n-1}+F_{n-2}$ (n>1) 근거 대조 문구: 재귀 방정식	(B 12.4 분할 정복 기반 피보나치 수 구하기, 인쇄 p.537 / PDF p.537)
U17-031	피보나치 행렬: $[[F_{n+1}, F_n], [F_n, F_{n-1}]] = [[1, 1], [1, 0]]^n$ 근거 대조 문구: 이 정사각 행렬을 n번 제곱	(B 12.4.2 분할 정복으로 피보나치 수를 구하는 방법, 인쇄 p.539 / PDF p.539)
U17-032	반복 거듭 제곱 근거 대조 문구: 반복 거듭 제곱	(B 12.3.1, 인쇄 p.533 / PDF p.533)
U17-033	분할 정복 거듭 제곱 근거 대조 문구: 분할 정복 거듭 제곱	(B 12.3.1, 인쇄 p.534 / PDF p.534)
U17-034	직접 재귀 피보나치 근거 대조 문구: 직접 재귀 피보나치	(B 12.4.1, 인쇄 p.539 / PDF p.539)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-035	분할 정복 피보나치 근거 대조 문구: 분할 정복 피보나치	(B 12.4.1, 인쇄 p.539 / PDF p.539)
U17-036	분할(Divide), 정복(Conquer), 결합(Combine)의 순서이다. 근거 대조 문구: 분할Divide	(B 12.1.2 분할 정복 알고리즘의 개념, 인쇄 p.525 / PDF p.525)

18 동적 계획법

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-001	동적 계획법은 문제가 여러 단계의 반복되는 부분 문제로 이루어졌을 때 각 단계의 부분 문제 답을 기반으로 전체 문제의 답을 구하는 방법이다. 근거 대조 문구: 각 단계에 있는 부분 문제의 답을 기반으로 전체 문제의 답	(B 13.1.2 동적 계획법의 개념, 인쇄 p.550 / PDF p.549)
U18-002	LCS 테이블의 각 요소가 부분 문제이고 가장 오른쪽 아래 모서리 요소가 전체 문제이다. 근거 대조 문구: 각 요소가 각 부분 문제	(B 13.3.2 동적 계획법 기반 LCS 알고리즘의 구현, 인쇄 p.560 / PDF p.559)
U18-003	분할 정복은 문제를 위에서 아래로 쪼개지만(Top-Down), 동적 계획법은 제일 작은 부분 문제부터 상위 문제로 풀어 올라간다(Bottom-Up). 근거 대조 문구: 제일 작은 부분 문제부터 상위에 있는 문제로 풀어 올라갑니다	(B 13.1.2 동적 계획법의 개념, 인쇄 p.550 / PDF p.549)
U18-004	분할 정복의 부분 문제는 독립적인 반면 동적 계획법의 각 단계 문제들은 이전 단계의 답에 의존한다. 근거 대조 문구: 그 이전 단계에 있는 문제들의 답에 의존	(B 13.1.2 동적 계획법의 개념, 인쇄 p.550 / PDF p.549)
U18-005	동적 계획법은 한 번 푼 부분 문제의 답을 다시 풀지 않도록 테이블 등에 저장한다. 근거 대조 문구: 테이블 등에 저장합니다.	(B 13.1.2 동적 계획법의 개념, 인쇄 p.550 / PDF p.549)
U18-006	최적 부분 구조(Optimal Substructure)는 전체 문제의 최적해가 부분 문제의 최적해로부터 만들어지는 구조이다. 근거 대조 문구: 전체 문제의 최적해가 부분 문제의 최적해로부터 만들어지는 구조	(B 13.1.2 동적 계획법의 개념, 인쇄 p.550 / PDF p.549)
U18-007	동적 계획법을 이용하려면 문제가 최적 부분 구조를 갖추어야 한다. 근거 대조 문구: 최적 부분 구조	(B 13.1.2 동적 계획법의 개념, 인쇄 p.550 / PDF p.549)
U18-008	문제를 부분 문제로 나눈다. 근거 대조 문구: 문제를 부분 문제로 나눕니다.	(B 13.1.2 동적 계획법의 개념, 인쇄 p.550 / PDF p.549)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-009	가장 작은 부분 문제부터 해를 구한 뒤 테이블에 저장한다. 근거 대조 문구: 가장 작은 부분 문제부터 해를 구한 뒤 테이블에 저장	(B 13.1.2 동적 계획법의 개념, 인쇄 p.550 / PDF p.549)
U18-010	테이블에 저장된 부분 문제의 해를 이용하여 점차 상위 부분 문제의 최적해를 구한다. 근거 대조 문구: 상위 부분 문제의 최적해를 구합니다.	(B 13.1.2 동적 계획법의 개념, 인쇄 p.550 / PDF p.549)
U18-011	피보나치 수는 선행되는 피보나치 수를 구해야 다음 수를 구할 수 있으므로 최적 부분 구조를 갖춘다. 근거 대조 문구: 최적 부분 구조를 갖추고 있다고	(B 13.2 동적 계획법 기반 피보나치 수 구하기, 인쇄 p.551 / PDF p.550)
U18-012	직접 재귀 피보나치 구현은 부분 피보나치 수를 얻기 위한 중복 계산을 수없이 수행한다. 근거 대조 문구: 중복 계산이 수없이 이루어지기 때문	(B 13.2.1 동적 계획법으로 피보나치 수를 구하는 방법, 인쇄 p.551 / PDF p.550)
U18-013	피보나치 알고리즘을 동적 계획법으로 재구성하면 $O(2^n)$의 성능을 $O(n)$으로 끌어올릴 수 있다. 근거 대조 문구: $O(2^n)$의 성능을 $O(n)$으로	(B 13.2.1 동적 계획법으로 피보나치 수를 구하는 방법, 인쇄 p.552 / PDF p.551)
U18-014	Fibonacci(0)과 Fibonacci(1)의 정의된 값을 테이블의 0번과 1번 인덱스에 넣는다. 근거 대조 문구: 테이블의 0번과 1번 인덱스에 넣어놓고	(B 13.2.1 동적 계획법으로 피보나치 수를 구하는 방법, 인쇄 p.553 / PDF p.552)
U18-015	Fibonacci(2)부터 이전 피보나치 수를 참조하여 계산하고 결과를 해당 인덱스에 저장한다. 근거 대조 문구: 이전 피보나치 수를 참조하여 계산을 반복	(B 13.2.1 동적 계획법으로 피보나치 수를 구하는 방법, 인쇄 p.553 / PDF p.552)
U18-016	완성된 테이블의 n번째 요소가 Fibonacci(n)의 결과이다. 근거 대조 문구: n번째 요소의 값이 Fibonacci(n)의 결과	(B 13.2.1 동적 계획법으로 피보나치 수를 구하는 방법, 인쇄 p.553 / PDF p.552)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-017	원문은 행렬을 이용한 O($\log_2 n$) 방식보다 좋은 성능을 얻기 위해서가 아니라 동적 계획법을 이해하기 위해 피보나치 수를 다시 다룬다. 근거 대조 문구: 동적 계획법을 이해하기 위해서	(B 13.2.1 동적 계획법으로 피보나치 수를 구하는 방법, 인쇄 p.553 / PDF p.552)
U18-018	수열(Sequence)은 어떤 물건이나 객체의 목록을 가리킨다. 근거 대조 문구: 어떤 물건이나 객체의 목록	(B 13.3 최장 공통 부분 수열, 인쇄 p.555 / PDF p.554)
U18-019	부분 수열(Subsequence)은 수열에서 일부 요소를 제거한 것이다. 근거 대조 문구: 일부 요소를 제거한 것을	(B 13.3 최장 공통 부분 수열, 인쇄 p.555 / PDF p.554)
U18-020	공통 부분 수열은 두 수열 사이에 공통적으로 존재하는 부분 수열이다. 근거 대조 문구: 두 수열 사이에 공통적으로 존재하는 부분 수열	(B 13.3 최장 공통 부분 수열, 인쇄 p.556 / PDF p.555)
U18-021	최장 공통 부분 수열(LCS)은 여러 공통 부분 수열 중 가장 긴 것이다. 근거 대조 문구: 여러 개의 공통 부분 수열 중 가장 긴 것	(B 13.3 최장 공통 부분 수열, 인쇄 p.556 / PDF p.555)
U18-022	두 문자열 중 하나의 길이가 0이면 공통 부분 수열이 존재하지 않으므로 LCS_LENGTH의 결과는 0이다. 근거 대조 문구: 결과는 0	(B 13.3.1 LCS 알고리즘, 인쇄 p.556 / PDF p.555)
U18-023	두 문자열의 마지막 요소가 같으면 그 마지막 요소를 뺀 두 문자열의 LCS 길이에 1을 더한다. 근거 대조 문구: 마지막 요소를 뺀 두 문자열끼리의 LCS 길이	(B 13.3.1 LCS 알고리즘, 인쇄 p.556 / PDF p.555)
U18-024	두 문자열 길이가 모두 0보다 크고 마지막 요소가 다르면 한쪽 마지막 요소를 제외한 두 부분 문제의 LCS 길이 중 큰 값을 취한다. 근거 대조 문구: 중에서 큰 쪽	(B 13.3.1 LCS 알고리즘, 인쇄 p.556-557 / PDF p.555-556)
U18-025	LCS 점화식에서 i=0 또는 j=0은 첫 번째 요소가 아니라 아무것도 없음을 나타낸다. 근거 대조 문구: 아무것도 없음을 나타냅니다.	(B 13.3.1 LCS 알고리즘, 인쇄 p.557 / PDF p.556)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-026	LCS 테이블 크기는 (i+1)×(j+1)이며 아무것도 없는 행과 열을 표현하기 위해 각각 1씩 더 크다. 근거 대조 문구: (i+1)×(j+1)	(B 13.3.1 LCS 알고리즘, 인쇄 p.557 / PDF p.556)
U18-027	모든 j에 대해 Table[0,j]를 0으로, 모든 i에 대해 Table[i,0]을 0으로 만든다. 근거 대조 문구: 0으로 만듭니다.	(B 13.3.2 동적 계획법 기반 LCS 알고리즘의 구현, 인쇄 p.560 / PDF p.559)
U18-028	테이블의 1번 행과 1번 열부터 시작하여 왼쪽 위에서 오른쪽 아래로 LCS 길이를 채운다. 근거 대조 문구: 1번 행(두 번째 행)과 1번 열(두 번째 열)	(B 13.3.2 동적 계획법 기반 LCS 알고리즘의 구현, 인쇄 p.561 / PDF p.560)
U18-029	X 길이가 m, Y 길이가 n일 때 동적 계획법의 LCS 길이 계산은 최악의 경우라도 O(nm)의 수행 시간이 소요된다. 근거 대조 문구: 최악의 경우라도 O(nm)	(B 13.3.2 동적 계획법 기반 LCS 알고리즘의 구현, 인쇄 p.561 / PDF p.560)
U18-030	LCS 자체는 LCS 테이블의 오른쪽 아래에서 왼쪽 위로 올라가며 요소를 추적하여 얻는다. 근거 대조 문구: 오른쪽 아래 모서리부터 시작해서 왼쪽 위 모서리	(B 13.3.2 동적 계획법 기반 LCS 알고리즘의 구현, 인쇄 p.563 / PDF p.562)
U18-031	LCS 추적은 i=0 또는 j=0이 될 때까지 반복한다. 근거 대조 문구: i=0 또는 j=0	(B 13.3.2 동적 계획법 기반 LCS 알고리즘의 구현, 인쇄 p.563 / PDF p.562)

U18-032	피보나치 테이블 계산 부분 (원문 코드 발췌)
	for (i=2; i<=N; i++)
	{
	FibonacciTable[i] = FibonacciTable[i-1] + FibonacciTable[i-2];
	}
	
	Result = FibonacciTable[N];
	근거 대조 문구: for (i=2; i<=N; i++)
		(B 13.2.2 동적 계획법 기반 피보나치 수 구하기 알고리즘의 구현, 인쇄 p.554 / PDF p.553)

U18-033	동적 계획법 기반 피보나치: O(n); 원문 직접 재귀 구현의 $O(2^n)$을 테이블 기반으로 재구성한 수행 시간. 근거 대조 문구: O(n)	(B 13.2.1 동적 계획법으로 피보나치 수를 구하는 방법, 인쇄 p.552 / PDF p.551)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-034	LCS 테이블 점화식: TABLE[i,j] = 0 (i=0 or j=0) TABLE[i,j] = TABLE[i−1,j−1]+1 (x_i=y_j) TABLE[i,j] = MAX(TABLE[i−1,j],TABLE[i,j−1]) (i,j>0 and $x_i \neq y_j$); i=0, j=0은 빈 경우이다. 문자열의 첫 요소는 각각 i=1, j=1이다. 근거 대조 문구: TABLE[i,j]	(B 13.3.1 LCS 알고리즘, 인쇄 p.557 / PDF p.556)
U18-035	분할 정복의 방향 근거 대조 문구: 분할 정복의 방향	(B 13.1.2, 인쇄 p.550 / PDF p.549)
U18-036	위에서 아래로 문제를 쪼갠다(Top-Down). 근거 대조 문구: 위에서부터 아래로 쪼개지만	(B 13.1.2, 인쇄 p.550 / PDF p.549)
U18-037	동적 계획법의 방향 근거 대조 문구: 동적 계획법의 방향	(B 13.1.2, 인쇄 p.550 / PDF p.549)
U18-038	제일 작은 부분 문제부터 상위 문제로 풀어 올라간다(Bottom-Up). 근거 대조 문구: 제일 작은 부분 문제부터	(B 13.1.2, 인쇄 p.550 / PDF p.549)
U18-039	부분 문제 근거 대조 문구: 부분 문제	(B 13.1.2, 인쇄 p.550 / PDF p.549)
U18-040	답의 저장 근거 대조 문구: 답의 저장	(B 13.1.2, 인쇄 p.550 / PDF p.549)

19 탐욕 알고리즘

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-001	탐욕 알고리즘은 각 단계의 부분 문제를 풀 때 근시안적으로 최적해를 구한다. 근거 대조 문구: 근시안적으로 최적해를 구한다고	(B 14.1 탐욕 알고리즘의 개요, 인쇄 p.573 / PDF p.571)
U19-002	압축은 문자열의 각 요소를 차례대로 읽으면서 허프만 트리의 해당 문자 접두어 코드로 변환한다. 근거 대조 문구: 해당 문자의 접두어 코드로 변환	(B 14.5.3 데이터 압축, 인쇄 p.591 / PDF p.589)
U19-003	탐욕 알고리즘은 반드시 최적해를 구해준다는 보장을 하지 못한다. 근거 대조 문구: 반드시 최적해를 구해준다는 보장은 하지 못합니다.	(B 14.1 탐욕 알고리즘의 개요, 인쇄 p.573 / PDF p.571)
U19-004	탐욕 알고리즘으로 풀 수 있는 문제도 최적 부분 구조를 갖고 있어야 한다. 근거 대조 문구: 최적 부분 구조를 갖고 있어야	(B 14.1 탐욕 알고리즘의 개요, 인쇄 p.573 / PDF p.571)
U19-005	해 선택은 현재 상태에서 부분 문제의 최적해를 구하여 부분해 집합에 추가하는 단계이다. 근거 대조 문구: 부분해 집합Solution Set	(B 14.1 탐욕 알고리즘의 개요, 인쇄 p.573 / PDF p.571)
U19-006	실행 가능성 검사는 새 부분해 집합이 문제의 조건을 위반하지 않는지 검사하는 단계이다. 근거 대조 문구: 문제의 조건을 위반하지 않는지를 검사	(B 14.1 탐욕 알고리즘의 개요, 인쇄 p.573 / PDF p.571)
U19-007	해 검사는 새 부분해 집합이 문제의 해인지 확인하고 완성되지 않았으면 해 선택부터 다시 시작하는 단계이다. 근거 대조 문구: 해 선택부터 다시 시작	(B 14.1 탐욕 알고리즘의 개요, 인쇄 p.573 / PDF p.571)
U19-008	거스름돈 줄이기 문제는 거스름돈으로 주는 지폐와 동전의 개수를 최소한으로 줄이는 문제이다. 근거 대조 문구: 지폐와 동전의 개수를 최소한으로	(B 14.2 거스름돈 줄이기 문제, 인쇄 p.574 / PDF p.572)
U19-009	해 선택에서 현재 고를 수 있는 가장 큰 단위의 동전 하나를 거스름돈에 추가한다. 근거 대조 문구: 가장 큰 단위의 동전을 하나 골라	(B 14.2 거스름돈 줄이기 문제, 인쇄 p.574 / PDF p.572)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-010	거스름돈이 내줄 액수를 초과하면 마지막 동전을 빼고 한 단계 작은 단위의 동전을 추가한다. 근거 대조 문구: 한 단계 작은 단위의 동전을 추가	(B 14.2 거스름돈 줄이기 문제, 인쇄 p.574 / PDF p.572)
U19-011	동전 총액이 내줄 액수와 일치하는지 검사하며 모자라면 다시 추가할 동전을 고른다. 근거 대조 문구: 거스름돈 액수가 모자라면	(B 14.2 거스름돈 줄이기 문제, 인쇄 p.574 / PDF p.572)
U19-012	원문의 400원 동전을 추가한 사례에서는 800원을 400원짜리 2개로 줄 수 있지만 탐욕 알고리즘은 500원짜리 1개와 100원짜리 3개를 고른다. 근거 대조 문구: 400원짜리 동전 2개	(B 14.2.2 탐욕 알고리즘의 중요한 속성, 인쇄 p.578-579 / PDF p.576-577)
U19-013	거스름돈 알고리즘의 해가 항상 최적은 아니라는 점은 탐욕 알고리즘의 중요한 속성이다. 근거 대조 문구: 해가 항상 최적은 아니다	(B 14.2.2 탐욕 알고리즘의 중요한 속성, 인쇄 p.579 / PDF p.577)
U19-014	최소 신장 트리는 그래프의 모든 정점을 최소 비용으로 연결하는 트리이며 사이클이 형성되어서는 안 된다. 근거 대조 문구: 모든 정점을 최소 비용으로 연결	(B 14.3 크루스칼 알고리즘 다시 보기, 인쇄 p.579 / PDF p.577)
U19-015	크루스칼 알고리즘은 모든 간선을 가중치 오름차순으로 정렬한 목록을 만든다. 근거 대조 문구: 가중치의 오름차순으로 정렬	(B 14.3 크루스칼 알고리즘 다시 보기, 인쇄 p.580 / PDF p.578)
U19-016	정렬한 간선 목록을 순회하면서 사이클을 형성하지 않는 간선을 최소 신장 트리에 추가한다. 근거 대조 문구: 사이클이 형성되면 안 됩니다.	(B 14.3 크루스칼 알고리즘 다시 보기, 인쇄 p.580 / PDF p.578)
U19-017	크루스칼의 해 선택은 가장 작은 가중치의 간선을 선택하는 일이다. 근거 대조 문구: 가장 작은 가중치의 간선을 선택	(B 14.3 크루스칼 알고리즘 다시 보기, 인쇄 p.580 / PDF p.578)
U19-018	크루스칼은 분리 집합으로 사이클을 탐지하며 간선 양끝 정점이 이미 같은 집합이면 그 간선이 사이클을 형성한다고 판단한다. 근거 대조 문구: 사이클 탐지를 위해 분리 집합	(B 14.3 크루스칼 알고리즘 다시 보기, 인쇄 p.580 / PDF p.578)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-019	크루스칼은 모든 정점이 하나의 집합에 들어 있으면 해가 완성되었다고 판단할 수 있다. 근거 대조 문구: 모든 정점이 하나의 집합	(B 14.3 크루스칼 알고리즘 다시 보기, 인쇄 p.582 / PDF p.580)
U19-020	데이크스트라 알고리즘은 그래프의 한 정점에서 다른 정점으로 향하는 가장 짧은 경로를 구한다. 근거 대조 문구: 한 정점에서 다른 정점으로 향하는 가장 짧은 경로	(B 14.4 데이크스트라 알고리즘 다시 보기, 인쇄 p.582 / PDF p.580)
U19-021	원문의 탐욕 관점에서는 최단 경로가 해 집합이고 각 정점이 부분 문제에 대한 해이다. 근거 대조 문구: 최단 경로가 해 집합이고 각 정점이 부분 문제에 대한 해	(B 14.4 데이크스트라 알고리즘 다시 보기, 인쇄 p.583 / PDF p.581)
U19-022	데이크스트라 알고리즘은 한 정점의 인접 정점들에 대한 정보를 이용하여 최단 경로를 구축한다. 근거 대조 문구: 한 정점의 인접 정점들에 대한 정보	(B 14.4 데이크스트라 알고리즘 다시 보기, 인쇄 p.585 / PDF p.583)
U19-023	고정 길이 코드는 모든 코드 길이가 똑같은 값을 갖는 코드 체계이다. 근거 대조 문구: 모든 코드의 길이가 똑같은 값	(B 14.5.1 고정 길이 코드와 접두어 코드, 인쇄 p.585 / PDF p.583)
U19-024	가변 길이 코드는 저장 공간을 절약하지만 데이터 처리가 번거롭다는 단점이 있다. 근거 대조 문구: 저장 공간을 절약한다는 장점	(B 14.5.1 고정 길이 코드와 접두어 코드, 인쇄 p.586 / PDF p.584)
U19-025	접두어 코드는 코드 집합의 어느 코드도 다른 코드의 접두어가 되지 않는 코드이다. 근거 대조 문구: 어느 코드도 다른 코드의 접두어가 되지 않는	(B 14.5.1 고정 길이 코드와 접두어 코드, 인쇄 p.586 / PDF p.584)
U19-026	접두어 코드는 무접두어 코드(Prefix Free Code)라고도 불린다. 근거 대조 문구: 무접두어 코드	(B 14.5.1 고정 길이 코드와 접두어 코드, 인쇄 p.586 / PDF p.584)
U19-027	기호의 빈도는 한 기호가 데이터 안에서 차지하는 비율을 말한다. 근거 대조 문구: 한 기호가 데이터 안에서 차지하는 비율	(B 14.5.2 허프만 트리 구축, 인쇄 p.587 / PDF p.585)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-028	빈도가 높은 기호에 짧은 접두어 코드를 부여하면 저장 공간을 아낄 수 있다. 근거 대조 문구: 빈도가 높은 기호에 작은 접두어 코드	(B 14.5.2 허프만 트리 구축, 인쇄 p.587 / PDF p.585)
U19-029	허프만 트리에서 왼쪽 자식은 0, 오른쪽 자식은 1을 나타내며 모든 기호는 잎 노드에만 기록된다. 근거 대조 문구: 모든 기호는 잎 노드에만 기록	(B 14.5.2 허프만 트리 구축, 인쇄 p.587 / PDF p.585)
U19-030	허프만 트리는 뿌리에서 잎에 이르는 경로로 기호의 접두어 코드를 표현하는 이진 트리이다. 근거 대조 문구: 접두어 코드를 표현하는 이진 트리	(B 14.5.2 허프만 트리 구축, 인쇄 p.588 / PDF p.586)
U19-031	허프만 트리 구성에서는 현재 빈도가 가장 작은 노드 2개를 선택한다. 근거 대조 문구: 빈도가 가장 작은 노드 2개	(B 14.5.2 허프만 트리 구축, 인쇄 p.589 / PDF p.587)
U19-032	선택한 두 노드의 부모를 새로 만들고 양쪽 자식으로 연결하며 부모 빈도를 두 자식 빈도의 합으로 둔다. 근거 대조 문구: 부모 노드의 빈도는 자식 노드 빈도의 합	(B 14.5.2 허프만 트리 구축, 인쇄 p.589 / PDF p.587)
U19-033	새 부모를 노드 집합에 추가하는 과정을 반복하여 노드가 하나만 남으면 그 노드가 허프만 트리가 된다. 근거 대조 문구: 단 하나의 노드만 남아	(B 14.5.2 허프만 트리 구축, 인쇄 p.589-590 / PDF p.587-588)
U19-034	원문의 aaabaacdd 사례는 ASCII라고 가정한 72비트 데이터를 000100001011111의 15비트로 압축한다. 근거 대조 문구: 72비트였는데 15비트로 압축	(B 14.5.3 데이터 압축, 인쇄 p.591 / PDF p.589)
U19-035	압축 해제는 압축에 사용한 허프만 트리와 결과 버퍼를 준비하고 뿌리 노드부터 시작한다. 근거 대조 문구: 허프만 트리와 압축 해제된 데이터가 담길 버퍼	(B 14.5.4 데이터 압축 해제, 인쇄 p.591 / PDF p.589)
U19-036	압축 데이터를 한 비트 읽어 0이면 왼쪽 자식, 1이면 오른쪽 자식으로 이동한다. 근거 대조 문구: 0이면 현재 노드의 왼쪽 자식 노드	(B 14.5.4 데이터 압축 해제, 인쇄 p.591 / PDF p.589)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-037	잎 노드를 만나면 그 기호를 버퍼에 추가하고 다시 뿌리 노드로 이동한다. 근거 대조 문구: 압축 해제 버퍼에 추가하고 뿌리 노드로 돌아	(B 14.5.4 데이터 압축 해제, 인쇄 p.592 / PDF p.590)

U19-038	GetChange() 핵심 반복 부분 (원문 코드 발췌)
	int ChangeAmount = Pay - Price;
	
	for (i=0; i<Size; i++)
	{
	Change[i] = CountCoins(ChangeAmount, CoinUnits[i]);
	ChangeAmount = ChangeAmount - (CoinUnits[i] * Change[i]);
	}
	근거 대조 문구: int ChangeAmount = Pay - Price;
		(B 14.2.1 거스름돈 계산 예제 프로그램, 인쇄 p.575 / PDF p.573)

U19-039	거스름돈 근거 대조 문구: 거스름돈	(B 14.2, 인쇄 p.574 / PDF p.572)
U19-040	크루스칼 근거 대조 문구: 크루스칼	(B 14.3, 인쇄 p.580 / PDF p.578)
U19-041	데이크스트라 근거 대조 문구: 데이크스트라	(B 14.4, 인쇄 p.585 / PDF p.583)
U19-042	허프만 트리 근거 대조 문구: 허프만 트리	(B 14.5.2, 인쇄 p.589 / PDF p.587)

20 백트래킹

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U20-001	백트래킹은 해가 될 후보를 찾고 해의 조건을 충족하지 못하는 후보를 제거하면서 최종 해를 찾는 기법이다. 근거 대조 문구: 해가 될 수 있는 조건을 충족하지 못하는 후보를 제거	(B 15.1 백트래킹의 개요, 인쇄 p.607 / PDF p.605)
U20-002	N개의 퀸 해를 담는 데 N개 요소의 배열 하나면 충분하며 인덱스는 행 번호, 값은 열 번호로 사용한다. 근거 대조 문구: 인덱스는 행 번호, 각 요소에 담긴 값은 열 번호	(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.629 / PDF p.627)
U20-003	백트래킹은 여러 후보해 중 특정 조건을 충족시키는 모든 해를 찾는 알고리즘이다. 근거 대조 문구: 특정 조건을 충족시키는 모든 해	(B 15.1.2 백트래킹의 개념, 인쇄 p.608 / PDF p.606)
U20-004	후보해는 해가 될 가능성을 가진 부분해의 조합이다. 근거 대조 문구: 해가 될 수 있는 가능성을 가진 부분해의 조합	(B 15.1.2 백트래킹의 개념, 인쇄 p.609 / PDF p.607)
U20-005	백트래킹이 다루는 문제는 대부분 후보해의 수가 굉장히 많다. 근거 대조 문구: 후보해의 수가 굉장히 많다는 특징	(B 15.1.2 백트래킹의 개념, 인쇄 p.609 / PDF p.607)
U20-006	백트래킹은 해를 찾는 비용을 줄이기 위해 방문할 노드 수를 최소화하는 것이 중요하다. 근거 대조 문구: 방문할 노드의 수를 최소화	(B 15.1.2 백트래킹의 개념, 인쇄 p.611 / PDF p.609)
U20-007	깊이 우선 탐색의 목적은 모든 노드 방문이고 백트래킹의 목적은 해를 찾는 것이므로 모든 노드를 방문할 필요는 없다. 근거 대조 문구: 백트래킹은 해를 찾는 것이 목적	(B 15.1.2 백트래킹의 개념, 인쇄 p.611 / PDF p.609)
U20-008	해를 찾는 과정은 뿌리에서 출발하며 뿌리도 하나의 부분해이다. 근거 대조 문구: 뿌리도 하나의 부분해	(B 15.1.2 백트래킹의 개념, 인쇄 p.612 / PDF p.610)
U20-009	현재 부분해에서 선택 가능한 다음 부분해 목록을 얻고 각 부분해를 하나씩 방문한다. 근거 대조 문구: 부분해들을 하나씩 방문	(B 15.1.2 백트래킹의 개념, 인쇄 p.612 / PDF p.610)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U20-010	방문한 부분해가 해의 조건을 만족하면 그 자리에서 다음 부분해들을 시도하고 그렇지 않으면 이전 부분해로 돌아와 다른 부분해를 시도한다. 근거 대조 문구: 이전 부분해로 돌아 나와 다른 부분해	(B 15.1.2 백트래킹의 개념, 인쇄 p.612 / PDF p.610)
U20-011	최종해를 얻거나 모든 경우를 확인해도 해가 없음을 알 때까지 반복한다. 근거 대조 문구: 모든 경우의 수를 확인해도 해가 없음을 확인	(B 15.1.2 백트래킹의 개념, 인쇄 p.612 / PDF p.610)
U20-012	미로를 나타내는 트리는 논리적 개념이므로 복잡한 트리 자료구조 없이도 미로 탈출 알고리즘을 구현할 수 있다. 근거 대조 문구: 논리적인 개념이지 물리적인 존재가 아니라는	(B 15.2.1 재귀 호출 기반 백트래킹, 인쇄 p.612 / PDF p.610)
U20-013	부분해 계산, 다음 부분 후보해 목록 확인, 각 부분 후보해 계산의 반복을 재귀 호출로 구현할 수 있다. 근거 대조 문구: 부분해 계산 → 다음 부분 후보해 목록 확인	(B 15.2.1 재귀 호출 기반 백트래킹, 인쇄 p.613 / PDF p.611)
U20-014	백트래킹은 현재 후보가 해의 조건을 만족하지 못할 때와 최종해를 확보한 경우에 처리한다. 근거 대조 문구: 조건을 만족하지 못했을 때와 최종해를 확보한 경우	(B 15.2.1 재귀 호출 기반 백트래킹, 인쇄 p.613 / PDF p.611)
U20-015	재귀 함수에서 백트래킹 조건을 만나 함수를 반환하면 상위 부분해의 함수로 제어가 돌아간다. 근거 대조 문구: 함수를 반환하는 것만으로 백트래킹	(B 15.2.1 재귀 호출 기반 백트래킹, 인쇄 p.614 / PDF p.612)
U20-016	MazeInfo는 너비 ColumnSize, 높이 RowSize, 동적으로 할당한 2차원 배열을 담는 char** Data로 미로를 표현한다. 근거 대조 문구: MazeInfo	(B 15.2.2 미로 탈출 알고리즘의 구현, 인쇄 p.614 / PDF p.612)
U20-017	Position 구조체는 수평 위치 X와 수직 위치 Y를 갖는다. 근거 대조 문구: 수평 위치를 나타내는 X와 수직 위치를 나타내는 Y	(B 15.2.2 미로 탈출 알고리즘의 구현, 인쇄 p.615 / PDF p.613)
U20-018	시작점 S를 현재 위치로 정하고 이동 방향을 북으로 설정한다. 근거 대조 문구: 이동 방향을 북으로 설정	(B 15.2.2 미로 탈출 알고리즘의 구현, 인쇄 p.616 / PDF p.614)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U20-019	현재 위치에서 가려는 방향의 이동 가능 여부를 확인하며 벽과 지나온 길은 이동 가능한 길이 아니다. 근거 대조 문구: 벽과 지나왔던 길은 이동 가능한 길이 아닙니다.	(B 15.2.2 미로 탈출 알고리즘의 구현, 인쇄 p.616 / PDF p.614)
U20-020	이동이 불가능하면 북→남→동→서 순으로 방향을 바꾸어 시도하고 어느 방향도 가능하지 않으면 이전 위치로 돌아간다. 근거 대조 문구: 북→남→동→서 순	(B 15.2.2 미로 탈출 알고리즘의 구현, 인쇄 p.616 / PDF p.614)
U20-021	출구를 찾거나 미로의 모든 길을 방문할 때까지 이동과 확인을 반복한다. 근거 대조 문구: 출구를 찾거나 미로 내의 모든 길	(B 15.2.2 미로 탈출 알고리즘의 구현, 인쇄 p.616 / PDF p.614)
U20-022	Solve()는 시작점을 찾고 MoveTo()를 호출하며 MoveTo()는 GetNextStep()으로 다음 위치를 얻어 재귀 호출한다. 근거 대조 문구: GetNextStep()	(B 15.2.2 미로 탈출 알고리즘의 구현, 인쇄 p.616 / PDF p.614)
U20-023	원문 MoveTo()는 현 위치를 MARKED로 표시하고 모든 방향이 실패하면 WAY로 되돌린 뒤 FAIL을 반환한다. 근거 대조 문구: Maze->Data[Current->Y][Current->X] = WAY;	(B 15.2.3 미로 탈출 알고리즘 예제 프로그램, 인쇄 p.619-620 / PDF p.617-618)
U20-024	8개의 퀸 문제는 8×8 체스판에서 퀸 8개를 서로 공격할 수 없도록 배치하는 모든 경우를 구하는 문제이다. 근거 대조 문구: 서로 공격할 수 없도록 배치하는 모든 경우	(B 15.3 8개의 퀸 문제, 인쇄 p.624 / PDF p.622)
U20-025	원문은 8개의 퀸 문제에서 44억 개 이상 후보해 중 실제 해 92개를 모두 찾아야 한다고 설명한다. 근거 대조 문구: 실제 해 92개를 모두 찾아내라고	(B 15.3 8개의 퀸 문제, 인쇄 p.625 / PDF p.623)
U20-026	원문은 4×4 이상의 다른 크기로 문제를 축소·확장할 수 있어 N개의 퀸 문제라고도 부른다. 근거 대조 문구: N개의 퀸 문제	(B 15.3 8개의 퀸 문제, 인쇄 p.625 / PDF p.623)
U20-027	원문의 4×4 사례에서 [1,2]–[2,4]–[3,1]–[4,3]으로 이어지는 첫 번째 해를 얻는다. 근거 대조 문구: [1,2]-[2,4]-[3,1]-[4,3]	(B 15.3.1 8개의 퀸이 만드는 해공간과 백트래킹, 인쇄 p.628 / PDF p.626)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U20-028	FindSolutionForQueen()은 부분해 Columns, 새 퀸의 행 Row, 체스판 크기 NumberOfQueens, 해 개수를 세는 SolutionCount를 매개 변수로 받는다. 근거 대조 문구: 모두 4개의 매개 변수	(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.630 / PDF p.628)
U20-029	새 퀸이 기존 퀸을 위협하면 함수를 반환하여 이전 부분해로 백트래킹한다. 근거 대조 문구: 함수를 반환하여 백트래킹	(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.631 / PDF p.629)
U20-030	위협 검사를 통과한 마지막 부분해가 Row==NumberOfQueens−1이면 해가 완성되어 출력한다. 근거 대조 문구: Row == NumberOfQueens−1	(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.631 / PDF p.629)
U20-031	IsThreatened()는 이전 퀸과 새 퀸의 열이 같으면 수직 위협으로 판단한다. 근거 대조 문구: 열과 같은 것이 있는지 확인	(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.632 / PDF p.630)
U20-032	두 퀸의 행 차이 절댓값과 열 차이 절댓값이 같으면 대각선으로 상호 위협한다. 근거 대조 문구: 대각선으로 상호 위협	(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.632 / PDF p.630)
U20-033	FindSolutionForQueen()이 새 행을 매개 변수로 실행하므로 원문 IsThreatened()에서는 수평 위협을 검사 대상에서 제외한다. 근거 대조 문구: 수평 위협은 고려 대상에서 제외	(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.632 / PDF p.630)
U20-034	해를 더 찾아야 하면 FindSolutionForQueen()을 재귀 호출하여 다음 행의 후보를 시도한다. 근거 대조 문구: 다시 FindSolutionForQueen() 함수를 재귀 호출	(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.631 / PDF p.629)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U20-035	FindSolutionForQueen()의 위협 검사·해 출력 부분 (원문 코드 발췌)
	
	if (IsThreatened(Columns, Row))
	return;
	
	if (Row == NumberOfQueens-1)
	{
	printf("Solution #%d : \n", ++(*SolutionCount));
	PrintSolution(Columns, NumberOfQueens);
	}
	근거 대조 문구: if (IsThreatened(Columns, Row))
		(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.631 / PDF p.629)

U20-036	8개의 퀸의 원문 후보해 수: $_{64}C_8$ = 64! / (8!(64−8)!); 체스판 64칸 중 8칸을 골라 퀸을 놓는 원문의 후보해 계산. 근거 대조 문구: 후보해의 수는 총	(B 15.3 8개의 퀸 문제, 인쇄 p.625 / PDF p.623)
U20-037	대각선 위협 조건: |행1−행2| = |열1−열2|; 이 등식을 만족하면 두 퀸이 대각선으로 상호 위협한다. 근거 대조 문구: |행1−행2|=|열1−열2|	(B 15.3.2 N개의 퀸 문제 풀이 알고리즘의 구현, 인쇄 p.632 / PDF p.630)
U20-038	깊이 우선 탐색·백트래킹 근거 대조 문구: 깊이 우선 탐색·백트래킹	(B 15.1.2, 인쇄 p.611 / PDF p.609)
U20-039	미로 근거 대조 문구: 미로	(B 15.2.2, 인쇄 p.616 / PDF p.614)
U20-040	N개의 퀸의 저장 근거 대조 문구: N개의 퀸의 저장	(B 15.3.2, 인쇄 p.629 / PDF p.627)
U20-041	재귀 호출의 되돌아가기 근거 대조 문구: 재귀 호출의 되돌아가기	(B 15.2.1, 인쇄 p.614 / PDF p.612)